

A

React & Next Ultimate Course



(In Hindi)

A React & Next Ultimate Course

(In Hindi)

Slides For Theory
Lectures

Please Go Through Them, As They
Are Very Important For Learning

WELCOME!

A React & Next Ultimate Course

(In Hindi)

This Section

Welcome

Must Watch Before Start!

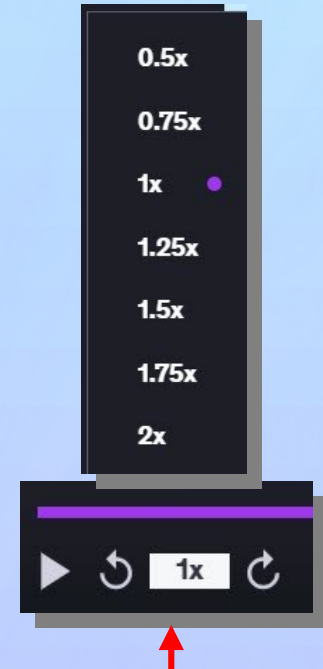
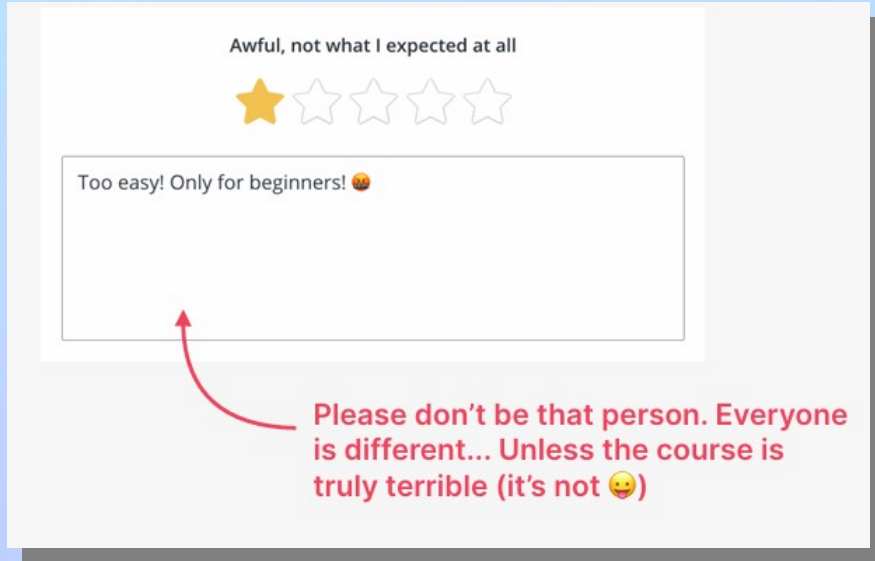
Some Quick **Considerations** Before We Start..



This course is for everyone! So please don't write a bad review right away if the course is too easy, or too hard, or progressing too slow, or too fast for you



To make the course perfect for **YOU: re watch** lectures, jump to **other sections**, watch the course with **slower** or **faster** speed, or ask **questions**



Some Quick Considerations Before We Start..



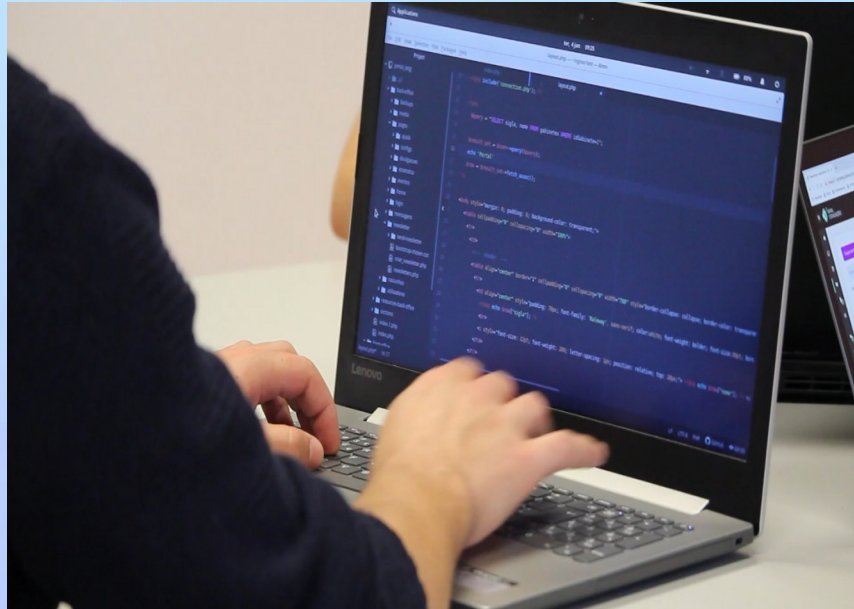
You don't need to watch the entire course in order to learn React! If you are in a hurry, can cut the course length in **HALF** by skipping optional sections and practice Levels

Section 1: Welcome	0 / 2 21min	▼
Section 2: Beginner Level - React Fundamentals[4Project]	0 / 2 1min	▼
Section 3: A Look at React	0 / 9 1hr 41min	▼
Section 4: Optional ES6 Essential JavaScript for React	0 / 15 2hr 17min	▼
Section 5: Start Working with Components Props, JSX	0 / 22 3hr 30min	▼
Section 6: State, Events and Forms Interactive Components	0 / 21 4hr 3min	▼
Section 7: Thinking In React State Management	0 / 17 3hr 48min	▼
Section 8: Optional Practice Project Eat & Split	0 / 9 1hr 49min	▼
Section 9: Intermediate Level [2 Projects]	0 / 2 1min	▼

Some Quick **Considerations** Before We Start..



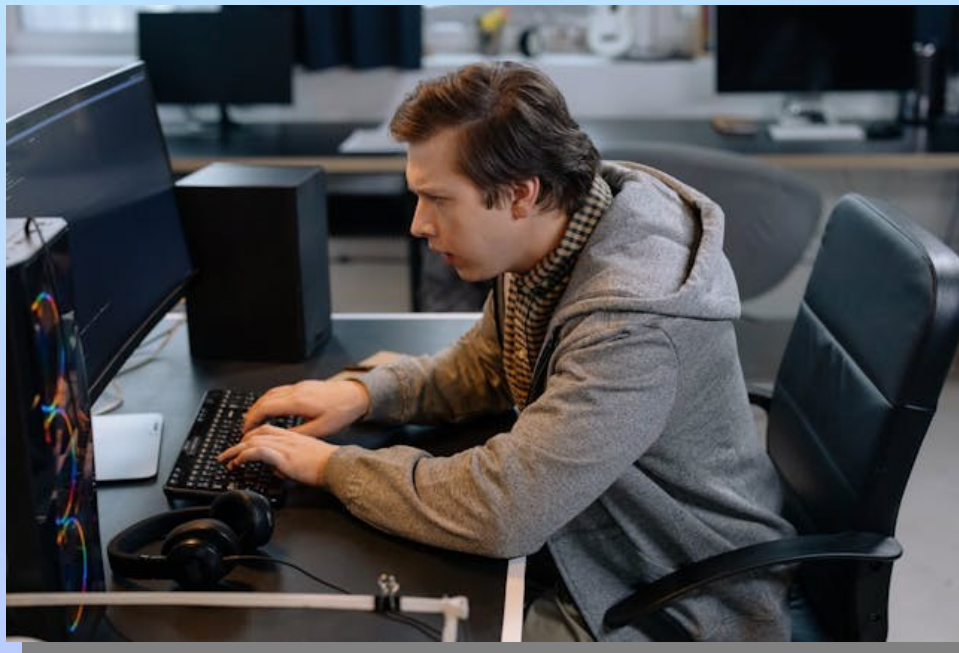
You need to code along with me! You will learn **ZERO** React skills by just sitting and watching me code. You really have to write code **YOURSELF!**



Some Quick **Considerations** Before We Start..



In the first sections of the course, don't worry about **WHY** and **HOW** things work, or about React **"best practices"**. While learning, we just want to make things **WORK**. We will worry about everything else **later in the course**



Some Quick **Considerations** Before We Start..



If you have a problem or a question, **start by trying to solve it yourself! This is essential for your progress.** If you can't solve it, check the **Q&A** section. If that doesn't help, you can **ask a new question.** Use a short description, and post code on codesandbox.io



The screenshot shows the CodeSandbox Q&A interface. A red arrow labeled '1' points to the 'Q&A' tab in the top navigation bar. Another red arrow labeled '2' points to the search input field. Below the search bar are filters for 'All lectures' and 'Sort by recommended', along with a 'Filter questions' button. At the bottom, it says 'All questions in this course (135)'.

1

2

Search all course questions

Filters: Sort by:

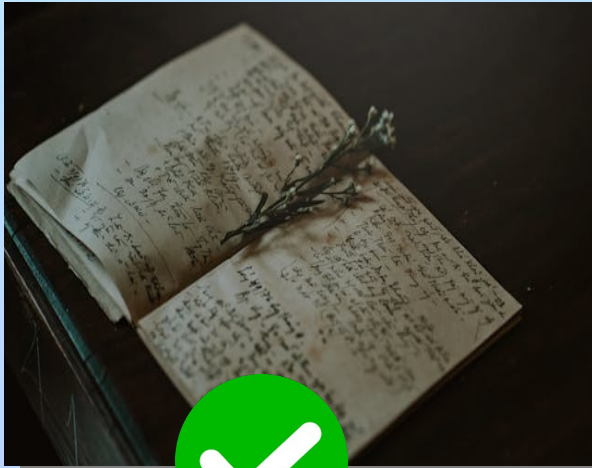
All lectures Sort by recommended Filter questions

All questions in this course (135)

Some Quick **Considerations** Before We Start..

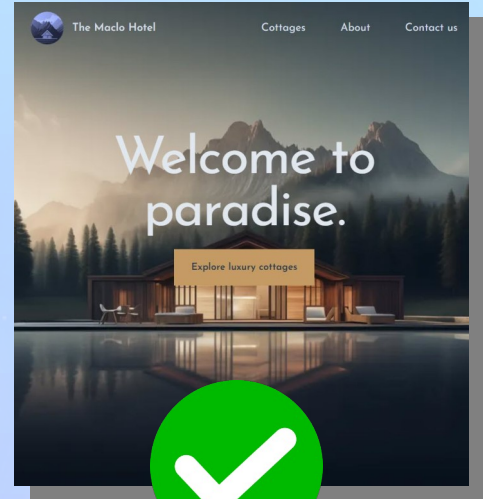


Before moving on from a section, make sure that you understand exactly what was covered. Take a break, review the code we wrote, review your notes, review the projects we built, and maybe even write some code yourself



```
import { NavLink } from "react-router-dom";
import styles from "../AppNav.module.css";

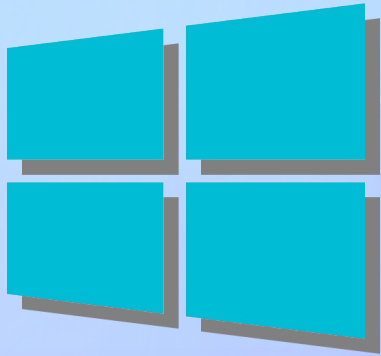
Complexity is 8 It's time to do something... | Tabnine | Edit | ...
function AppNav() {
  return (
    <nav className={styles.nav}>
      <ul>
        <li>
          <NavLink to="cities">Cities</NavLink>
        </li>
        <li>
          <NavLink to="countries">Countries</li>
        </li>
      </ul>
    </nav>
  );
}
```



Some Quick **Considerations** Before We Start..



I recorded this course on a Window, but everything works the exact same way on Mac or Linux. If something doesn't work on your computer, it's NOT because you're using a different OS



Some Quick **Considerations** Before We Start..



Most importantly, have fun! It's so rewarding to see an app come to life that **YOU** have built **YOURSELF!** So if you're feeling frustrated, stop whatever you're doing, and come back later!



A real fun ! Believe me

Level 1



React Fundamentals

A Look At React

A React & Next Ultimate Course

(In Hindi)

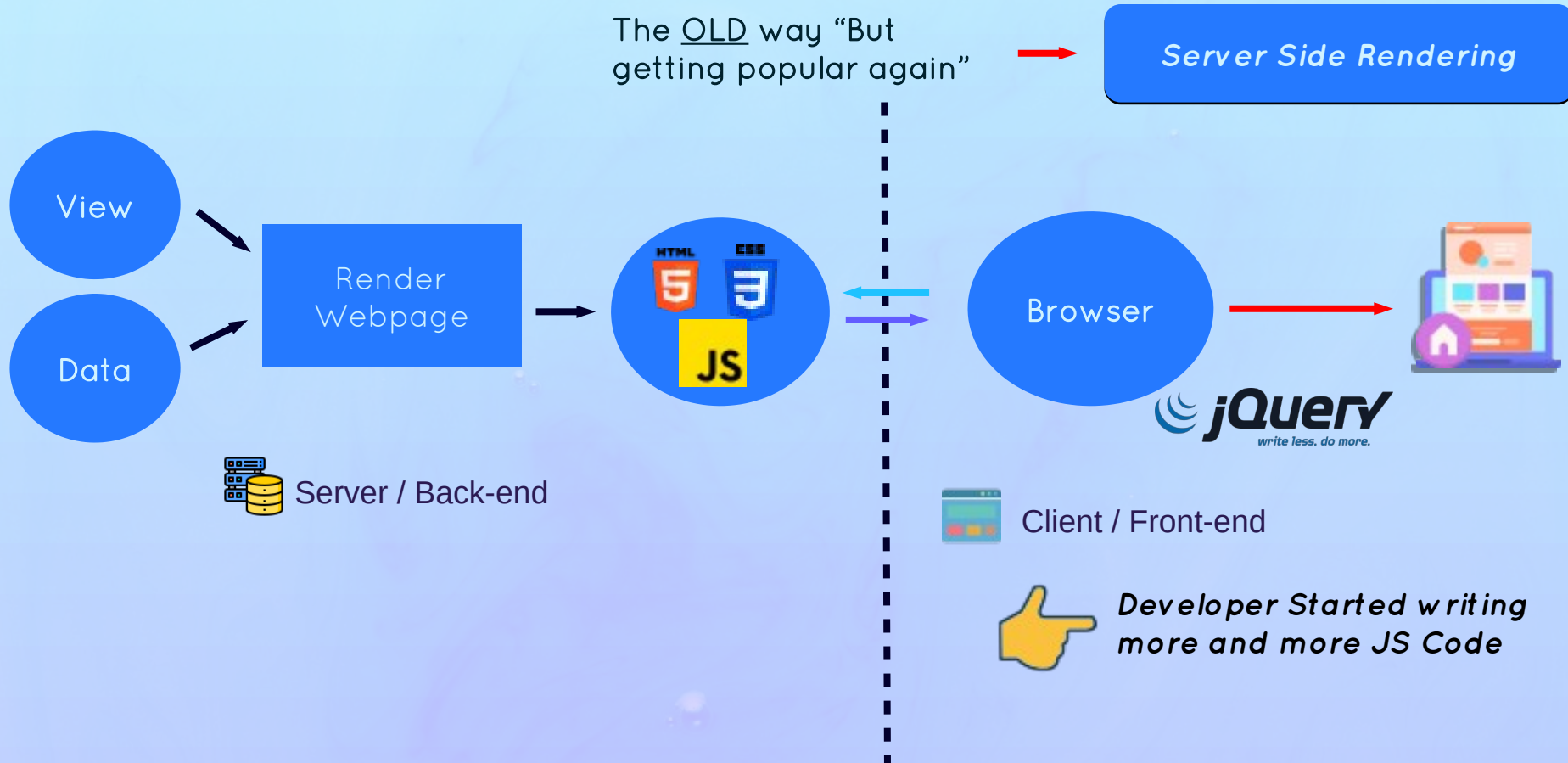
Section

A Look At React

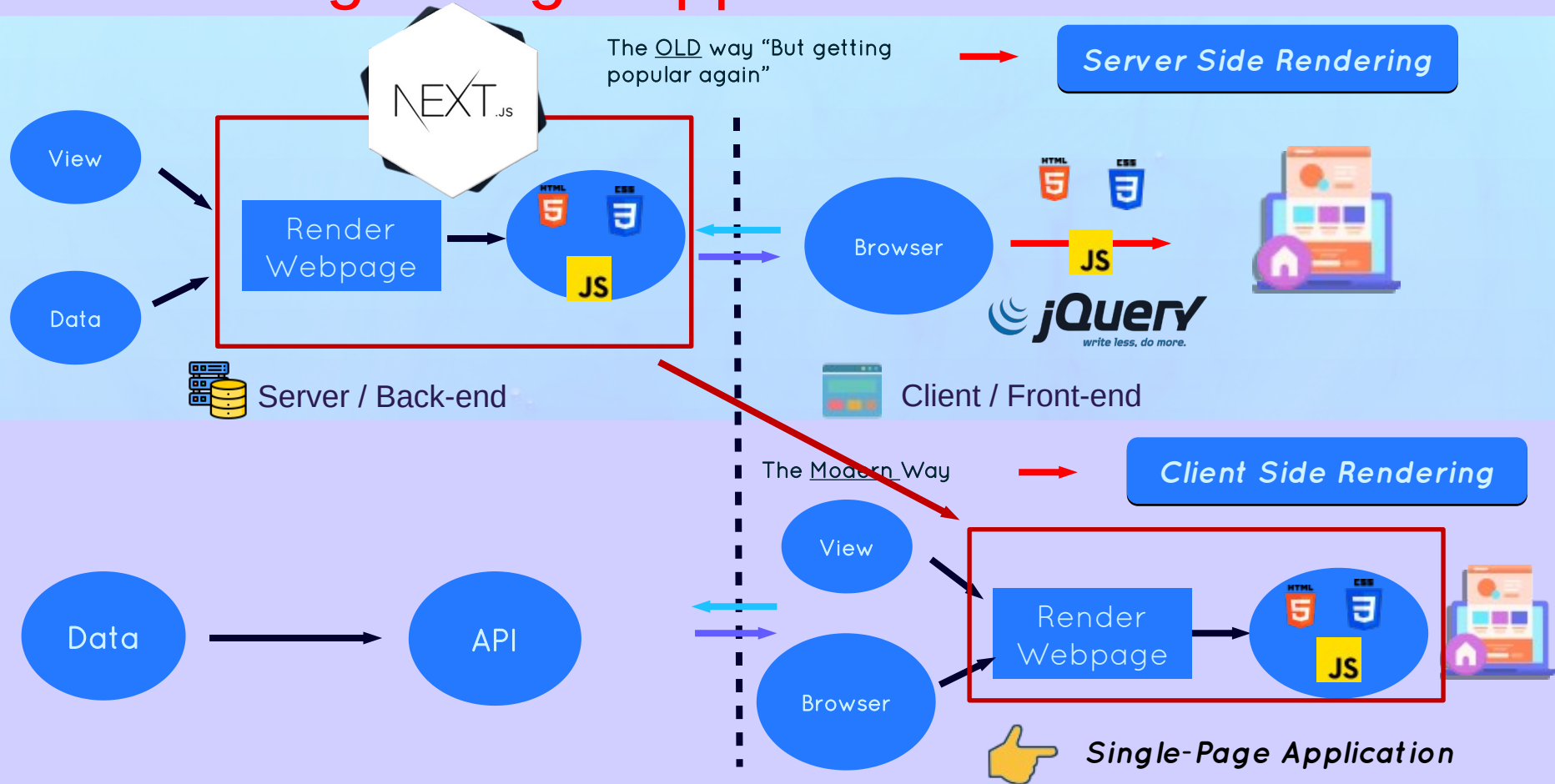
Lecture

Why Do We Need
Front-end Framework?
Or Framework Exists

Rise of Single Page Applications



Rise of Single Page Applications



Single-Page App with JavaScript?



Front-end Application are all about

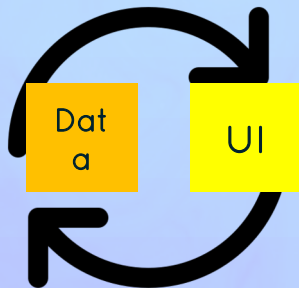
Handling Data + Displaying
data in user Interface



**User interface needs to
stay sync with data**



**Very Hard Problem to
Solve!**



Problems With



- 1 Requires lots of direct **DOM Manipulation** and **traversing**



“Spaghetti Code”

- 2 Data state usually **stored in DOM**, shared across entire app



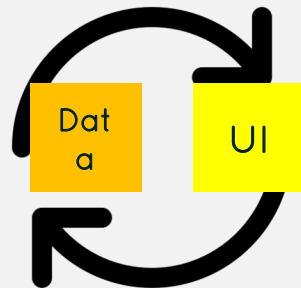
“Hard to reason + bugs”

Why do Front-end Frameworks Exist?

1

JavaScript front-end frameworks exist because...

Keeping User Interface in Sync
With Data is **Really Hard** and a
Lot of Work



Front-end frameworks **solve this problem** and take hard work away from **developers**



2

They enforce a “**Correct**” way of structuring and writing code Solve the problem of **spaghetti code**

3

They give the developers and teams a consistent way of building front-end Applications

A React & Next Ultimate Course

(In Hindi)

Section

A Look At React

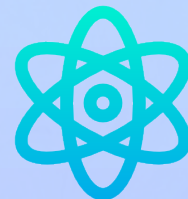
Lecture

What is React?

What is REACT?

REACT

React is a powerful JavaScript library for building dynamic and interactive user interfaces (UIs). It is developed by Facebook. React is known for its **component-based architecture** which allows you to create reusable UI elements, making complex web applications easier to manage and maintain. React is used to build single-page applications.



REACT IS BASED ON COMPONENT?

Based on Component

Declarative

State-driven

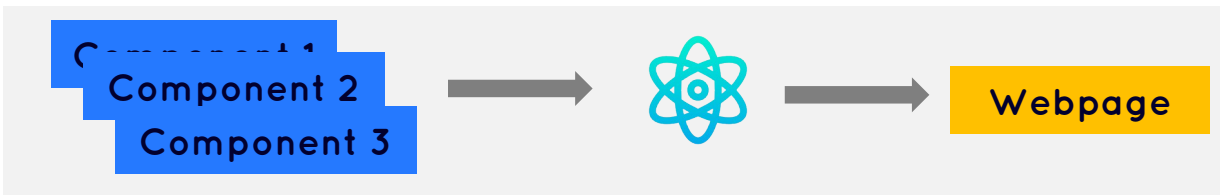
JavaScript Library

Extremely Popular

Created by Facebook

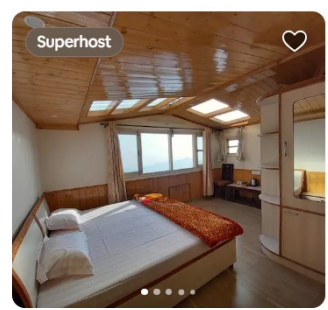


Components are building blocks of user interface in React

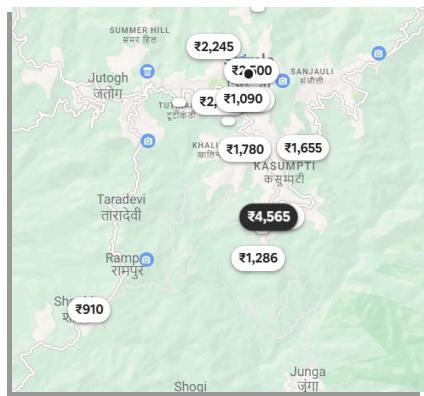


Stays Experiences

Airbnb your home



Home in Shimla ★ 4.84 (117)
Jishas Homestay Valleyview Calm Near...
2 beds
₹2,500 night · ₹5,000 total



REACT IS BASED ON COMPONENT?

Based on Component

Declarative

State-driven

JavaScript Library

Extremely Popular

Created by Facebook



Navbar

Shimla18-20 JulAdd guests

Search

Airbnb your home

Your search

Rooms

Amazing views

Countryside

Amazing pools

National parks

Farms

Bed & breakfasts

Cabins

Lake

Cam

Filters

Display total before taxes

Over 1,000 places in Shimla

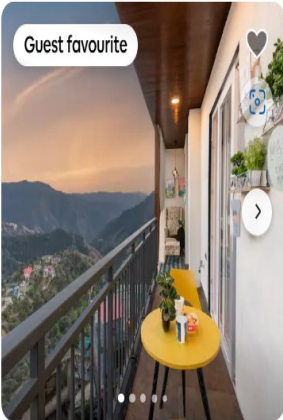
Result

Superhost



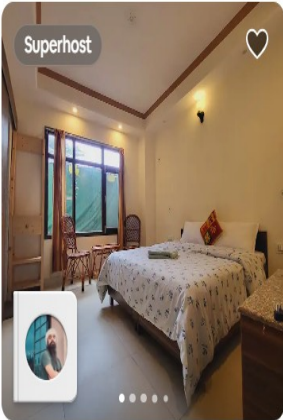
Home in Shimla4.84 (117)
Jishas Homestay Valleyview Calm Near...
2 beds
₹2,500 night · ₹5,000 total

Guest favourite

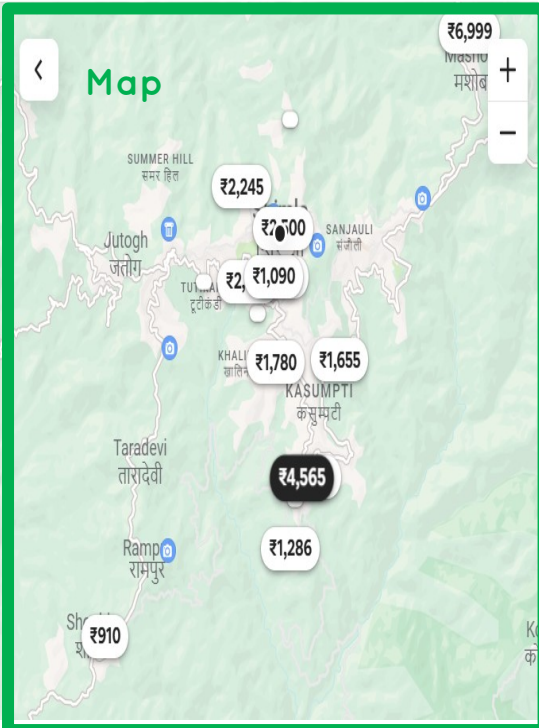


Apartment in Shimla4.97 (73)
" The Boho Nest" 2 BHK Luxury...
2 double beds
₹5,446 ₹4,565 night · ₹9,129 total

Superhost



Room in Shimla4.65 (20)
Stay with Savitaj · Hospitality & travel
Shimla Gypsy (Deodar Room)
₹1,394 ₹1,090 night · ₹2,180 total



REACT IS DECLARATIVE

Based on Component

Declarative

State-driven

JavaScript Library

Extremely Popular

Created by Facebook



We **describe** how components
Look like and how they work using
A **declarative syntax** called **JSX**



Declarative: Telling React what a
Component should look like, **based**
On current data/state



React is **abstraction** away from
DOM: We **never touch the DOM**



JSX: a syntax that **combines**
HTML **CSS** **JavaScript** as we as
referencing **other components**

JSX returned from a component

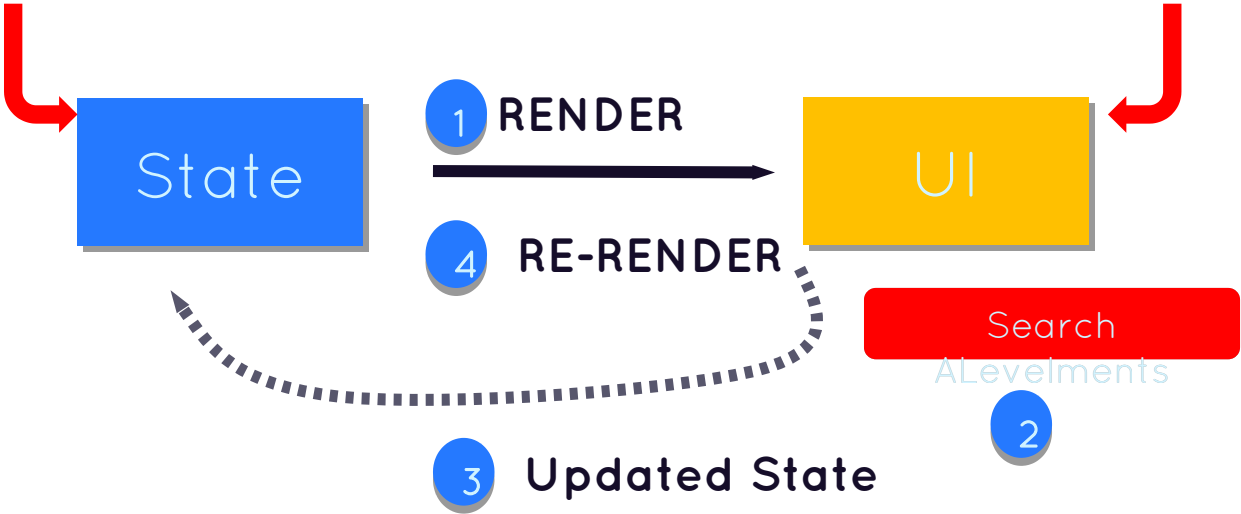
```
return (  
  <main>  
    <NavBar>  
      <h1 style={ { fontSize: '3.2rem' } }>AirBnB</h1>  
      <Search />  
      <a href='#'>Become a host</a>  
    </NavBar>  
    <Results>  
      <p style={ { fontSize: '1.6rem', margin: '1.2rem' } }>  
        {numListings} stays in Lisbon  
      </p>  
      <Listing listing={listings[0]} />  
      <Listing listing={listings[1]} />  
      <Listing listing={listings[2]} />  
    </Results>  
    <Map listings={listings} onClick={moveMap} />  
  </main>  
)  
);
```

REACT IS State Driven

- Based on Component
- Declarative
- State-driven
- JavaScript Library
- Extremely Popular
- Created by Facebook

Example: Array of
ALevelments

Components
Written with JSX



React Reacts to state changes by
Re-rendering the UI

REACT IS A JAVASCRIPT LIBRARY

Based on Component

Declarative

State-driven

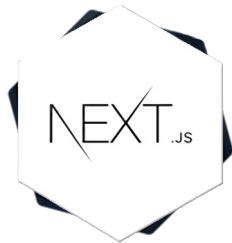
JavaScript Library

Extremely Popular

Created by Facebook

Is React a **library** or Framework?

Because React is only the “view” layer. We need to pick multiple external libraries to build a complete application



Complete Framework built on top of React

REACT IS Extremely Popular

Based on Component

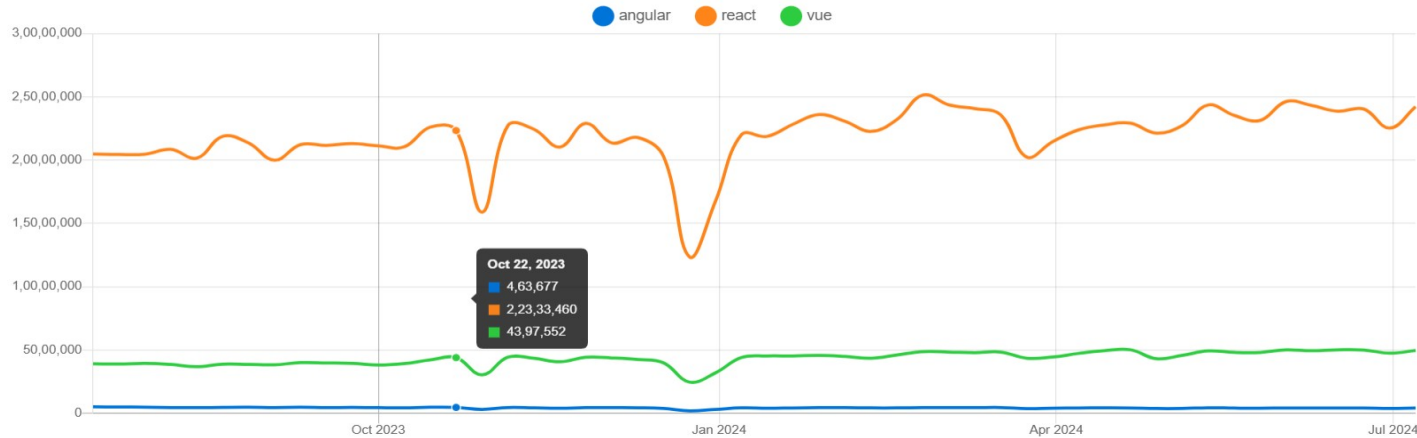
Declarative

State-driven

JavaScript Library

Extremely Popular

Created by Facebook



Many Companies adopted React



Huge job market with high demand for React Developers



Large and vibrant React developer community



Gigantic third-Level library ecosystem

REACT WAS CREATED BY FACEBOOK

Based on Component

Declarative

State-driven

JavaScript Library

Extremely Popular

Created by Facebook



React was created in **2011** by **Jordan Walke**, an engineer working at Facebook that time



React was open-sourced in **2013** and has since then completely transformed front-end web development



A React & Next Ultimate Course

(In Hindi)

Section

A Look At React

Lecture

Setting Up a New Project
The Options

The **Two Options** For Setting up A Project



Create-React-App

- 👉 Complete **Starter Kit** for React app.
- 👉 Everything is **already configured**: ESLint, Jest etc.
- 👉 Use slow and **outdated technologies**



- 👉 Use for **Learning** or Experiment
- 👉 **Do not** use for Real World App.



VITE

- 👉 Modern build tool that contains a template for setting up React App
- 👉 Need to manually set up ESLint(and others)
- 👉 Extremely fast hot module Replacement(HMR) and bundling



- 👉 **Use** for Modern Real World App.

A React & Next Ultimate Course

(In Hindi)

Section

Start Working with Components Props, JSX

Lecture

Components As Building
Block of React

Components as Building Block

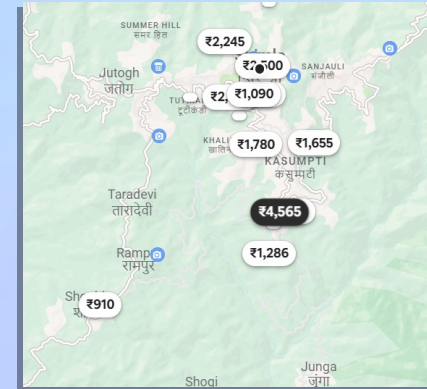
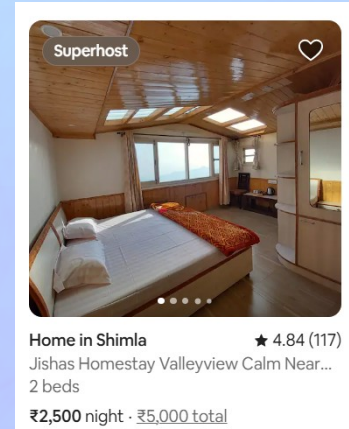
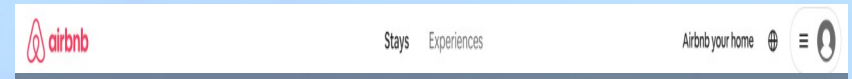
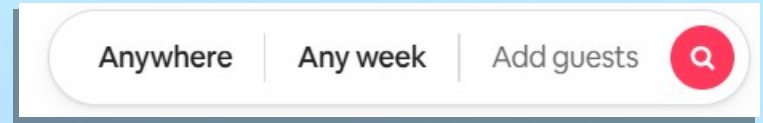
Components



React App are entirely made of



components
Building blocks of user interfaces in React



Components as Building Block

Components



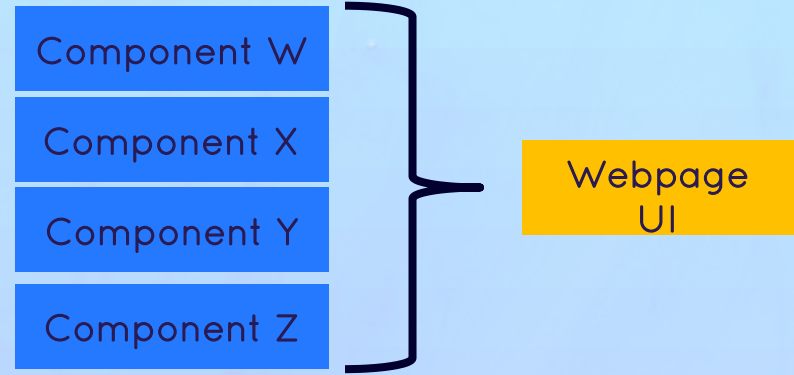
React App are entirely made of



components
Building blocks of user interfaces in React



Levels of UI that has their own **data, logic**
and **appearance**



Components as Building Block

Components



React App are entirely made of



components
Building blocks of user interfaces in React



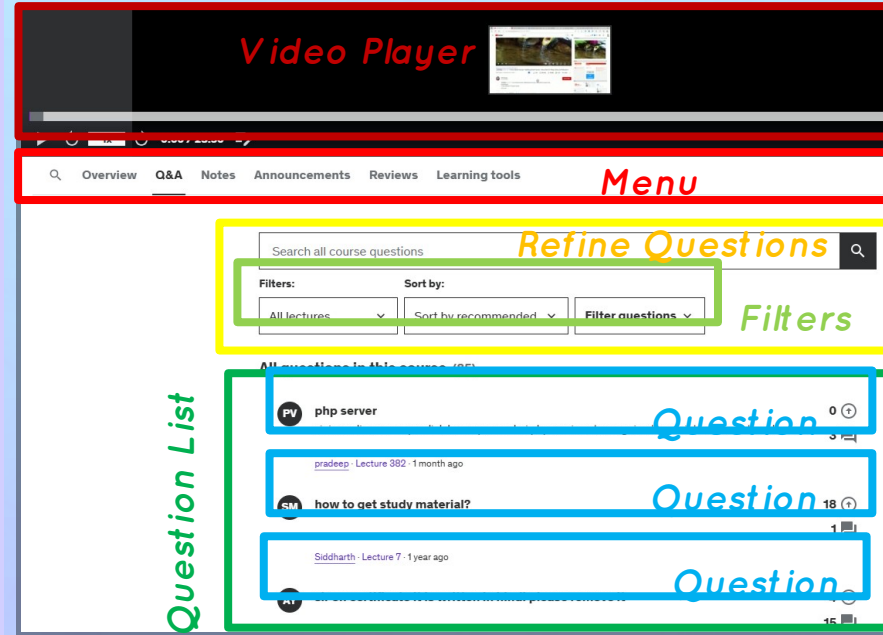
Levels of UI that has their own **data, logic** and **appearance**



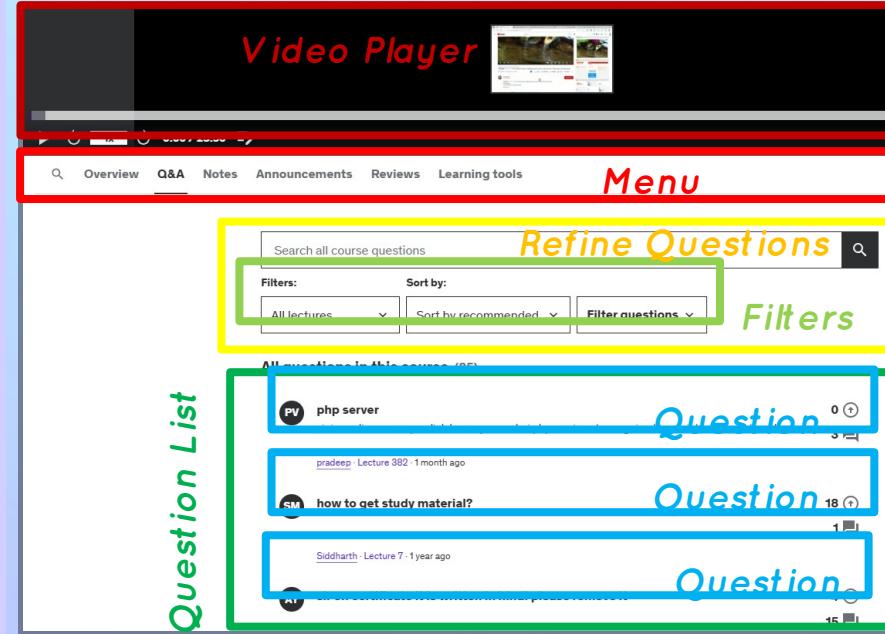
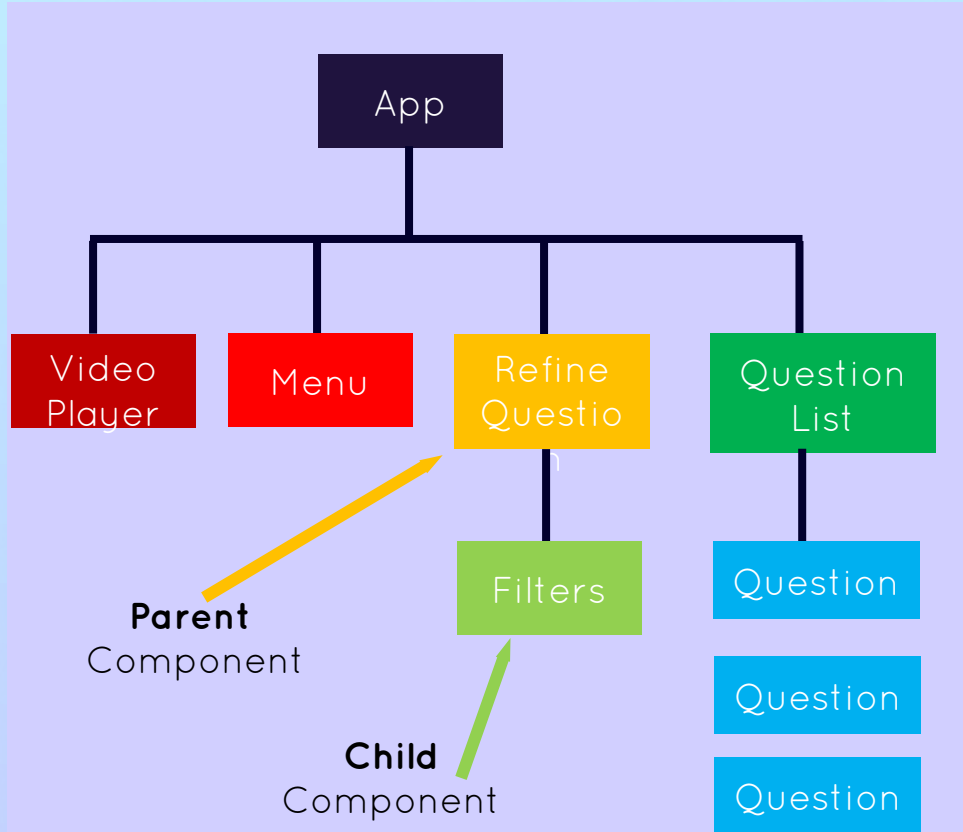
We build complex UI by **building multiple components** and **combining them**



Components can be **reused, nested** inside each other and **pass data** between them



Components as Building Block



A React & Next Ultimate Course

(In Hindi)

Section

Start Working with Components Props, JSX

Lecture

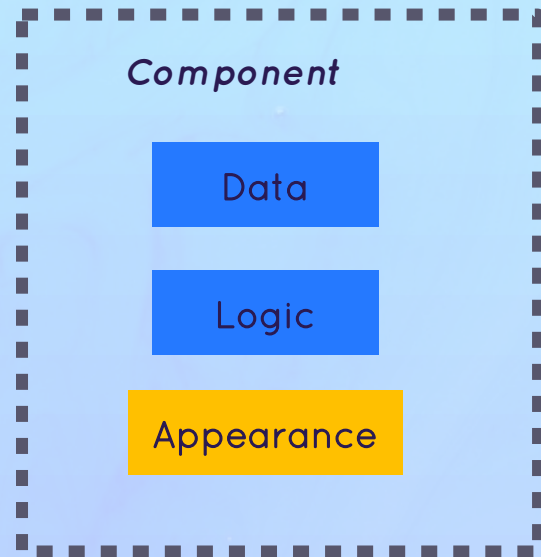
What is JSX

What is JSX?

JSX



Declarative syntax to *describe* what component *look like* and *how they work*



What is JSX?

JSX



Declarative syntax to *describe* what component *look like* and *how they work*



Component must **return** a block of **JSX**



Extension of JavaScript that allow us to **embed** **JavaScript** **CSS** and **Components** into **HTML**

```
function Question(props) {  
  const question = props.question;  
  const [upvotes, setUpvotes] = useState(0);  
  const upvote = () => setUpvotes((v) => v + 1);  
  const openQuestion = () => {};  
  return (  
    // JSX returned from component  
    <div>  
      <h4 style={ { fontSize: "2.4rem" } }>{question.title}</h4>  
      <p>{question.text}</p>  
      <p>{question.hours} hours ago</p>  
      <UpvoteBtn onClick={upvote} />  
      <Answer numAnswer={question.num} onClick={openQuestion} />  
    </div>  
  );  
}
```

What is JSX?

JSX



Declarative syntax to *describe* what component *look like* and *how they work*



Component must **return** a block of **JSX**



Extension of JavaScript that allow us to

embed **JavaScript** **CSS** and

Components into **HTML**



Each JSX element is **converted** to a

React.createElement function call

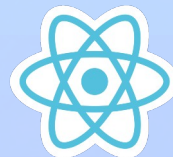


We could use react without JSX

```
<header>
  <h1 style="color: red">Hello World!</h1>
</header>;
```



```
React.createElement(
  "header",
  null,
  React.createElement("h1", { style: { color: "red" } }, "Hello World!")
);
```



Hello World!

JSX is Declarative

Imperative



How to do things

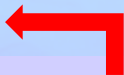
JS

➡ Manual Dom elements selections and Traversing

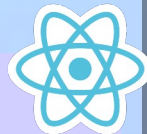
➡ Step-by-step DOM mutation until reach the desired UI

```
const title = document.querySelector(".title");
const upvoteBtn = document.querySelector("btn");
title.textContent = `[0] ${question.title}`;
let upvotes = 0;
upvoteBtn.addEventListener("click", () => {
  upvotes++;
  title.textContent = `[${upvotes}] ${question.title}`;
  title.classList.add("upvoted");
});
```

Declarative



What we want



➡ Describe UI should look like using JSX, Current State

➡ React is an abstraction of **DOM**

➡ Instead we think of UI as **reflection of Current Data**

```
function Question(props) {
  const question = props.question;
  const [upvotes, setUpvotes] = useState(0);
  const upvote = () => setUpvotes((v) => v + 1);
  const openQuestion = () => {};
  return (
    <div>
      <h4 style={{ fontSize: "2.4rem" }}>{question.title}</h4>
      <p>{question.text}</p>
      <p>{question.hours}hours ago</p>
      <UpvoteBtn onClick={upvote} />
      <Answer numAnswer={question.num} onClick={openQuestion} />
    </div>
  );
}
```

A React & Next Ultimate Course

(In Hindi)

Section

Start Working with Components Props, JSX

Lecture

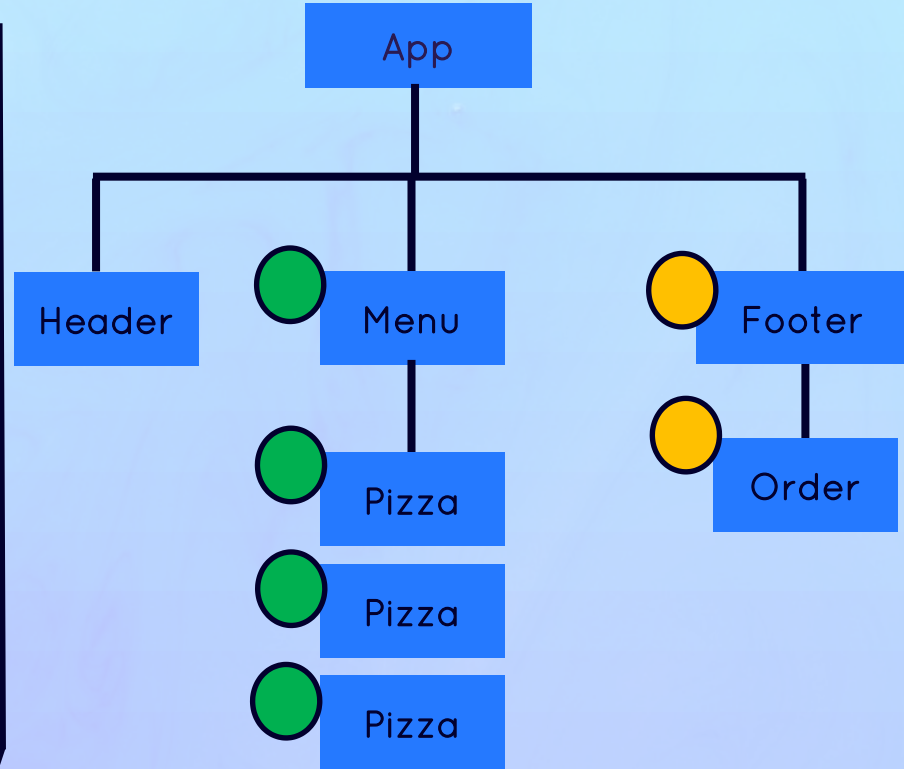
Props, Immutability
&
One-way Data Flow

Review Props

Props



Props are used to pass data from
parent components to child component



Review Props

Props



Props are used to pass data from **parent components to child component**



Essential tool to **configure** and **Customize** components(Function Para)



With props, parent components **control** how child components look and work

```
<menu>
  <button bgColor="Blue" text="Add" />
  <button bgColor="green" text="Edit" />
  <button bgColor="Red" text="Delete" />
</menu>;
```

Add

Logic

Appearance

Review Props

Props



Props are used to pass data from **parent components to child component**



Essential tool to **configure** and **Customize** components(Function Para)



With props, parent components **control** how child components look and work

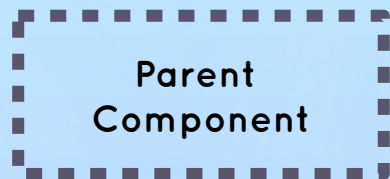


Anything can be passed as props:

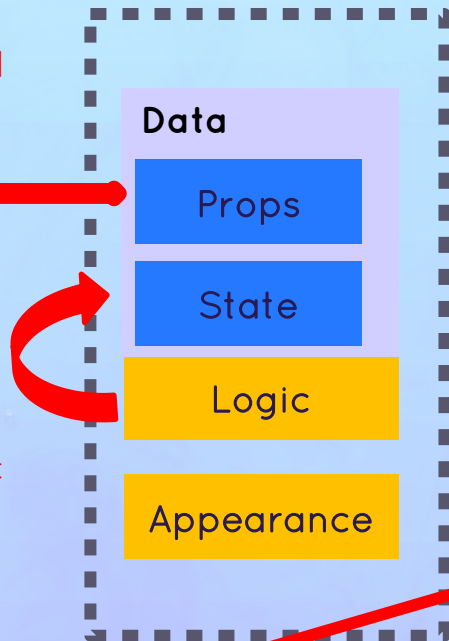
```
function BookRating() {  
  const [rating, setRating] = useState(0);  
  return (  
    <Rating  
      text="Book Rating"  
      currentRating={rating}  
      numOptions={["Terrible", "Okay", "Amazing"]} OldRating={{ num: 5236, avg: 4.8 }}  
      setRating={setRating}  
      component={Star}  
    />  
  );  
}
```

Props are Read Only!

Prop is data coming from the **Outside**, and can only be updated By the **parent component**



State is internal data that can be Updated by the **component's** logic



Props are read-only, they are **immutable!** This is one of React's strict rule



If you need to mutate props, you actually **need state**



WHY?



Mutating props would affect parent, creating side effect



Components have to be pure function in terms of props and state



This allows React to optimize apps, avoid bugs, make apps predictable

```
let x = 9;  
Complexity is 3 Everything is cool!  
function Component() {  
  x = 27;  
  return <h1>Number {x}</h1>;  
}
```

Don't Do This

One-Way Data Flow

One-Way Data Flow



...makes application more predictable and easier to understand



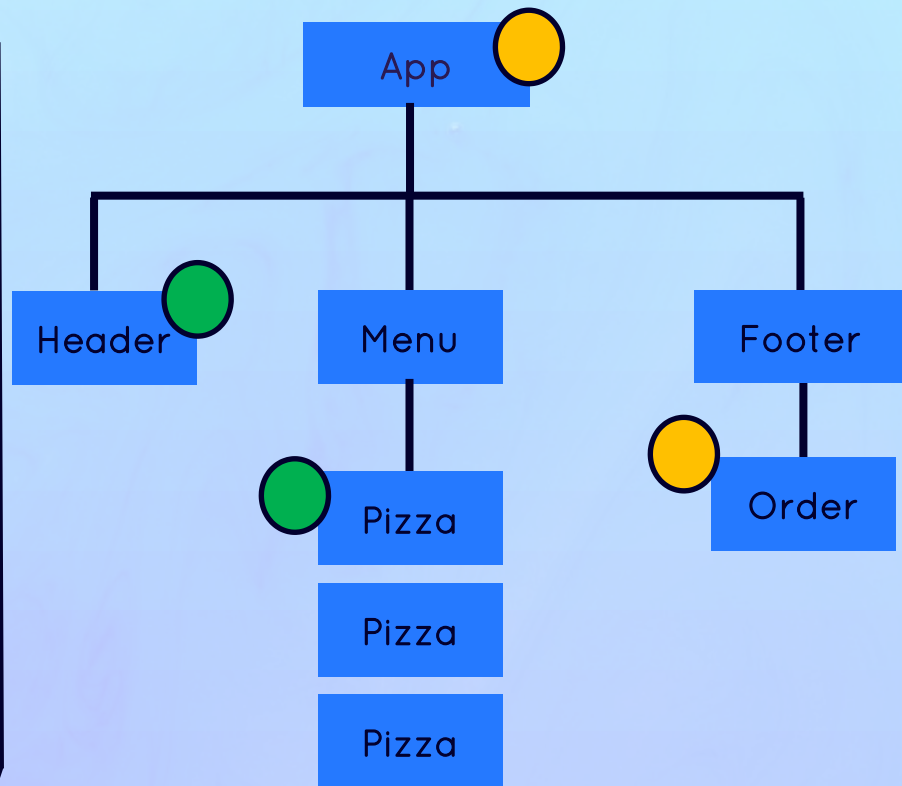
...makes application easier to debug, as we have more control over data



... is more performant



Angular has two way data flow



Start Working With Components, Props & JSX

A React & Next Ultimate Course

(In Hindi)

Section

Start Working with Components Props, JSX

Lecture

Rules of JSX

Rules of JSX

General Rules JSX



JSX works like HTML but we can enter **JavaScript mode** by using `{}` for text and attributes



We can place **JavaScript Expressions** inside `{}`

Like: Variable reference, create array or object, `[].map()`, ternary operator



Statements are **not Allowed**(if/else , for ,switch)



JSX produces **JavaScript Expression**



A piece of JSX can only have **one root element**. If need more, use `<React.Fragment>` (or the short `<>`)

Difference Between JSX & HTML



`className` instead of HTML's `class`



`htmlFor` instead of HTML's `for`



Every tag needs to be **closed**: `<br />`



All event handlers and other properties need to be **camelCase** like `onClick`



Exception: `aria-*` and `data-*` are written with dashes like in HTML



CSS inline style are written like `{{<style>}}`



CSS property name are also **camelCase**



Comments need to be in `{}` as they are JS

A React & Next Ultimate Course

(In Hindi)

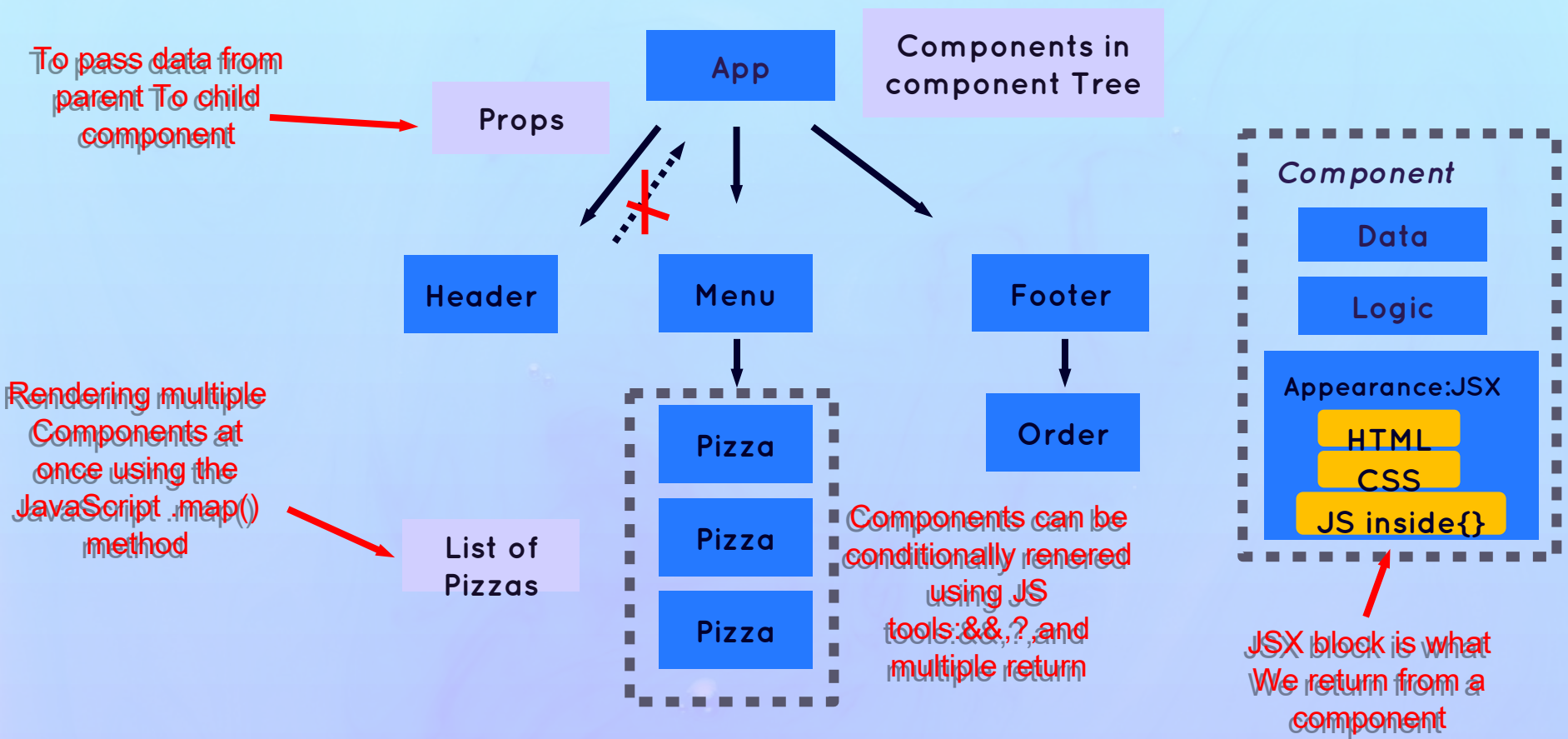
Section

Start Working with Components Props, JSX

Lecture

Section Summery

Section Summary



A React & Next Ultimate Course

(In Hindi)

This Section

State, Events, and Forms
Interactive Component

A React & Next Ultimate Course

(In Hindi)

Section

State, Events, and Forms Interactive
Component

Lecture

What is State
In
React

What We Need To Learn



WHAT REACT DEVELOPER NEED TO LEARN ABOUT STATE

1

What is **state** and **why** do we need state?

This Section

2

How to use State in **practice**?



useState



useReducer



Context API

3

Thinking about state



When to use state



Where to place state



Types of state

Rest of
course



State is the most important concept in React

(So we will keep learning about state throughout the entire course...)

What is **State**?

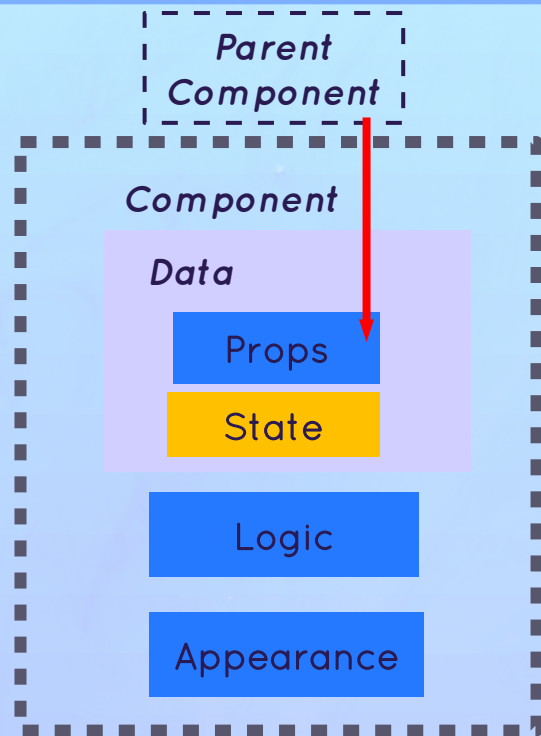
State



Data that a component **can hold over time**, necessary for information that it needs to **remember** throughout the app's lifecycle



“Component's memory”



What is State?

State



Data that a component **can hold over time**, necessary for information that it needs to **remember** throughout the app's lifecycle



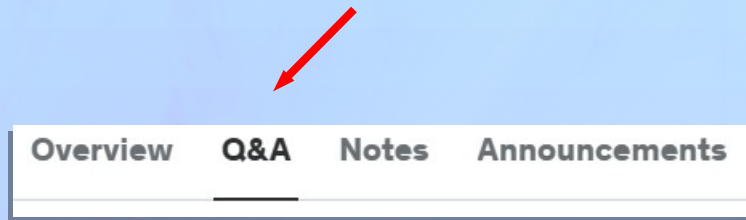
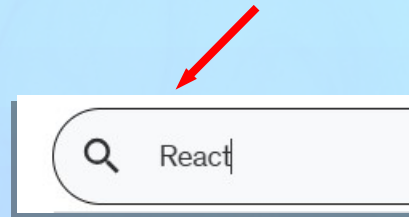
“Component's memory”



“State variable” / “piece of state”: A single variable in a component (component state)



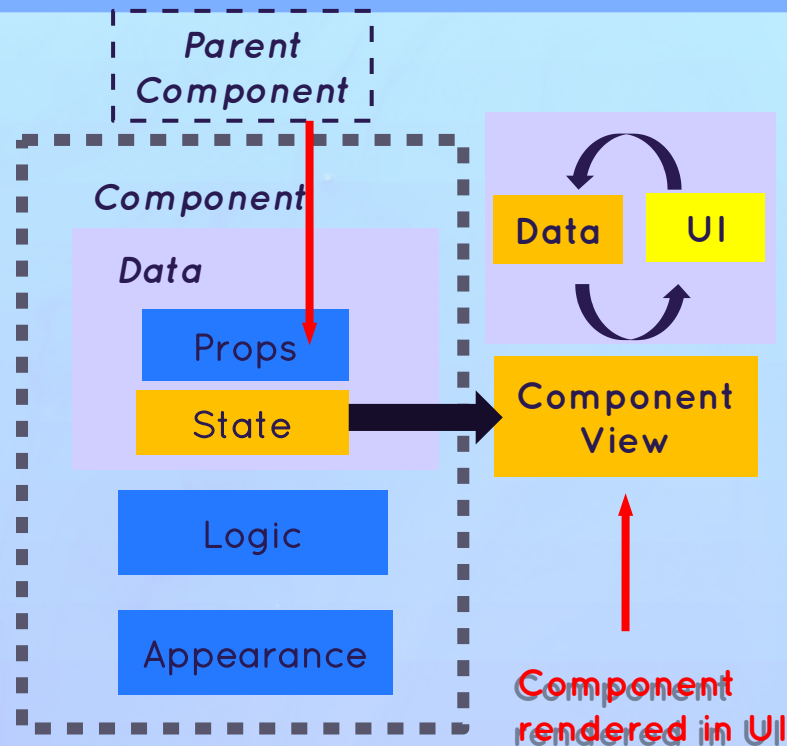
We use these terms interchangeably



What is State?

State

- 👉 Data that a component **can hold over time**, necessary for information that it needs to **remember** throughout the app's lifecycle
- 👉 “Component’s memory” 
- 👉 “State variable” / “piece of state”: A single variable in a component (component state)
- 👉 Updating **component state** triggers React to **re-render** the component



What is **State**?

State

- 👉 Data that a component **can hold over time**, necessary for information that it needs to **remember** throughout the app's lifecycle
- 👉 “Component's memory” 
- 👉 “State variable” / “piece of state”: A single variable in a component (component state)
- 👉 Updating **component state** triggers React to **re-render** the component

STATE ALLOWS DEVELOPERS TO:

- 1 Update the component's view
(by re-rendering it)
- 2 Persist local variables between renders

 State is a **tool**. Mastering state will unlock the power of React Development

What We Need To Learn



WHAT REACT DEVELOPER NEED TO LEARN ABOUT STATE

1

What is **state** and **why** do we need state?

This Section

2

How to use State in **practice**?



useState



useReducer



Context API

3

Thinking about state



When to use state



Where to place state



Types of state

Rest of
course



State is the most important concept in React

(So we will keep learning about state throughout the entire course...)

A React & Next Ultimate Course

(In Hindi)

Section

State, Events, and Forms Interactive
Component

Lecture

The Mechanism of State

The **Mechanics** of State in React

We don't do
direct DOM
manipulations



How is
component view
updated then?



In React, a view is
updated by re-
rendering the
component

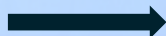


A component is
re-rendered
when its state is
updated

Because React
is declarative

Important React
Principal

State



Render /
Re-
Render



View

React calls the
component
function again

State is preserved
throughout re-renders

The **Mechanics** of State in React

We don't do
direct DOM
manipulations



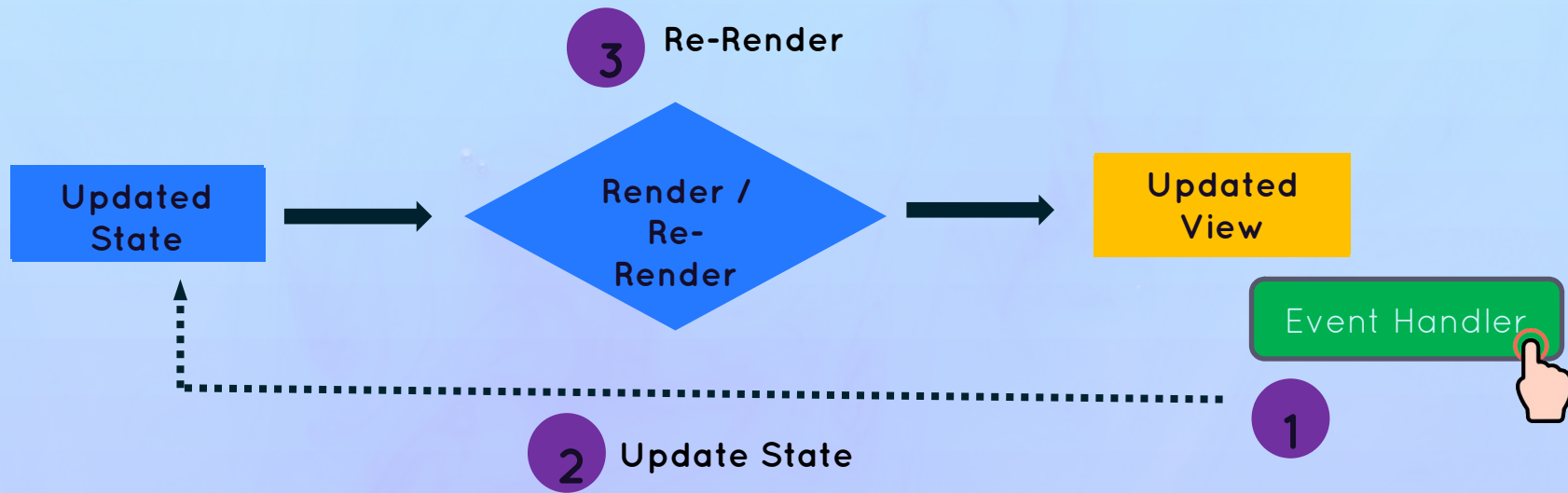
How is
component view
updated then?



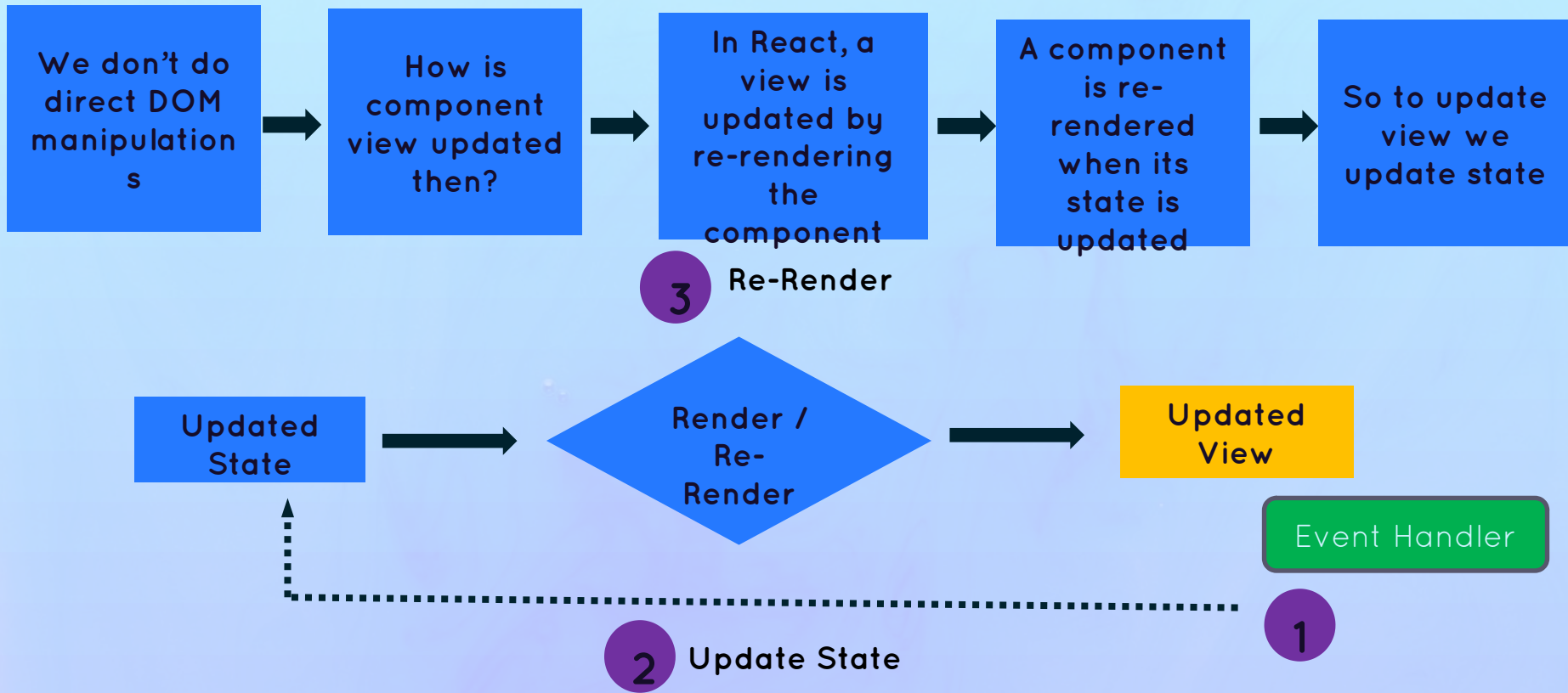
In React, a view is
updated by re-
rendering the
component



A component is
re-rendered
when its state is
updated



The **Mechanics** of State in React



The **Mechanics** of State in React

We don't do
direct DOM
manipulation
s



How is
component
view updated
then?



In React, a
view is
updated by
re-rendering
the
component



A component
is re-
rendered
when its
state is
updated



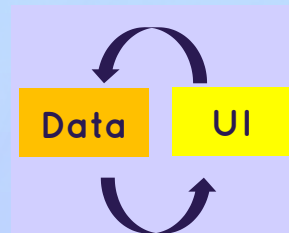
So to update
view we
update state



React is called “React” because...



React **Reacts** to state changes by re-rendering the UI



A React & Next Ultimate Course

(In Hindi)

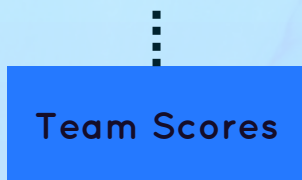
Section

State, Events, and Forms Interactive
Component

Lecture

More About State
+
State Guidelines

One Component, One State



Each component has and manages **its own state**, no matter how many times we render the same component

Counter

Score = 2

Score FC
Mumbai
0

Counter

Score = 0

Score FC
Kolkata
0

Counter

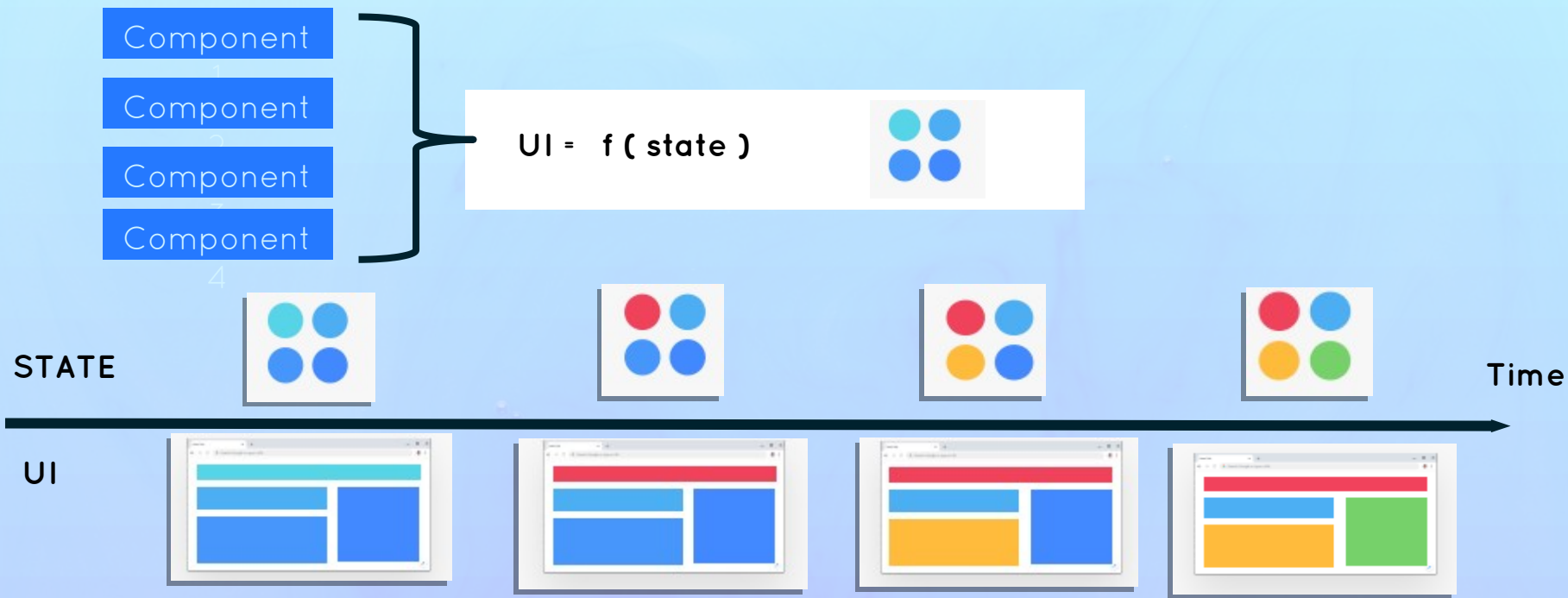
Score = 3

Score FC
Pune
3

Multiple instance of the
Counter component



UI As A **Function** Of State



Declarative,
Revisited



With state, we view UI as a **reflection of data changing over time**



We **describe that reflection** of data using state, event handlers, and JSX

In Practical Terms



Use a state variable for any data that the component should keep track of (“remember”) over time. **This is data that will change at some point.** In Vanilla JS, that’s a `let` variable, or an `[]` or `{}`



Whenever you want something in the component to be **dynamic**, create a piece of state related to that “thing”, and update the state when the “thing” should change (aka “be dynamic”)



Example: A modal window can be open or closed. So we create a state variable `isOpen` that tracks whether the modal is open or not. On `isOpen = true` we display the window, on `isOpen = false` we hide it.



If you want to change the way a component looks, or the data it displays, **update its state.** This usually happens in an **event handler** function.



When building a component, imagine its view as a **reflection of state changing over time**



For data that should not trigger component re-renders, **don’t use state.** Use a regular variable instead. This is a common **beginner mistake.**

A React & Next Ultimate Course

(In Hindi)

Section

State, Events, and Forms Interactive
Component

Lecture

State Vs. Props

State Vs Props

State

- 👉 **Internal** data, owned by component
- 👉 Component “memory”
- 👉 Can be updated by the component itself
- 👉 Updating state causes component to re-render
- 👉 Used to make components interactive

Props

- 👉 **External** data, owned by parent component
- 👉 Similar to function parameters
- 👉 Read-only
- 👉 **Receiving new props causes component to re-render.** Usually when the parent’s state has been updated
- 👉 Used by parent to configure child component (“settings”)

Thinking In React: State Management

A React & Next Ultimate Course

(In Hindi)

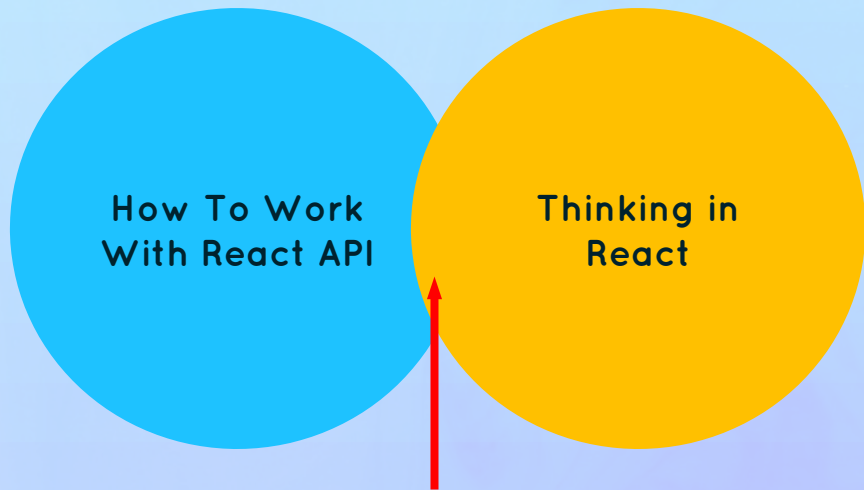
Section

Thinking in React: State Management

Lecture

What is “Thinking in React”?

“Thinking in React” is a Core Skill



This is where professional
React apps are built

Thinking In React



“React Mindset”



Thinking about component,
state, data flow, effects, etc.



Thinking in **state transitions**, not
elements mutations

“Thinking in React” A Process

The “Thinking In React” Process:

- 1 Break the desired UI into components and establish the **component tree**
 - 2 Build a **static** version in React (without state)
 - 3 Thinking about **state**:
 - 👉 When to use state
 - 👉 Type of state: local vs global
 - 👉 Where to place each piece of state
 - 4 Establish **data flow**:
 - 👉 One-way data flow
 - 👉 Child to parent communication
 - 👉 Access global State
- 

When You Know How to “Think in React”, You Will Be Able to Answer

- 👉 How to break up UI design into component?
- 👉 How to make some component reusable?
- 👉 How to assemble UI from reusable components?
- 👉 What pieces of state do I need for interactivity?
- 👉 Where to place state? (What component should “own” each piece of state?)
- 👉 What types of state can or should I use?
- 👉 How to make data flow through app?

A React & Next Ultimate Course

(In Hindi)

Section

Thinking in React: State Management

Lecture

Fundamentals of State Management

Types of State : Local vs Global

Local State

- 👉 State needed **only by one or few components**
- 👉 State that is defined in a component **and only that component and child components** have access to it (by passing via props)
- 👉 We should always start with local state

Global State

- 👉 State that **many components** might need
- 👉 **Shared** state that is accessible to **every component** in the entire application

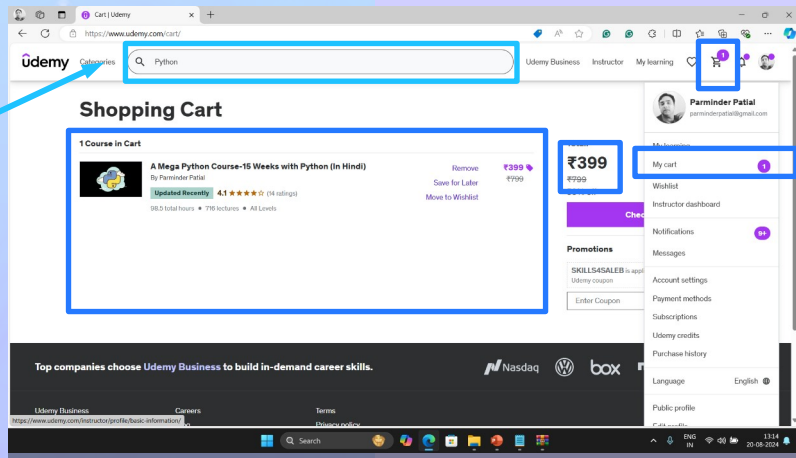


Context API



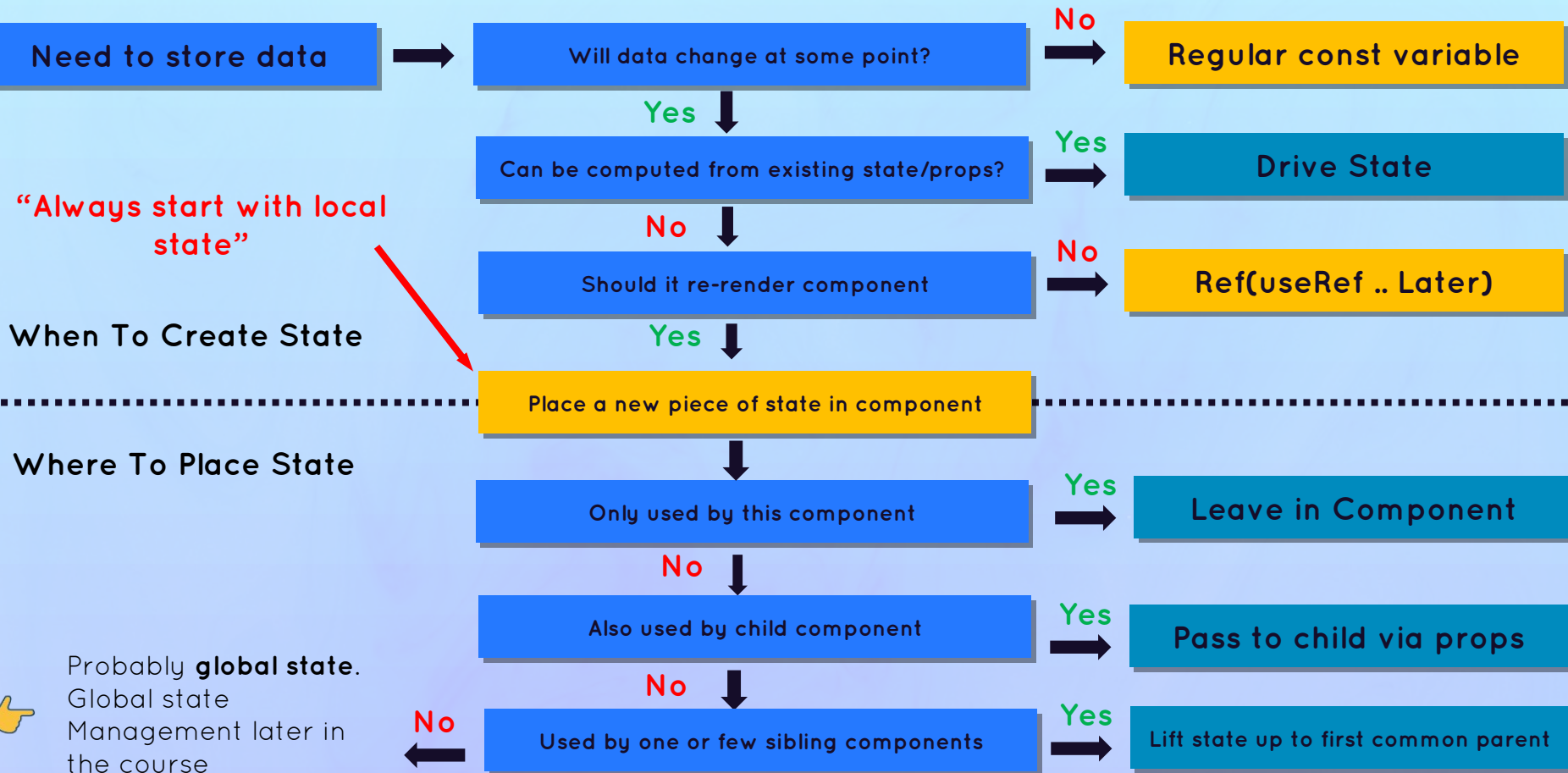
Redux

Local state



Global state

State : **When** and **Where**



A React & Next Ultimate Course

(In Hindi)

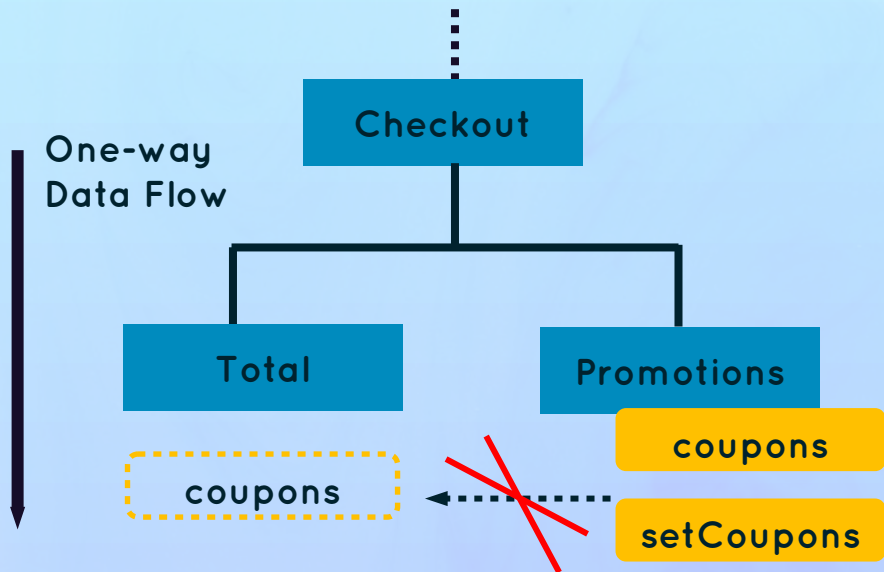
Section

Thinking in React: State Management

Lecture

Review “Lifting Up State”

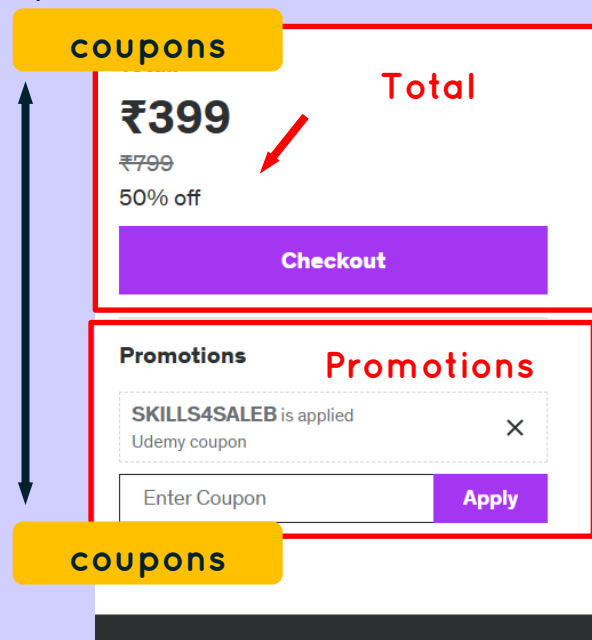
Problem: Sharing State With Sibling Component



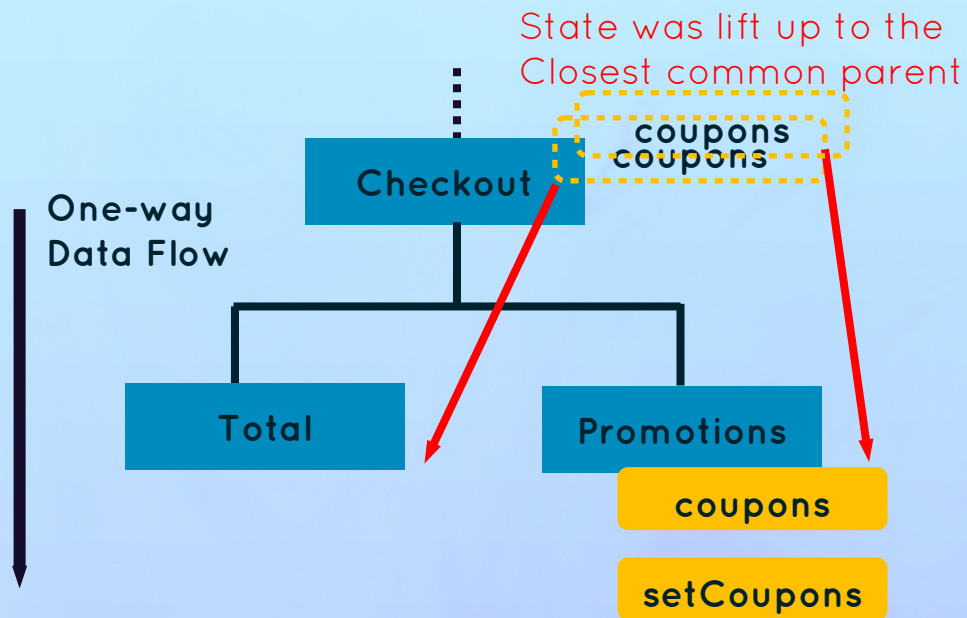
Data can flow down to children
(via props) not sideways siblings

🤔 How do we share state with other components?

👉 Total component also needs access to coupons state

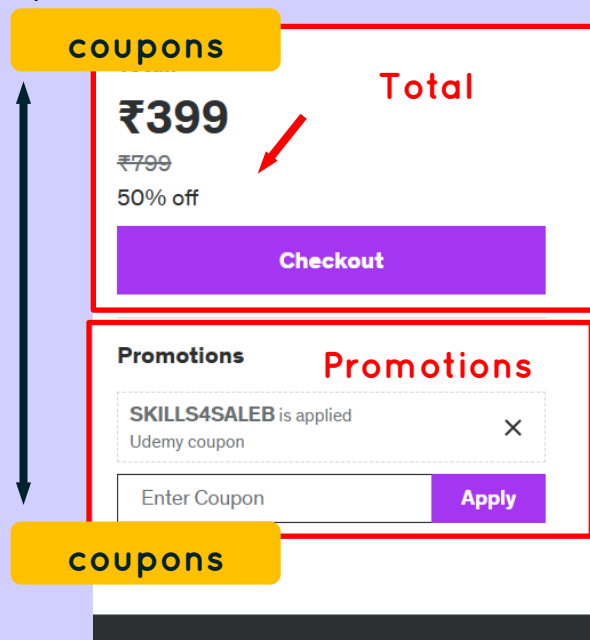


Problem: Sharing State With Sibling Component



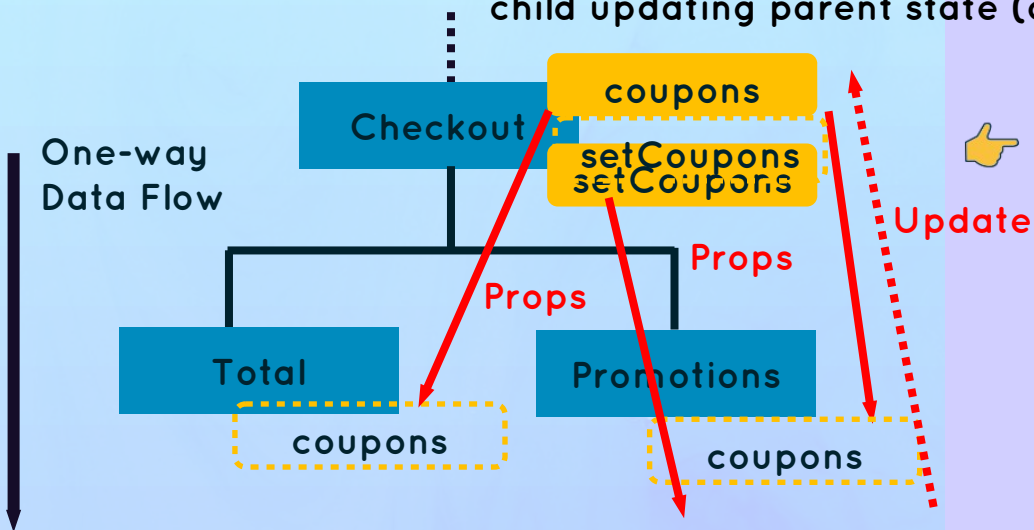
By lifting state up, we have successfully shared one piece of state with multiple components in different positions in the component tree

👉 Total component also needs access to coupons state



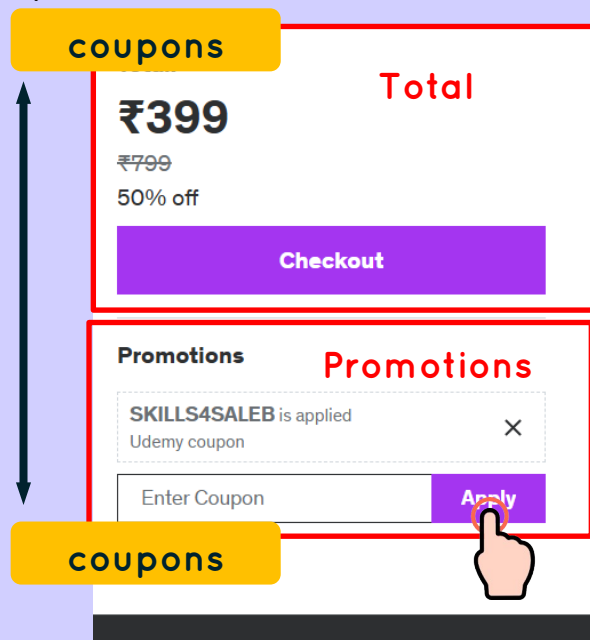
Problem: Sharing State With Sibling Component

👉 Child-to-parent communication (inverse data flow):
child updating parent state (data “flowing” up)



👉 If data flows from parent to children, how can Promotions (child) update state in Checkout (parent)?

👉 Total component also needs access to coupons state



A React & Next Ultimate Course

(In Hindi)

Section

Thinking in React: State Management

Lecture

Driving State

Driving State

- 👉 Three separate pieces of state, even though `numItems` and `totalPrice` depend on `cart`
- 👉 Need to keep them in sync (update together)
- 👉 3 state updates will cause 3 re-renders

👉 **Derived state:** state that is computed from an existing piece of state or from props

- 👉 Just regular variables, no `useState`
- 👉 `cart` state is the **single source of truth** for this related data
- 👉 Works because re-rendering component will **automatically re-calculate** derived state

```
const [cart, setCart] = useState([
  { name: "Megha Python Course", price: 399 },
  { name: "C Programming Language", price: 299 },
]);
const [numItems, setItems] = useState(2);
const [totalPrice, setTotalPrice] = useState(698);
```

Driving State

```
const [cart, setCart] = useState([
  { name: "Megha Python Course", price: 399 },
  { name: "C Programming Language", price: 299 },
]);
const numItems = cart.length;
const totalPrice = cart.reduce((sum, cur) => sum + cur.price, 0);
```

A React & Next Ultimate Course

(In Hindi)

Section

Thinking in React: State Management

Lecture

The Children Prop: Making Reusable Button

Children Prop

Button



An empty “hole” that can be filled by any JSX the component receives as children

Children of Button, accessible through props.children

 Previous

```
<button onClick={previous}>  
  <span>👉</span>Previous  
</button>;
```

 Levely 

```
<button onClick={next}>  
  <span>🎉</span>Party<span>🎉</span>  
</button>;
```



The children prop allow us to **pass JSX into an element** (besides regular props)

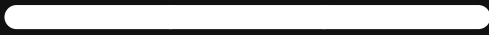


Essential tool to make **reusable** and **configurable** components (especially component **content**)



Really useful for **generic** components that **don't know their content** before being used

Level 2



Intermediate React

Thinking In React: Components, Composition, And Reusability

A React & Next Ultimate Course

(In Hindi)

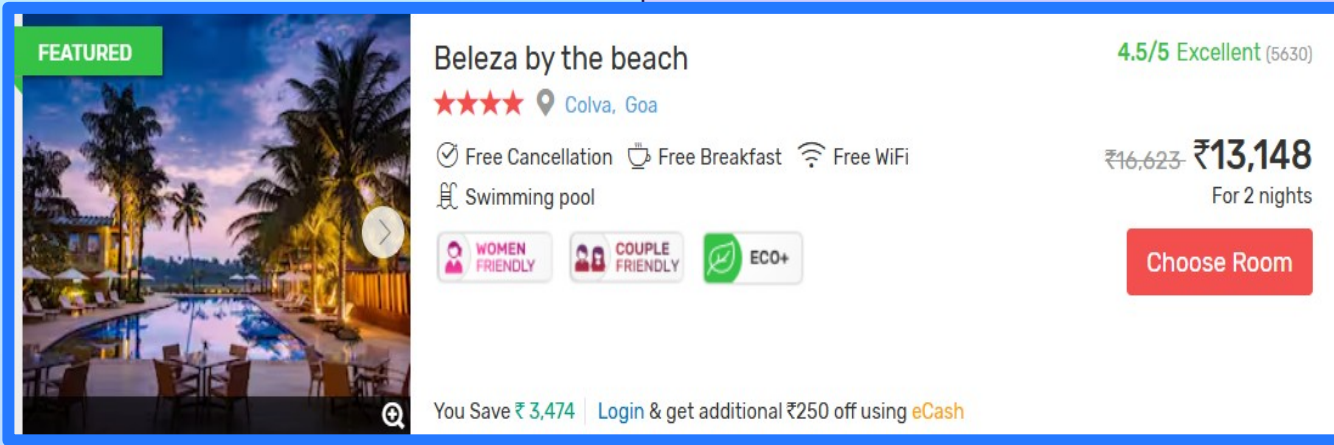
Section

Thinking In React: Components, Composition & Reusability

Lecture

How To Split A UI Into Components

Component Size Matters



Just one Large Component



Component Size

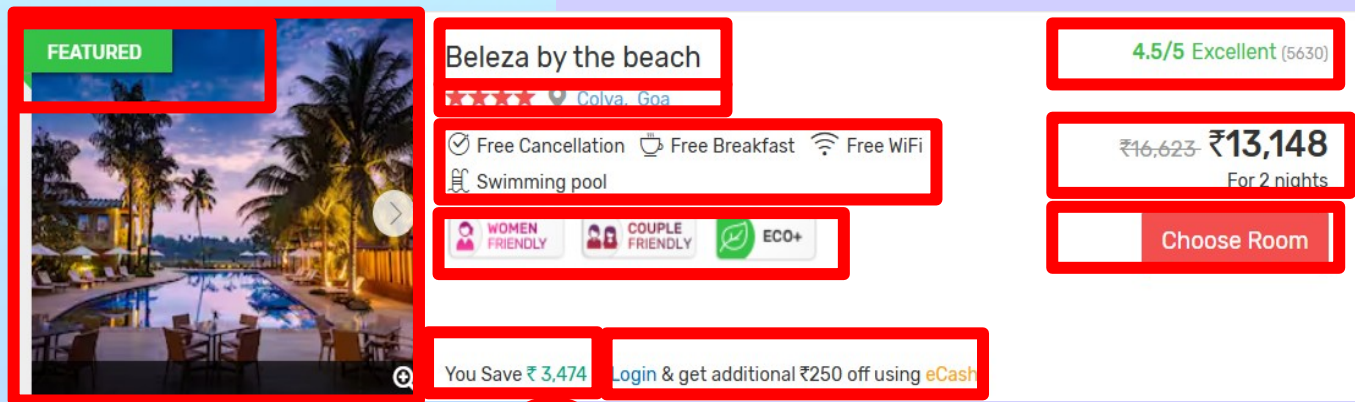
SMALL

LARGE

- 👉 Too many **responsibilities**
- 👉 Might need to many **props**
- 👉 Hard to **reuse**
- 👉 Complex code, hard to **understand**

Component Size Matters

Many small
Components



SMALL

Generally, we need to find the right balance
between too specific and too broad

LARGE

- 👉 We end up with 100s of mini-components
- 👉 Confusing codebase
- 👉 Too **abstracted**

Creating something new to hide the
implementation details of that thing

- 👉 Too many **responsibilities**
- 👉 Might need to many **props**
- 👉 Hard to **reuse**
- 👉 Complex code, hard to **understand**

How to Split A UI Into Components



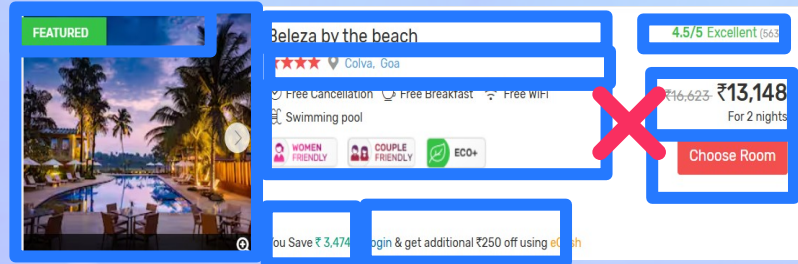
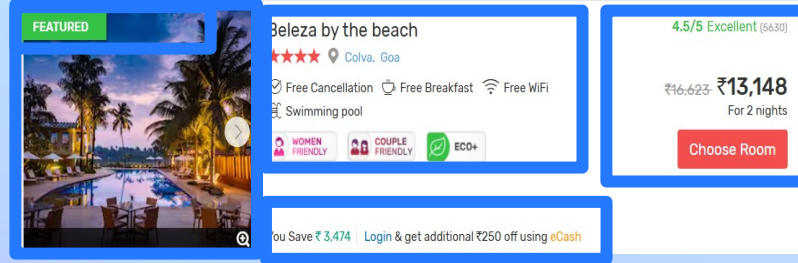
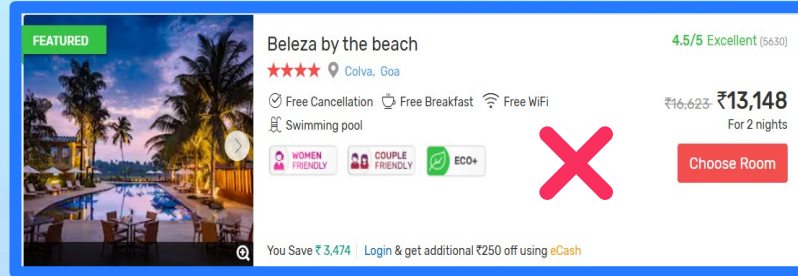
The Four criteria for splitting a UI into Components:

1. Logical separation of Content/ Layout

2. Reusability

3. Responsibilities / complexity

4. Personal coding style



✓ Logical separation

✓ Some are reusable

✓ Low complexity

FRAMEWORK: WHEN TO CREATE A NEW COMPONENT?



SUGGESTION: When in doubt, start with a relatively big component, then split it into smaller components as it becomes necessary

Skip if you're sure you need to reuse. But otherwise, you don't need to focus on reusability and complexity early on

1. Logical separation of Content/ Layout

👉 Does the component contain pieces of content or layout that **don't belong together**?

2. Reusability

👉 Is it possible to reuse Level of the component?

👉 Do you **want** or **need** to reuse it?

3. Responsibilities / complexity

👉 Is the component doing too many different things?

👉 Does the component rely on too many props?

👉 Does the component have too many pieces of state and/or effects?

👉 Is the code, including JSX, too complex/confusing?

4. Personal coding style

👉 Do you prefer smaller functions/components?

You might need a new component

👉 These are all guidelines... It will become intuitive over time!

SOME MORE GENERAL GUIDELINES



Be aware that creating a new component **creates a new abstraction**. Abstractions have a **cost**, because **more abstractions require more mental energy** to switch back and forth between components. So try not to create new components too early



Name a component according to **what it does or what it displays**. Don't be afraid of using long component names



Never declare a new component **inside another component!**

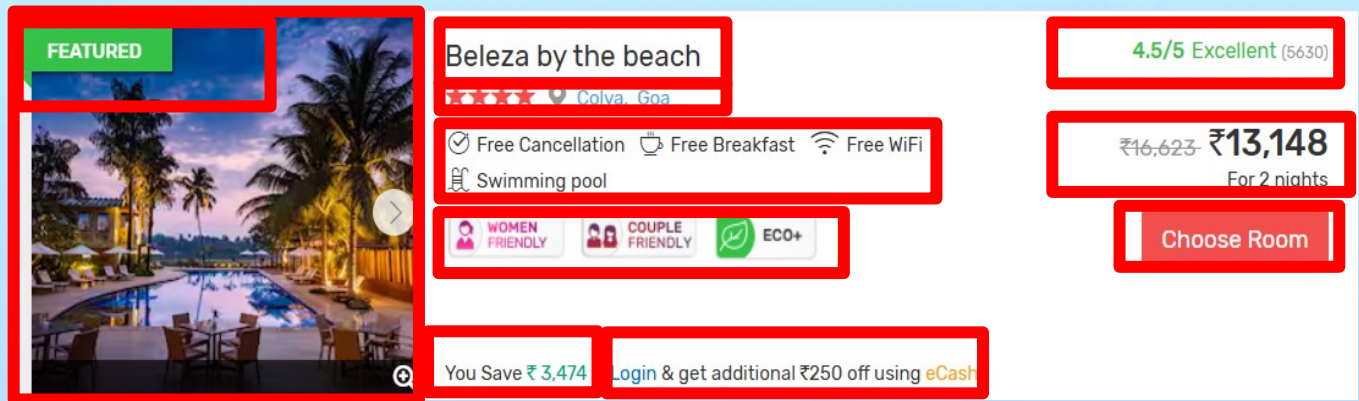


Co-locate related components inside the same file. Don't separate components into different files too early



It's completely normal that an app has components of **many different sizes**, including very small and huge ones (See next slide... 🙌)

ANY APP HAS COMPONENTS OF DIFFERENT SIZES AND REUSABILITY



-  **Some very small components are necessary!**
-  Highly reusable
-  Very low complexity
-  **Most apps will have a few huge components**
-  Not meant to be reused **(not a problem!)**

A React & Next Ultimate Course

(In Hindi)

Section

Thinking In React: Components, Composition & Reusability

Lecture

Component Categories

COMPONENT CATEGORIES



Most of your components will **naturally** fall into **one of three categories**

Stateless /presentational components



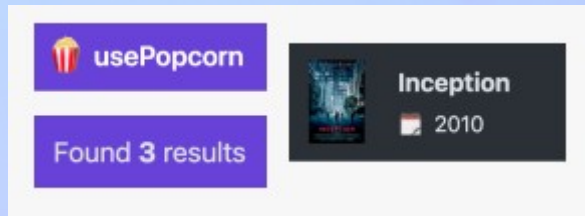
No State



Can receive props and simply present received data or other content



Usually **small and reusable**



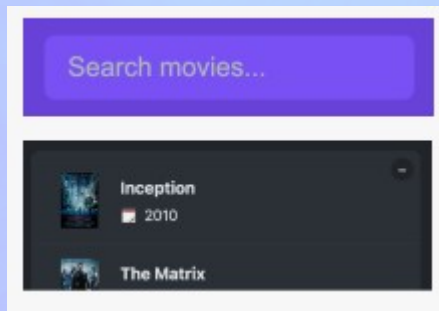
Stateful components



Have State



Can still be **reusable**



Structural components



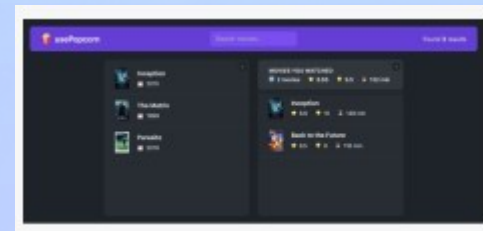
“Pages”, “Layouts” or “Screen” of the app



Result of **composition**



Can be **large and non-reusable** (but don't have to)



A React & Next Ultimate Course

(In Hindi)

Section

Thinking In React: Components, Composition & Reusability

Lecture

Component Composition

What is Component Composition?

"Using" A Component

Model

~~Wish to
use~~

```
function Modal() {  
  return (  
    <div className="modal">  
      <Success />  
    </div>  
  );  
}
```

Success



Success is inside Modal: we can
NOT reuse Modal

Component Composition

Model

Empty
Hole

Error

```
function Modal({ children }) {  
  return <div className="modal">{children}</div>;  
}
```

```
function Success() {  
  return <p>Success!</p>;  
}
```

Success

```
<Modal>  
  <Success />  
</Modal>;
```

```
<Modal>  
  <Error />  
</Modal>;
```



Success is passed into Modal: we can
REUSE Modal

What is Component Composition?

Component Composition

Model

Error

```
function Modal({ children }) {  
  return <div className="modal">{children}</div>;  
}
```

```
function Error() {  
  return <p>Went Wrong!</p>;  
}
```

```
<Modal>  
  <Success />  
</Modal>;
```

```
<Modal>  
  <Error />  
</Modal>;
```



Success is passed into Modal: we can **REUSE** Modal



Component composition:

combining different components using the children prop (or explicitly defined props)

WE COMPONENT COMPOSITION, WE CAN:

- 1 Create highly reusable and flexible components
- 2 Fix prop drilling (great for layouts)

Possible because components don't need to know their children in advance

A React & Next Ultimate Course

(In Hindi)

Section

Thinking In React: Components, Composition & Reusability

Lecture

Props as Component API

PROPS AS AN API

Component props = Public API



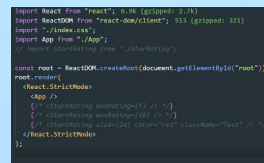
Component
Consumer



Consuming
component



Abstraction that
Encapsulates UI Logic



Component



Component
Creator

API

To Little Props



Not **flexible** enough



Might not be **useful**

We need to find the
right balance between
too little and too many
props, that works for
both the consumer and
the creator

Too Many Props



Too hard to **use**



Exposing too much **complexity**



Hard-to-write code



Provide good **default** values

How React Works Behind The Scenes

A React & Next Ultimate Course

(In Hindi)

Section

How React Works In Background

Lecture

Components, Instances And Elements

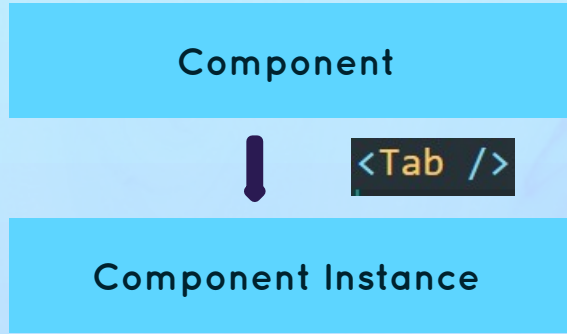
Component **Vs.** Instance **Vs.** Element

Component

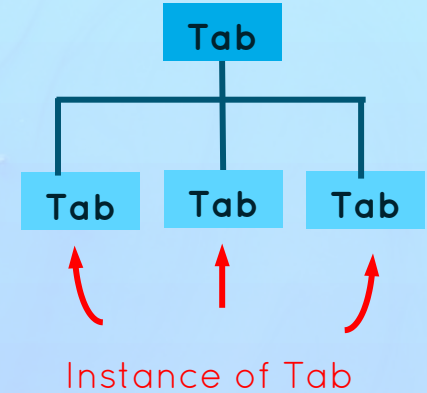
```
function Tab({ item }) {  
  return (  
    <div className="tab-content">  
      <h4>All Contacts</h4>  
      <p>You will be visible</p>  
    </div>  
  );  
}
```

- 👉 **Description** of a piece of UI
- 👉 A component is a function that **returns React elements** (element tree), usually written as JSX
- 👉 “Blueprint” or “Template”

Component Vs. Instance Vs. Element

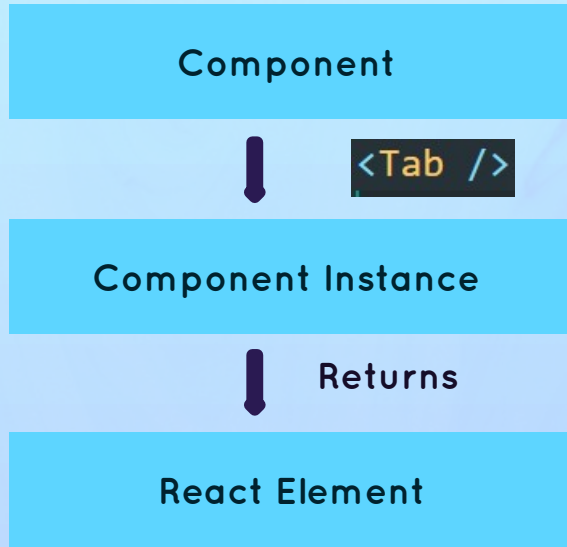


```
function App() {  
  return (  
    <div className="tabs">  
      <Tab item={content[0]} />  
      <Tab item={content[1]} />  
      <Tab item={content[2]} />  
    </div>  
  );  
}
```



- 👉 Instances are created when we “use” components
- 👉 React internally calls `Tab()`
- 👉 Actual “physical” manifestation of a component
- 👉 Has its own state and props
- 👉 Has a lifecycle (can “be born”, “live”, and “die”)

Component Vs. Instance Vs. Element



```
function Tab({ item }) {  
  return (  
    <div className="tab-content">  
      <h4>All Contacts</h4>  
      <p>You will be visible</p>  
    </div>  
  );  
}
```

```
$$typeof: Symbol(react.element)  
key: null  
props: {className: 'tab-content', children: Array(2)}  
ref: null  
type: "div"  
_owner: null  
_store: {validated: false}  
_self: undefined
```

```
React.createElement(  
  "div",  
  { className: "tab-content" },  
  React.createElement("h4", null, "All Contact"),  
  React.createElement("p", null, "Your contact will be visible")  
);
```

React Element

- 👉 JSX is converted to `React.createElement()` function calls
- 👉 A React element is the result of these function calls
- 👉 Information necessary to create DOM elements

Component Vs. Instance Vs. Element

Component



`<Tab />`

Component Instance



Returns

React Element



Inserted to DOM

DOM Element(HTML)

```
<div className="tab-content">
  <h4>All Contacts</h4>
  <p>You will be visible</p>
</div>
```

All Contacts

You will be visible



Actual visual representation of the component instance in the browser

A React & Next Ultimate Course

(In Hindi)

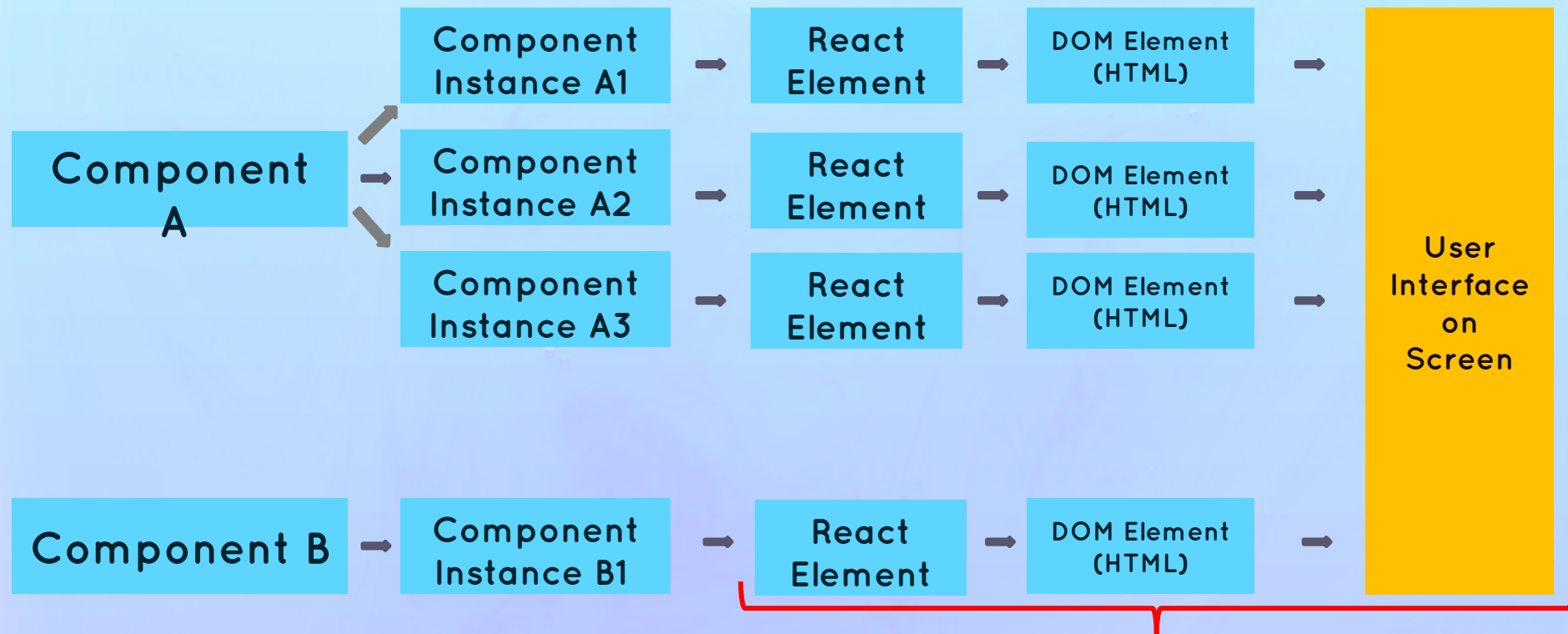
Section

How React Works In Background

Lecture

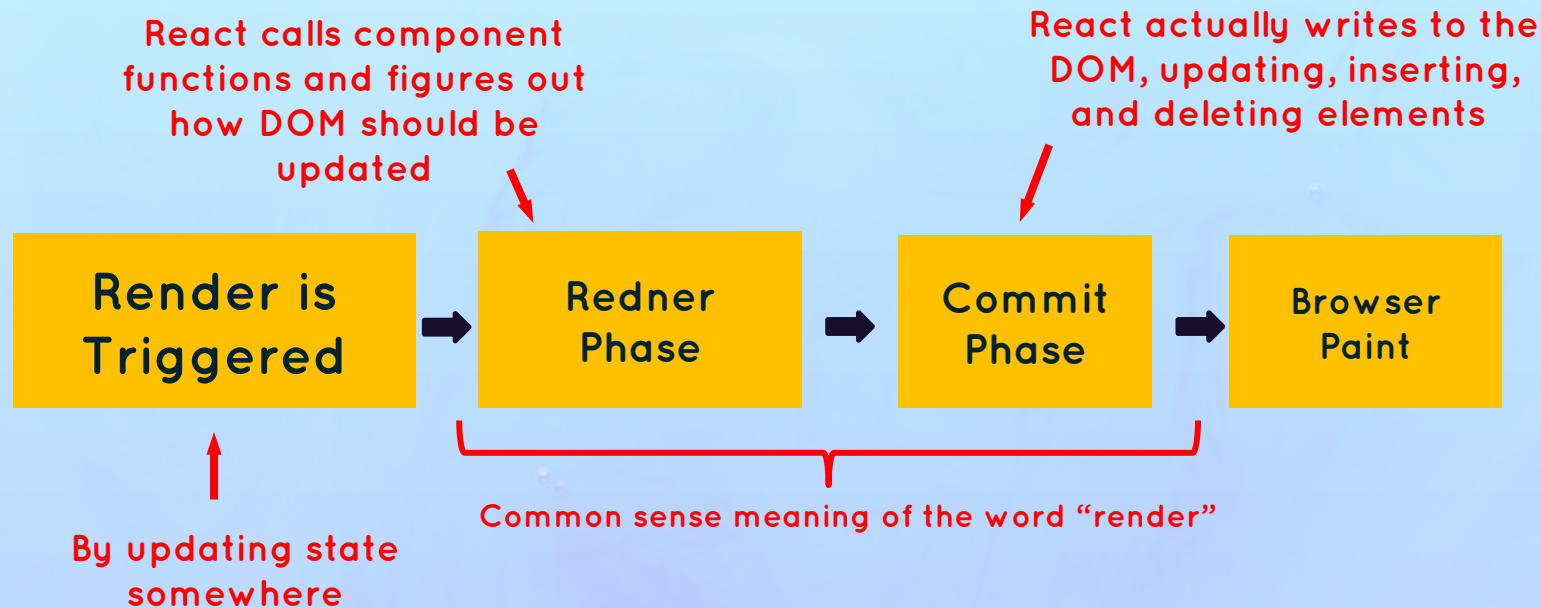
How Rendering Works: an Overview

Quick **Recap** before We Start



🤔 How does this process actually work?

Overview: How Components Are **Displayed** on The Screen



In React, rendering is **NOT** updating the DOM or displaying elements on the screen. **Rendering** only happens **internally** inside React, it does not **produce visual changes**.

How Renders Are Triggered

[1] Render Is Triggered

THE TWO SITUATIONS THAT TRIGGER RENDERS:

- 1 **Initial render** of the application
- 2 **State is updated** in one or more component instances (**re-render**)

- 👉 The render process is triggered for the **entire application**
- 👉 **In practice**, it looks like React only re-renders the component where the state update happens, but that's not how it **works behind the scenes**
- 👉 Renders are **not** triggered immediately, but **scheduled** for when the JS engine has some "free time". There is also batching of multiple setState calls in event handlers

A React & Next Ultimate Course

(In Hindi)

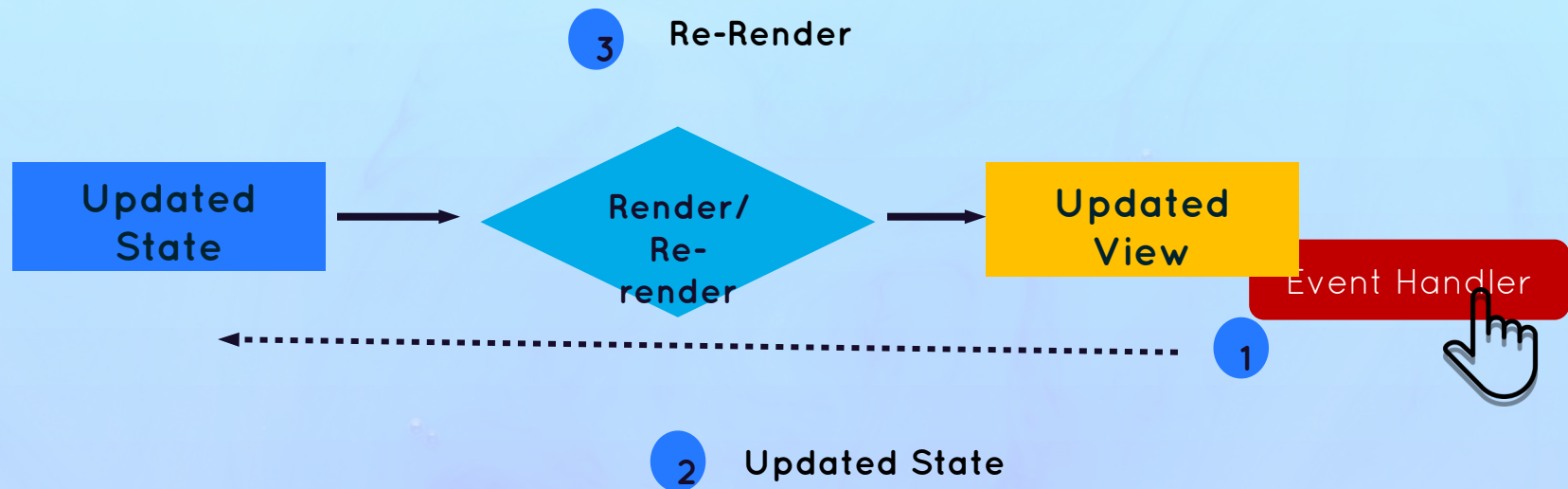
Section

How React Works In Background

Lecture

How Rendering Works: The Render Phase

Review: The Mechanics Of State In React

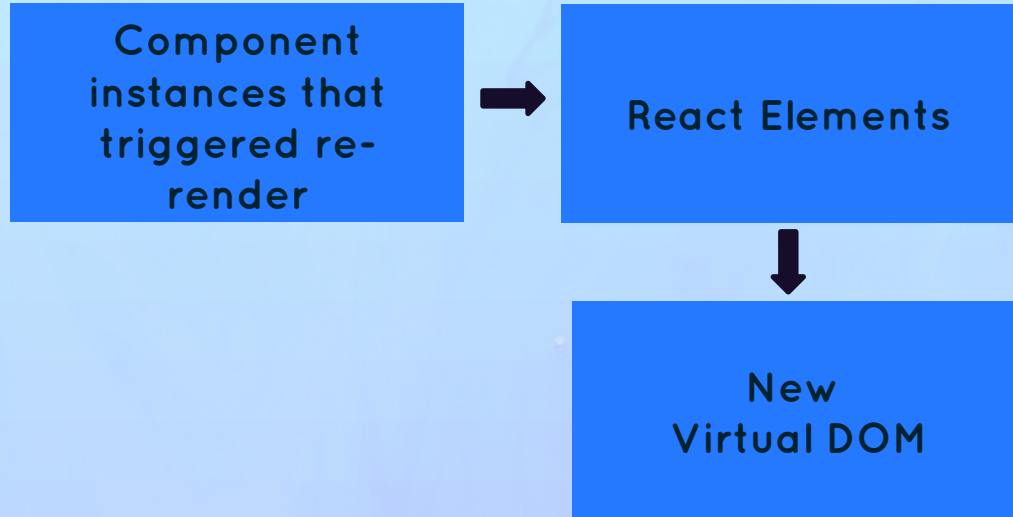


NOT TRUE #1: RENDERING IS UPDATING THE SCREEN / DOM

NOT TRUE #2: REACT COMPLETELY DISCARDS OLD VIEW (DOM) ON RE-RENDER

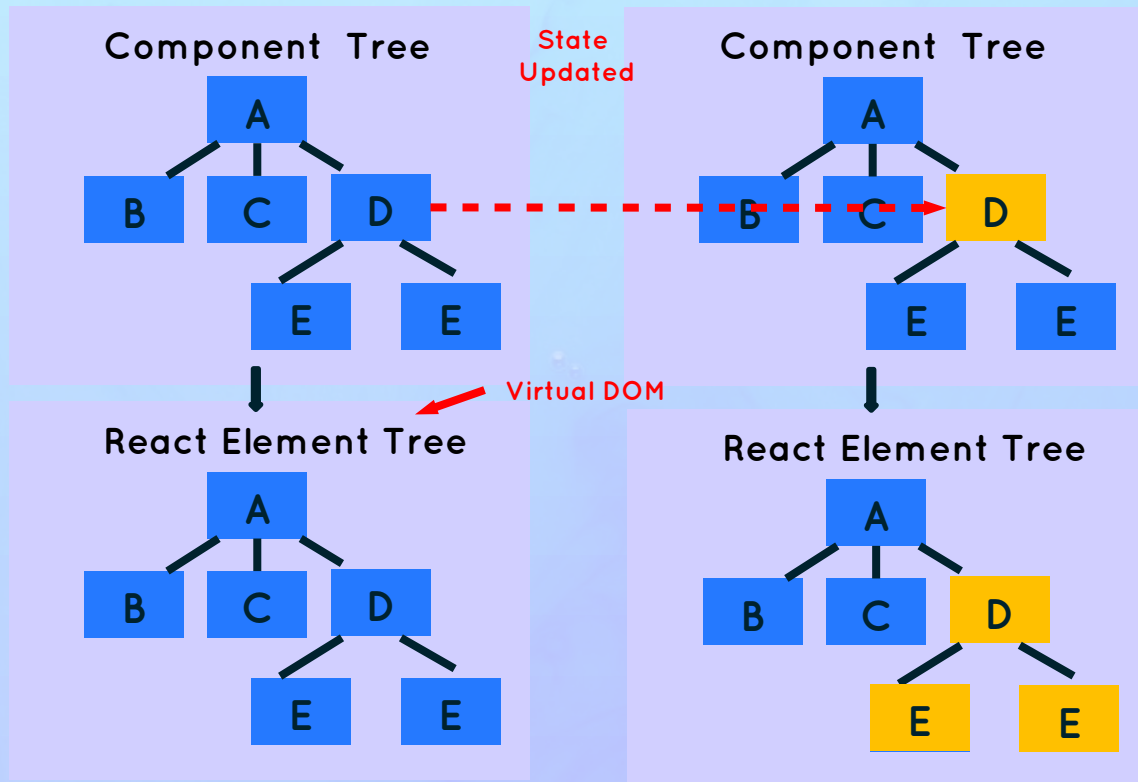
The Render Phase

[2] Render Phase



The Virtual DOM (React Element Tree)

1 Initial Render → Re-Render



Virtual DOM: Tree of all React elements created from all instances in the component tree



Cheap and fast to create multiple trees



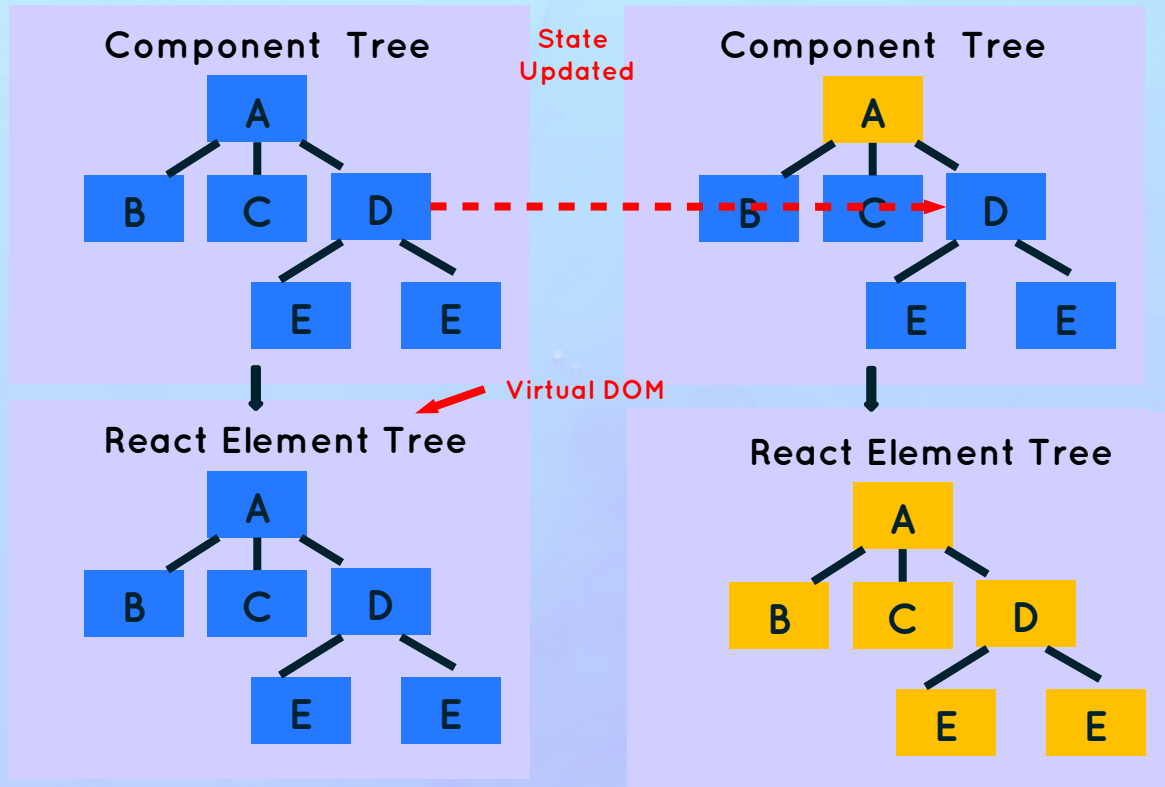
Nothing to do with “shadow DOM”



Rendering a component will **cause all of its child components to be rendered as well** (no matter if props changed or not)

The Virtual DOM (React Element Tree)

1 Initial Render → Re-Render



Virtual DOM: Tree of all React elements created from all instances in the component tree



Cheap and fast to create multiple trees



Nothing to do with “shadow DOM”



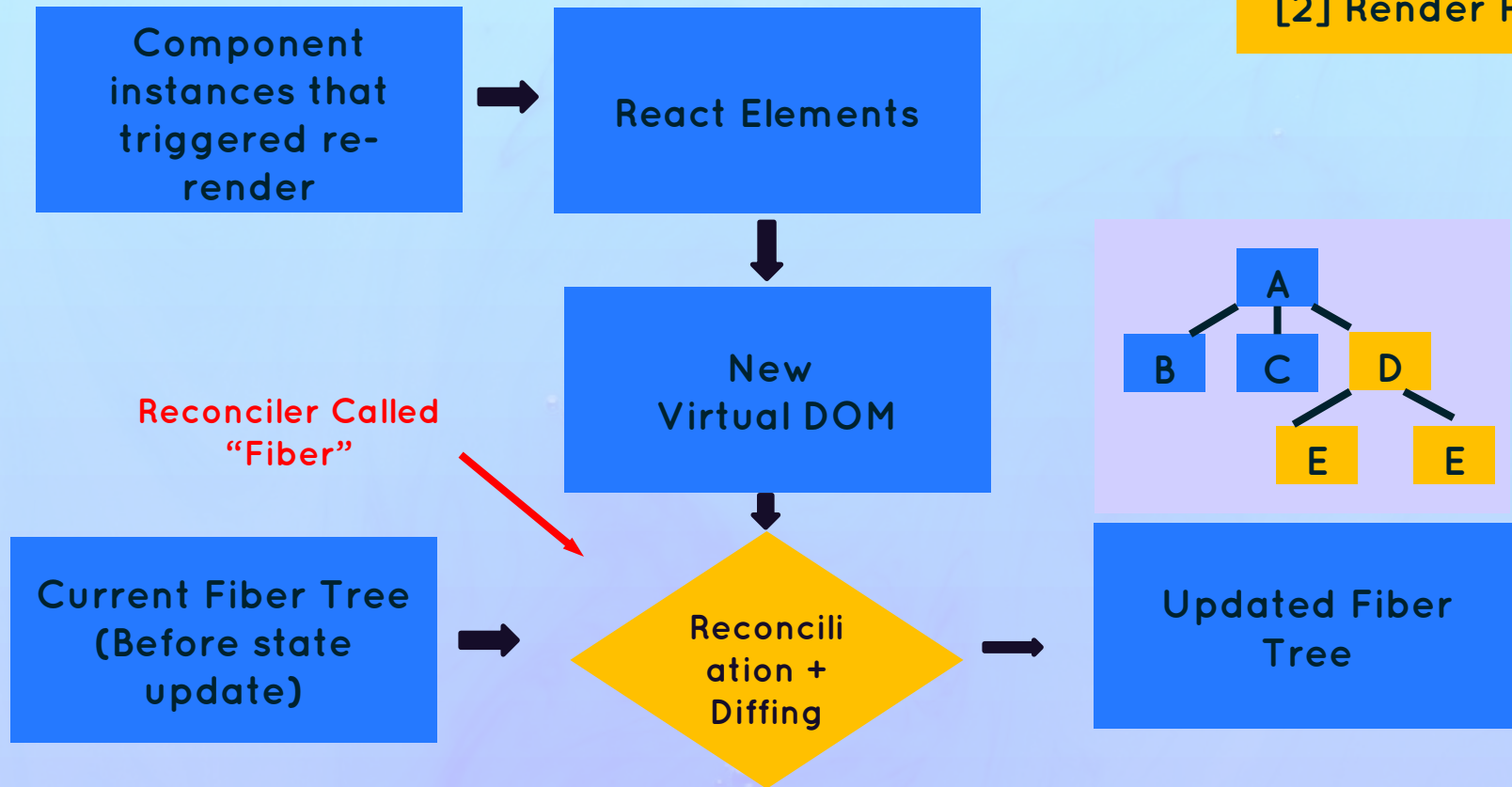
Rendering a component will cause all of its child components to be rendered as well (no matter if props changed or not)



Necessary because React doesn't know whether children will be affected

The Render Phase

[2] Render Phase



What is Reconciliation and Why Do We Need It?

👉 Why not update the entire DOM whenever state changes somewhere in the app?

↓ Because

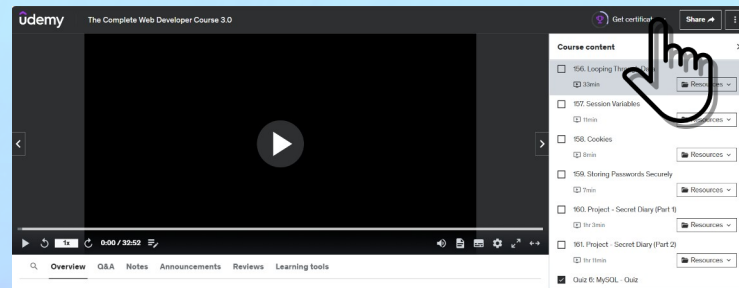
👉 That would be inefficient and wasteful:

- 1 Writing to the DOM is (relatively) **slow**
- 2 Usually only a **small Level of the DOM** needs to be updated

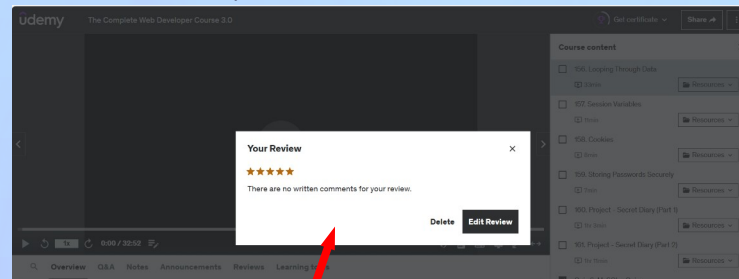
👉 React **reuses** as much of the existing DOM as possible

↓ How

♥ **Reconciliation:** Deciding which DOM elements actually need to be **inserted, deleted, or updated**, in order to reflect the latest state changes



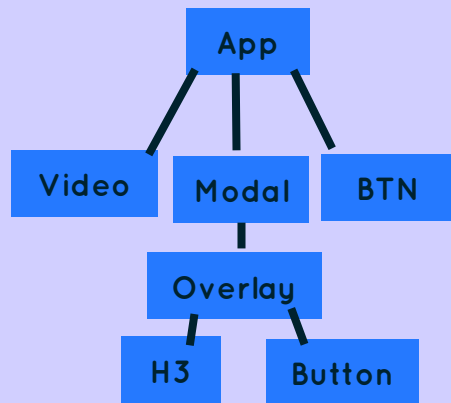
↓ showModal = true



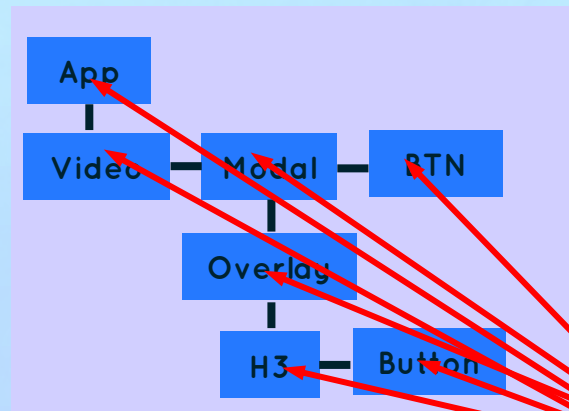
Only these new DOM elements are created

The Reconciler : Fiber

React
Element
Tree
(Virtual
DOM)



On Initial
Render



Fiber
Tree

Fiber
"Unit of work"

Current State

Props

Side Effects

Used Hooks

Queue of Work

- 👉 **Fiber tree:** internal tree that has a "fiber" for each component instance and DOM element
- 👉 Fibers are **NOT** re-created on every render
- 👉 Work can be done **asynchronously**



Rendering process can be split into "Unit of work" chunks, tasks can be prioritized, and work can be **paused, reused, or thrown away**

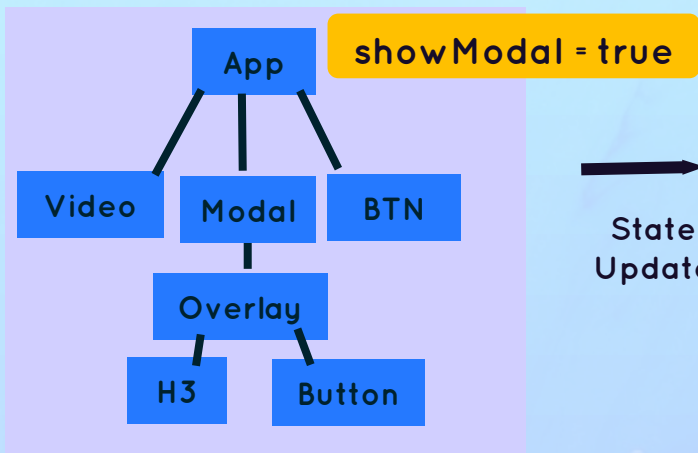


Enables **concurrent features** like Suspense or transitions

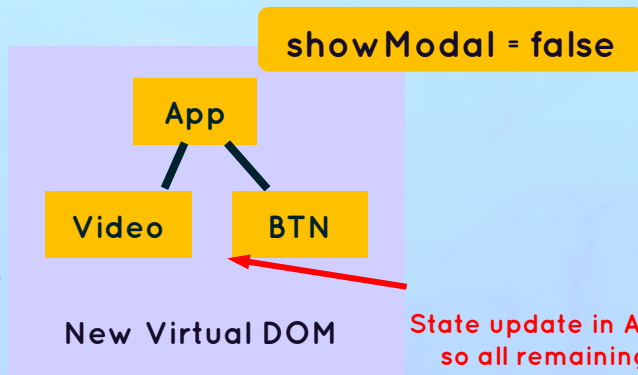


Long renders won't block JS engine

Reconciliation In Action

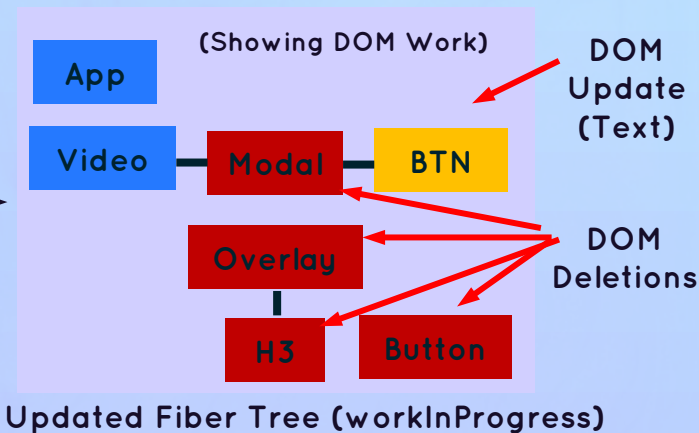
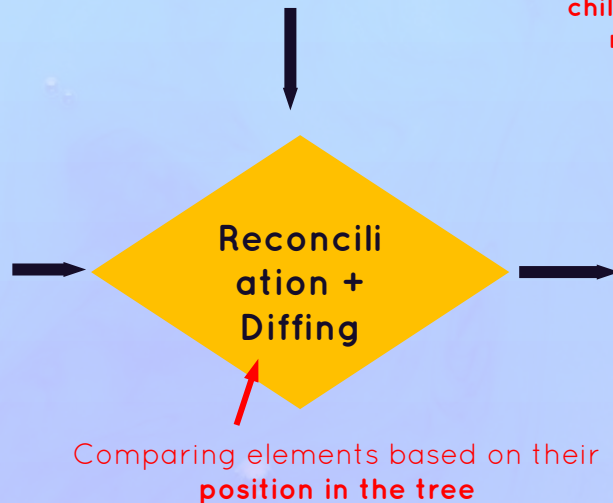
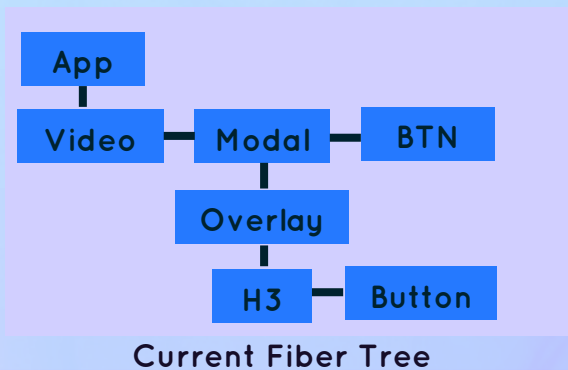


State Update

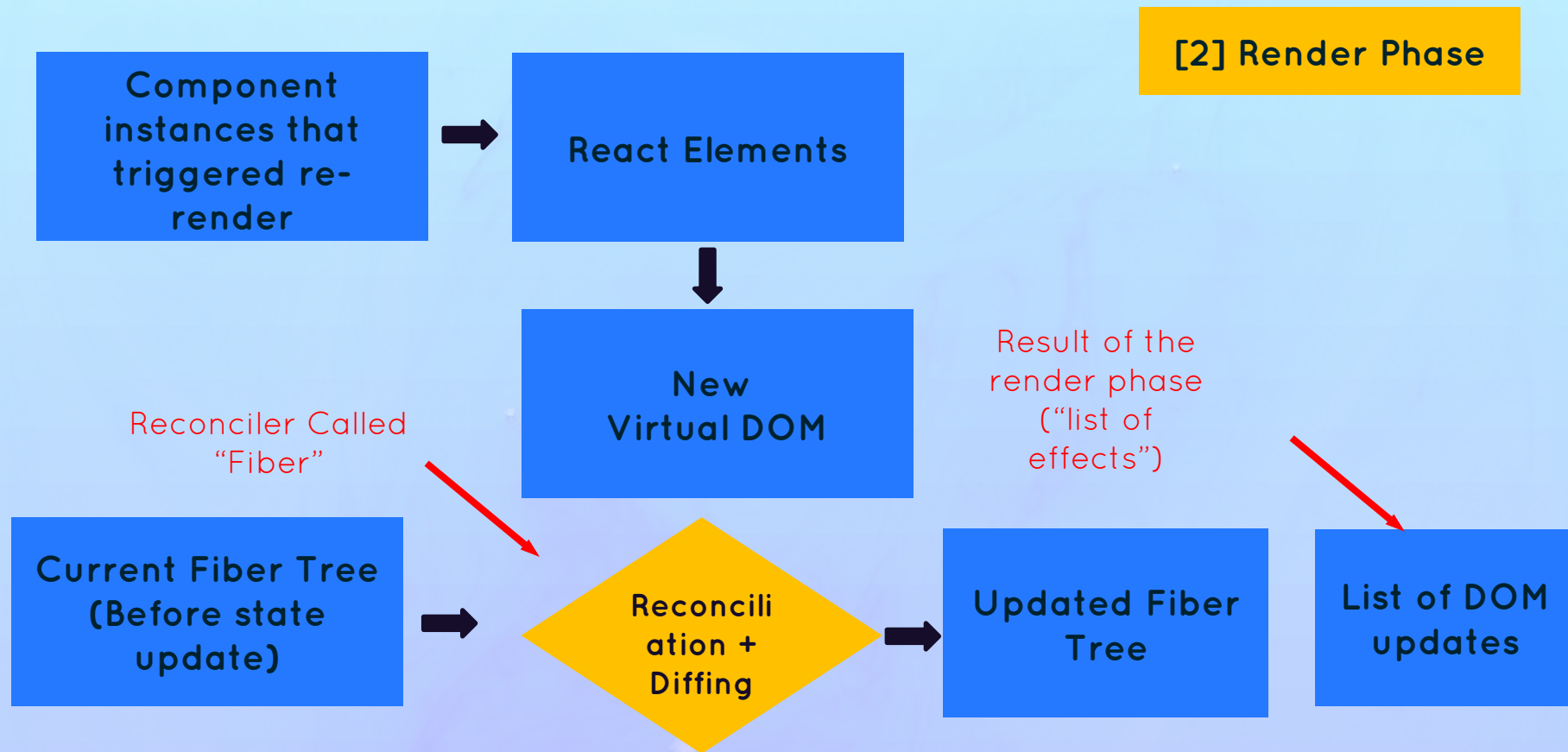


State update in App, so all remaining children are re-rendered

```
<App>
  <video />
  {showModal} && (
    <Modal>
      <Overlay>
        <h3>Rate Course!</h3>
        <button>5 ⭐</button>
      </Overlay>
    </Modal>
  )
  <BTN>{showModal} ? "Rate" : "Hides"</BTN>
</App>;
```



The Render Phase



A React & Next Ultimate Course

(In Hindi)

Section

How React Works In Background

Lecture

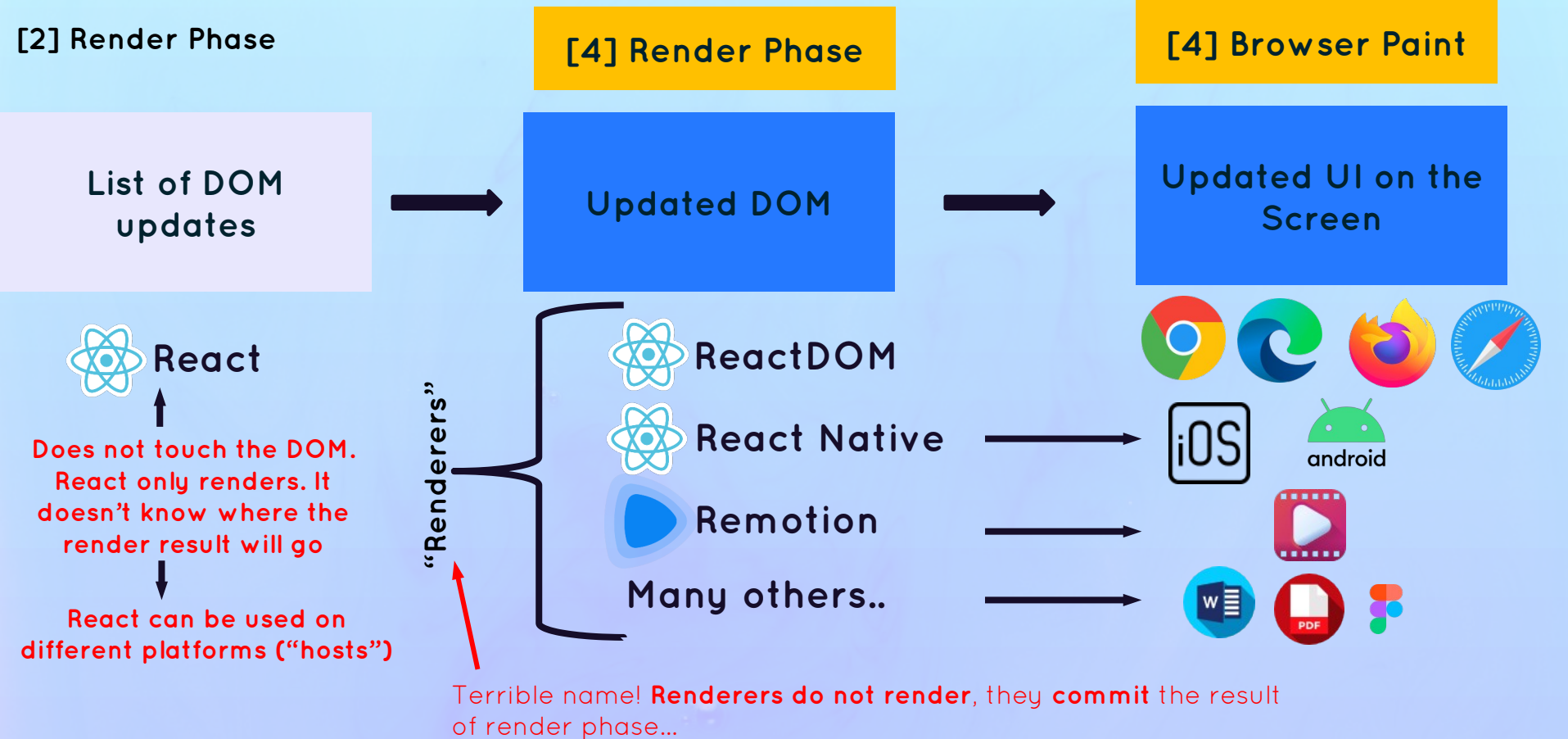
How Rendering Works: The Commit Phase

The Commit Phase and Browser Paint

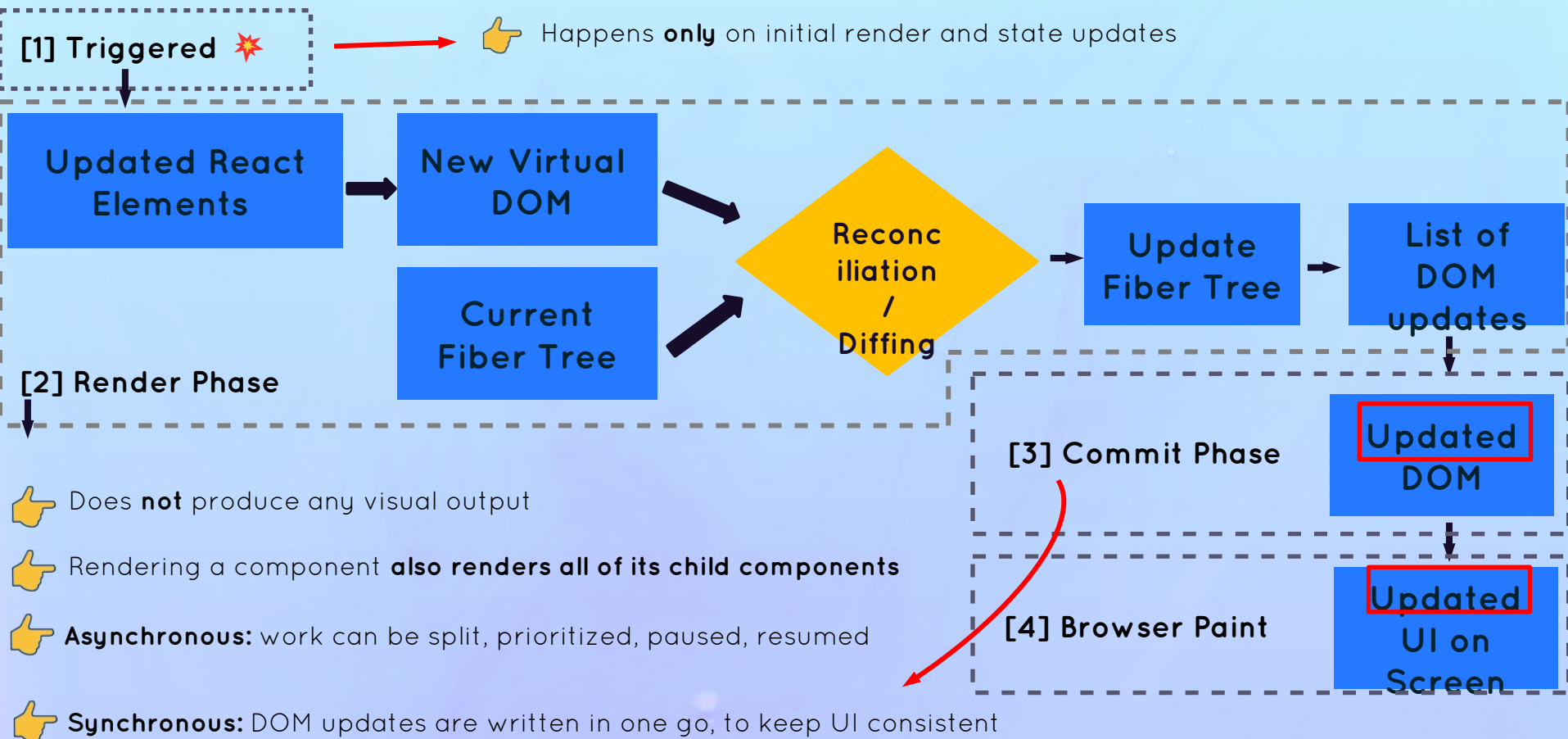


- 👉 **React writes to the DOM:** insertions, deletions, and updates (list of DOM updates are “flushed” to the DOM)
- 👉 **Committing is synchronous:** DOM is updated in one go, it can’t be interrupted. This is necessary so that the DOM never shows Levelial results, ensuring a consistent UI (in sync with state at all times)
- 👉 After the commit phase completes, the workInProgress fiber tree becomes the current tree **for the next render cycle**

The **Commit** Phase and Browser **Paint**



Recap: Putting All Together



A React & Next Ultimate Course

(In Hindi)

Section

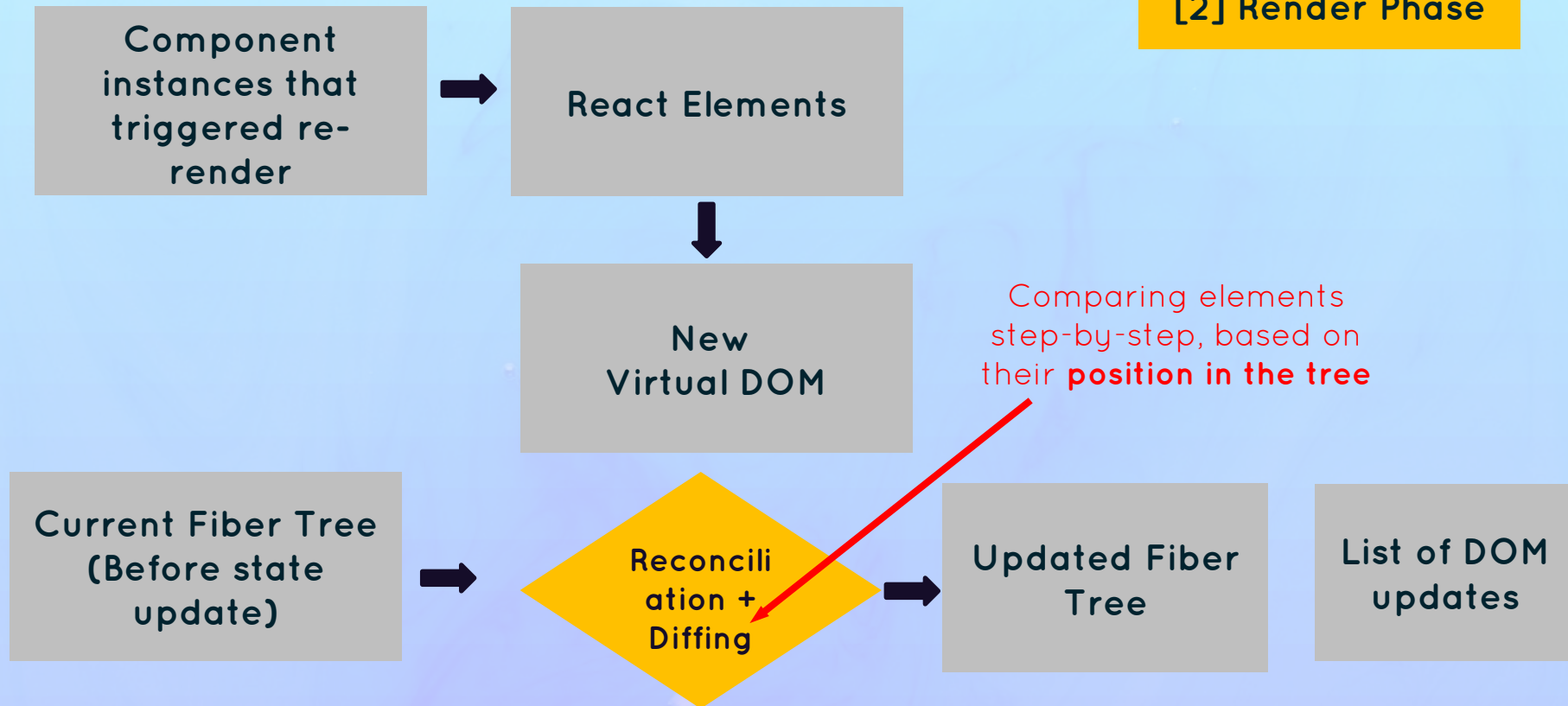
How React Works In Background

Lecture

How Diffing Works

The Render Phase

[2] Render Phase



How Diffing Works

👉 Diffing uses 2 fundamental assumptions (rules):

- 1 Two elements of different types will **produce different trees**
- 2 Elements with a stable key prop **stay the same across renders**

👉 This allows React to go from **1,000,000,000** [$O(n^3)$] to **1000** [$O(n)$] operations per 1000 elements

1 Same Position, **Different** Element

2 Same Position, **Same** Element

```
<div>  
  <searchBar />  
</div>  
<main>...</main>
```

→
Different DOM
Element

```
<header>  
  <searchBar />  
</header>  
<main>...</main>
```

```
<div>  
  <searchBar />  
</div>  
<main>...</main>
```

→
Different React
Element (Component
instance)

```
<div>  
  <ProfileView />  
</div>  
<main>...</main>
```

👉 React assumes **entire sub-tree is no longer valid**

👉 Old components are destroyed and removed from DOM, **including state**

👉 Tree might be rebuilt if children stayed the same (**state is reset**)

How Diffing Works

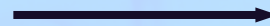
👉 Diffing uses 2 fundamental assumptions (rules):

- 1 Two elements of different types will **produce different trees**
- 2 Elements with a stable key prop **stay the same across renders**

👉 This allows React to go from **1,000,000,000** [$O(n^3)$] to **1000** [$O(n)$] operations per 1000 elements

2 Same Position, **Same** Element

```
<div className="hidden">
  <SearchBar />
</div>
<main>...</main>
```



Same **DOM**
element

```
<div className="active">
  <SearchBar />
</div>
<main>...</main>
```

```
<div>
  <SearchBar wait="{1}" />
</div>
<main>...</main>
```



Same **React**
Element(Component instance)

```
<div>
  <SearchBar wait="{5}" />
</div>
<main>...</main>
```



Element will be kept (as well as child elements), **including state**



New props / attributes are passed if they changed between renders



Sometimes this is **not** what we want... Then we can use the key prop

A React & Next Ultimate Course

(In Hindi)

Section

How React Works In Background

Lecture

The Key Prop

What is the **Key** Prop?

Key Prop

- 👉 Special prop that we use to tell the diffing algorithm that an element is **unique**
- 👉 Allows React to **distinguish** between multiple instances of the same component type
- 👉 When a key **stays the same across renders**, the element will be kept in the DOM (even if the position in the tree changes)

1 Using keys in lists

- 👉 When a key **changes between renders**, the element will be destroyed and a new one will be created (even if the position in the tree is the same as before)

2 Using keys to reset state

1. Keys in **Lists** [Stable Key]



No Keys

```
<ul>
  <Question question={q[1]} />
  <Question question={q[2]} />
</ul>;
```



Adding New List Item

```
<ul>
  <Question question={q[0]} />
  <Question question={q[1]} />
  <Question question={q[2]} />
</ul>;
```

Same elements, **but different position in tree**, so they are removed and recreated in the DOM (bad for performance)



With Keys

```
<ul>
  <Question key="q1" question={q[1]} />
  <Question key="q2" question={q[2]} />
</ul>;
```



Adding New List Item

```
<ul>
  <Question key="q0" question={q[0]} />
  <Question key="q1" question={q[1]} />
  <Question key="q2" question={q[2]} />
</ul>;
```

Different position in the tree, but the key stays the same, so the elements will be kept in the DOM



Always use keys!

2. KEY PROP TO **RESET STATE** [CHANGING KEY]



If we have the same element at the same position in the tree, the **DOM element and state** will be kept

```
<QuestionBox>
  <Question
    question={{
      title: "React Vs JS",
      body: "Why we use React ",
    }}
  />
</QuestionBox>;
```

New Question In
Same Position

```
<QuestionBox>
  <Question
    question={{
      title: "Best course for React",
      body: "an Ultimate React ",
    }}
  />
</QuestionBox>;
```

Question state (answer):

React allows us to build app faster

Question state (answer):

React allows us to build app faster

State was
preserved. NOT
what we want

2. KEY PROP TO **RESET STATE** [CHANGING KEY]



With Keys



If we have the same element at the same position in the tree, the **DOM element and state** will be kept

```
<QuestionBox>
  <Question
    question={{
      title: "React Vs JS",
      body: "Why we use React ",
    }}
    key="q12"
  />
</QuestionBox>
```

New Question In
Same Position

```
<QuestionBox>
  <Question
    question={{
      title: "Best course for React",
      body: "an Ultimate React ",
    }}
  />
</QuestionBox>
```

Question state (answer):

React allows us to build app faster

Question state (answer):

State was
RESET

A React & Next Ultimate Course

(In Hindi)

Section

How React Works In Background

Lecture

Rules For Render Logic Pure Components

The Two Types Of Logic In React Components

1. Render Logic

- 👉 Code that lives at the **top level** of the component function
- 👉 Levelicipates in **describing** how the component view looks like
- 👉 Executed **every time** the component renders

2. Event Handler Function

- 👉 Executed as a **consequence of the event** that the handler is listening for (change event in this example)
- 👉 Code that actually **does things**: update state, perform an HTTP request, read an input field, navigate to another page, etc.

```
Complexity is 9 It's time to do something...
function Question({ question }) {
  const [newAnswer, setNewAnswer] = useState("");
  const numAnswer = question.answer.length ?? 0;
  Complexity is 3 Everything is cool!
  const handleNewAnswer = function (e) {
    if (question.close) return;
    setNewAnswer(e.target.value);
  };
  Complexity is 5 Everything is cool!
  const createList = function () {
    return (
      <ul>
        {question.answer.map((q) => (
          <li>{q}</li>
        ))}
      </ul>
    );
  };
}

return (
  <div>
    <h3>{question.title}</h3>
    <p>{question.body}</p>
    {question.hasAnswer ? (
      createList()
    ) : (
      <input value={newAnswer} onChange={handleNewAnswer} />
    )}
  </div>
);
```

Refresher: Functional Programming Principles

 **Side effect:** dependency on or modification of any data outside the function scope. “Interaction with the outside world”. Examples: mutating external variables, HTTP requests, writing to DOM.

 **Side effects are not bad!** A program can only be useful if it has some interaction with the outside world

 **Pure function:** a function that has no side effects.

 Does **not** change any variables outside its scope

 Given the **same input**, a pure function always returns the **same output**

Side Effect:
Outside variable
mutation

Unpredictable
output (date
changes)



Pure Function

```
function circleArea(r) {  
  return 3.14 * r * r;  
}
```



Impure Function

```
const areas = {};  
function circleArea(r) {  
  areas.circle = 3.14 * r * r;  
}
```



Impure Function

```
function circleArea(r) {  
  const date = Date.now();  
  const area = 3.14 * r * r;  
  return `${date}: ${area}`;  
}
```

Rules For Render Logic

❓ **Components must be pure when it comes to render logic:** given the same props (input), a component instance should always return the same JSX (output)

❓ **Render logic must produce no side effects:** no interaction with the “outside world” is allowed. So, in render logic:

- ❓
 - 👉 Do NOT perform **network requests** (API calls)
 - 👉 Do NOT start **timers**
 - 👉 Do NOT directly **use the DOM API**
 - 👉 Do NOT **mutate objects or variables** outside of the function scope
 - 👉 Do NOT **update state (or refs)**: this will create an infinite loop!

This is why we
can't mutate
props!



Side effects are allowed (and encouraged) in **event handler functions!**
There is also a special hook to **register side effects** (useEffect)

A React & Next Ultimate Course

(In Hindi)

Section

How React Works In Background

Lecture

Status Update Batching

How State Updates Are **Batched**

👉 Renders are **not** triggered immediately, but **scheduled** for when the JS engine has some “free time”. There is also batching of multiple `setState` calls in event handlers

```
const [answer, setAnswer] = useState("");
const [best, setBest] = useState(true);
const [solved, setSolved] = useState(false);

const reset = function () {
  setAnswer("");
  console.log(answer);
  setBest(true);
  setSolved(false);
};

return (
  <div>
    <button onClick={reset}>Reset</button>
    { /* ... */ }
  </div>
);
```

Event
Handler
Function

How State Updates Are **Batched**

Event Handler Function

```
const reset = function ()  
  setAnswer("");  
  console.log(answer);  
  setBest(true);  
  setSolved(false);  
};
```

New State

answer = ''

best = true

solved = false

Render + Commit

Render + Commit

Render + Commit

This is NOT how React updates
multiple pieces of state in the
same event handler

How State Updates Are Batched

Event Handler Function

```
const reset = function ()  
  setAnswer("");  
  console.log(answer);  
  setBest(true);  
  setSolved(false);  
};
```

New State

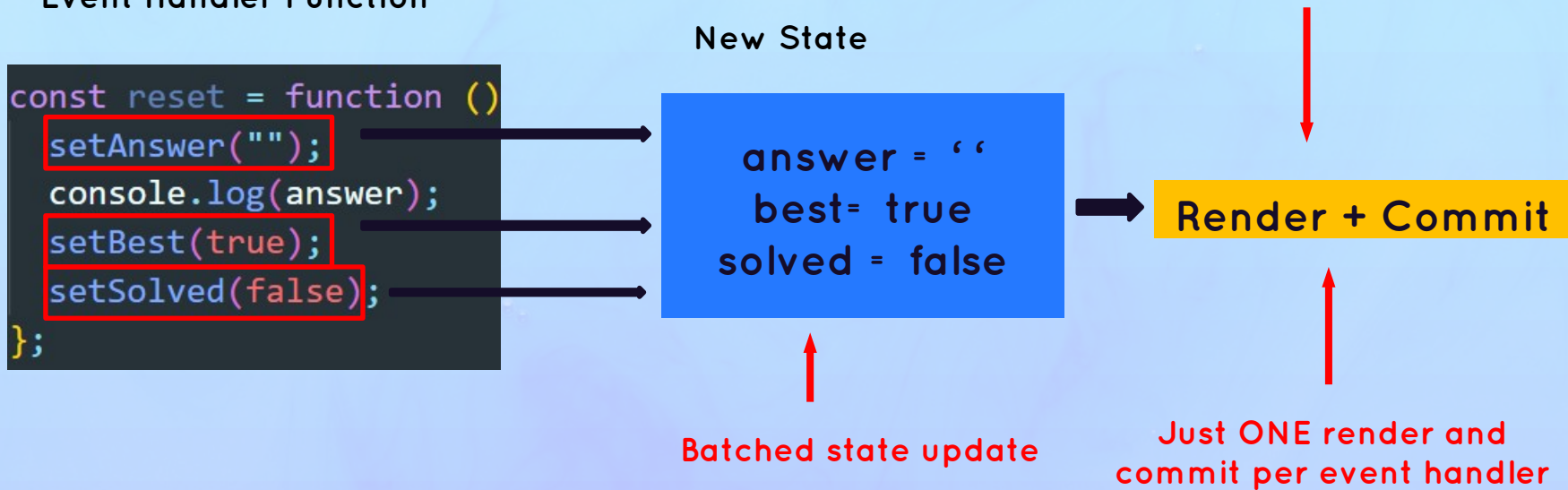
```
answer = ''  
best = true  
solved = false
```

Batched state update

No **wasted renders**, better
for performance

Render + Commit

Just ONE render and
commit per event handler



Updating State Is **Asynchronous**

Event Handler Function

```
const reset = function ()  
  setAnswer("");  
  console.log(answer);  
  setBest(true);  
  setSolved(false);  
};
```



What will the value of answer be at this point?

State is stored in the Fiber tree during
render phase



At this point, re-render has **not happened yet**



Therefore, **answer** still contains **current state**, not the updated state ('')



UPDATING STATE IN REACT IS ASYNCHRONOUS



Stale state



Updated state variables are **not** immediately available after setState call, but only after the re-render



This also applies when **only one** state variable is updated



If we need to update state **based on previous update**, we use setState with callback (setAnswer(answer=>...))

BATCHING BEYOND EVENT HANDLER FUNCTIONS



We can **opt out** of automatic batching by wrapping a state update in `ReactDOM.flushSync()` (but you will never need this)

```
const reset = function ()  
  setAnswer("");  
  console.log(answer);  
  setBest(true);  
  setSolved(false);  
};
```

We now get automatic batching at all times, everywhere



AUTOMATIC BATCHING IN..

React 17

React 18+

Event Handlers

```
<button onClick={reset}>Reset</button>
```



Timeouts

```
setTimeout(reset, 1000);
```



Promises

```
fetchStuff().then(reset);
```



Native Events

```
e1.addEventListener("click", reset);
```



A React & Next Ultimate Course

(In Hindi)

Section

How React Works In Background

Lecture

How Events Works In React

DOM REFRESHER: EVENT PROPAGATION AND DELEGATION

Event

1 Capturing Phase

- By default, event handlers listen to events on the target **and during the bubbling phase**
- We can **prevent bubbling** with `e.stopPropagation()`

2 Target Element

Event



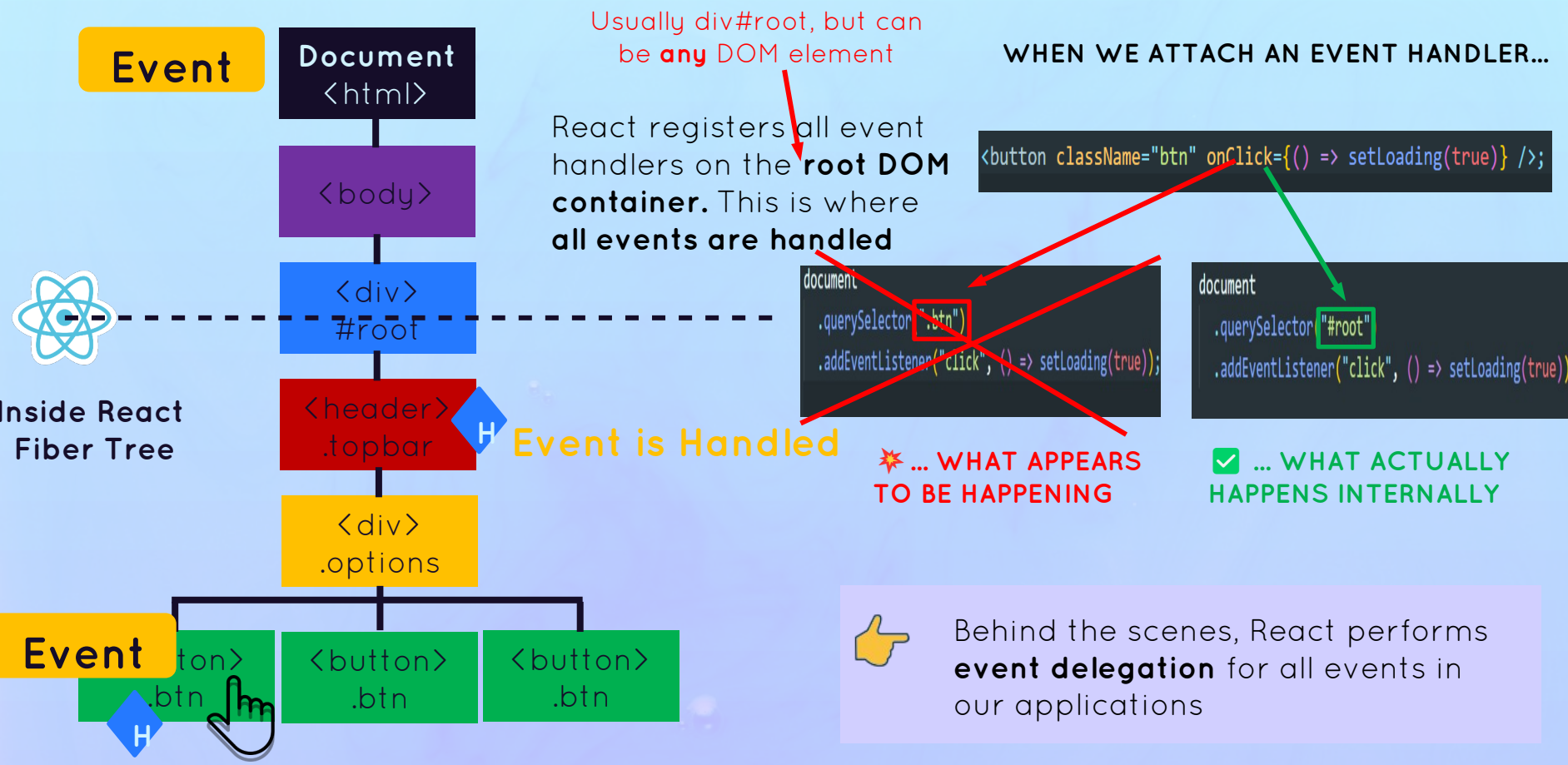
DOM tree (not Fiber tree or React element tree)

3 Bubbling Phase

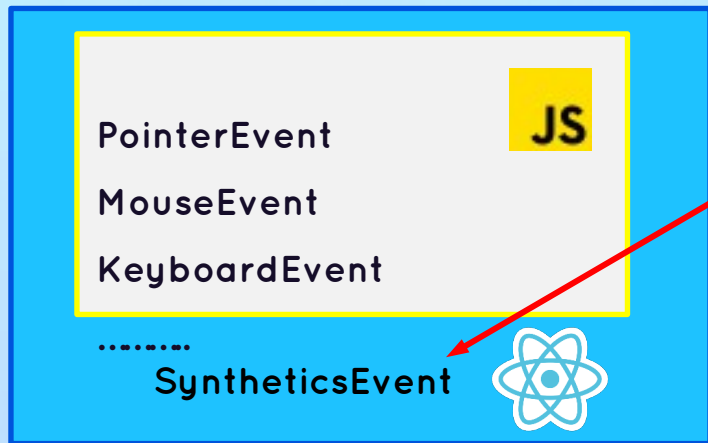
Event Delegation

- Handling events for multiple elements centrally in **one single parent element**
- Better for performance and memory, as it needs only **one handler function**
- 1 Add handler to **parent** (`.options`)
 - 2 Check for **target** element (`e.target`)
 - 3 If **target** is one of the `<button>`s, handle the event
- Very common in vanilla JS apps, but not so much in React apps

How React Handles Events



Synthetic Events



```
<input onChange={e => setText(e.target.value)} />;
```

- 👉 Wrapper around the DOM's native event object
- 👉 Has **same interface** as native event objects, like `stopPropagation()` and `preventDefault()`
- 👉 Fixes browser inconsistencies, so that events work in the exact **same way in all browsers**
- 👉 **Most synthetic events bubble** (including focus, blur, and change), except for scroll

Event
Handler In

- 👉 Attributes for event handlers are named using **camelCase** (onClick instead of onclick or click)
- 👉 Default behavior can **not** be prevented by returning false (only by using `preventDefault()`)
- 👉 Attach "Capture" if you need to handle during **capture phase** (example: `onClickCapture`)



VS



A React & Next Ultimate Course

(In Hindi)

Section

How React Works In Background

Lecture

Libraries Vs Frameworks
&
React Ecosystem

FIRST, AN ANALOGY

All In One Kit



 **Ease of mind:** All ingredients are included


 **No choice:** You're stuck with the kit's ingredients


Separate Ingredients



 **Freedom:** You can choose the best ingredients


 **Decision fatigue:** You need to research and buy all ingredients separately


FIRST, AN ANALOGY

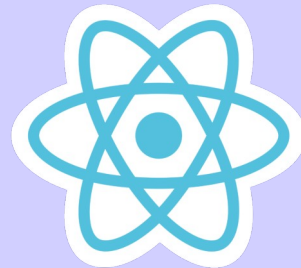
All In One Kit



❓ **Ease of mind:** All ingredients are included

❓ **No choice:** You're stuck with the kit's ingredients

Separate Ingredients



❓ **Freedom:** You can choose the best ingredients

❓ **Decision fatigue:** You need to research and buy all ingredients separately

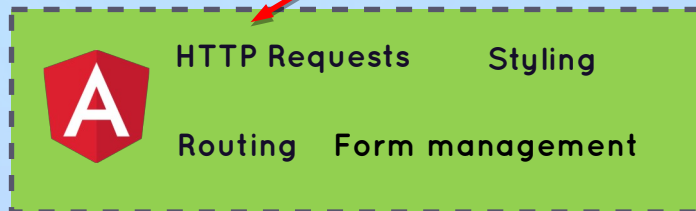
FRAMEWORK VS. LIBRARY



Framework

“All-in-one kit”

Frameworks include everything



LARGE-SCALE ANGULAR APP

❓ **Ease of mind:** Everything you need to build a complete application **is included** in the framework (“batteries included”)

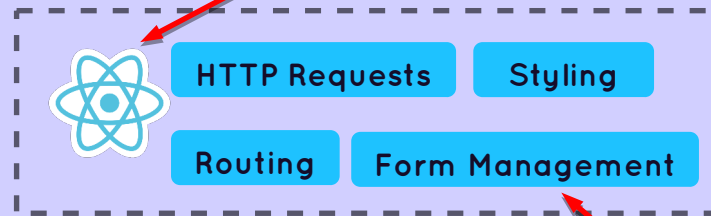
❓ **No choice:** You’re stuck with the framework’s tools and conventions (which is not always bad!)



Library

“Separate ingredients”

“View” library



LARGE-SCALE REACT APP

External libraries

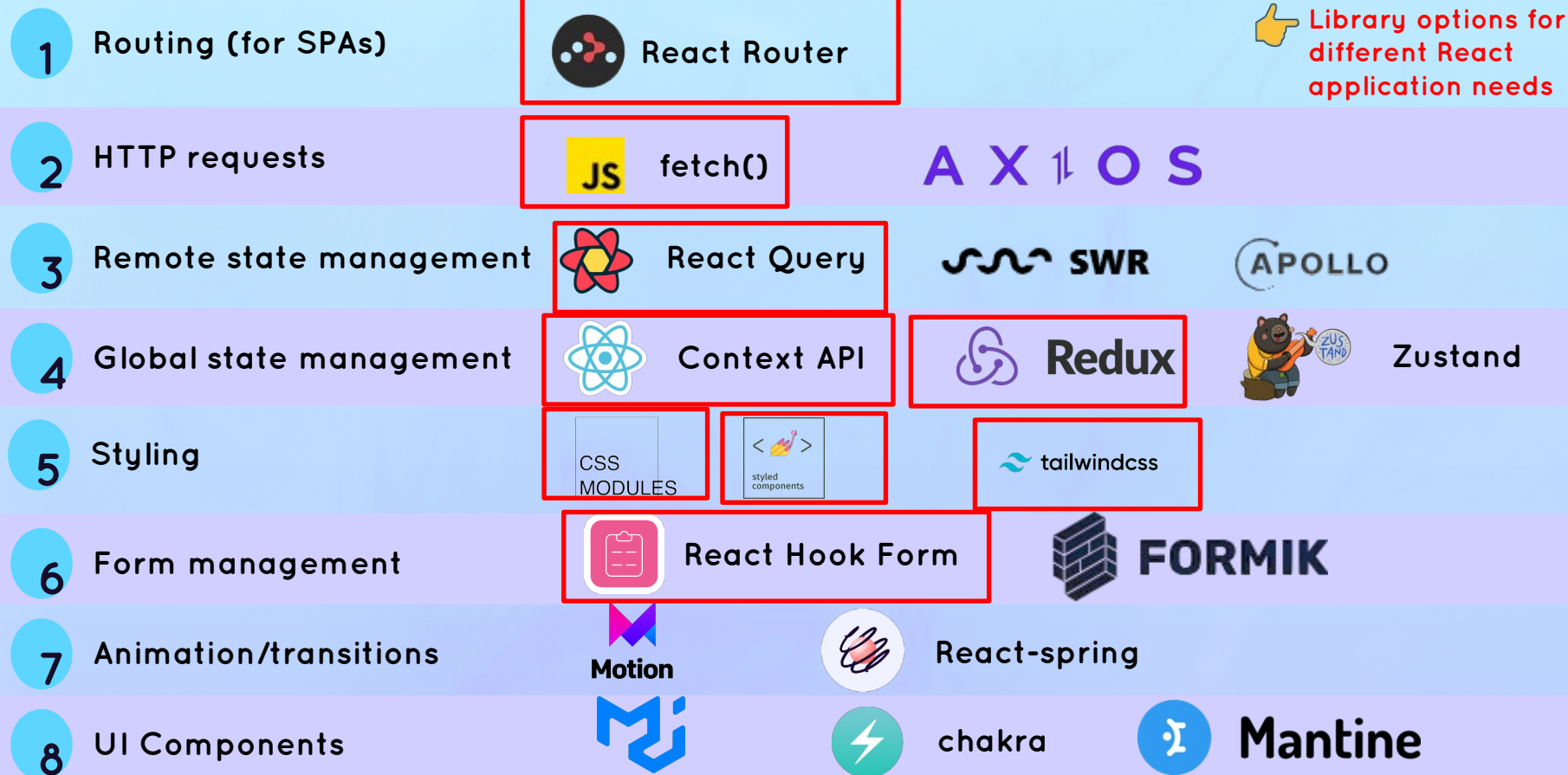
❓ **Freedom:** You can (or need to) **choose multiple 3rd-Level libraries** to build a complete application



❓ **Decision fatigue:** You need to **research, download, learn, and stay up-to-date** with multiple external libraries



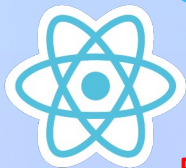
React 3rd-Level Library Ecosystem



FRAMEWORKS BUILT ON TOP OF REACT

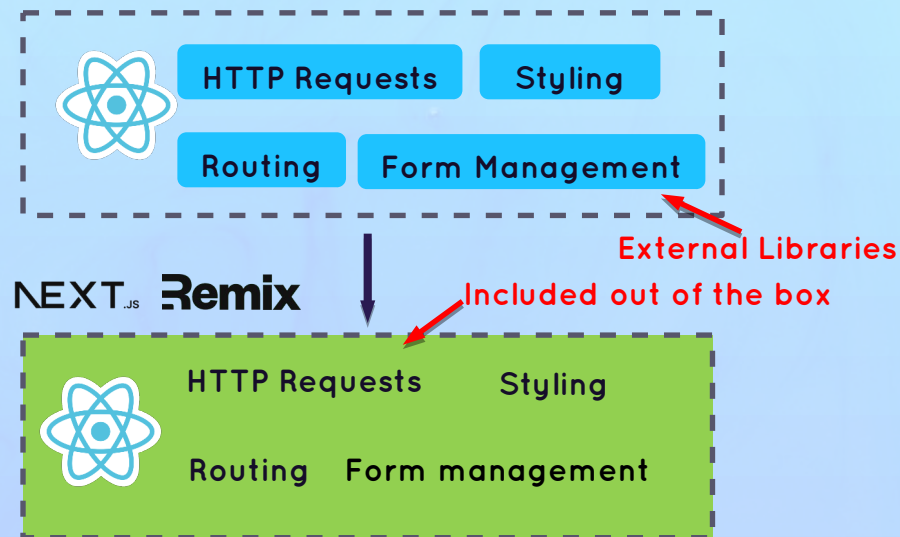
NEXT
Remix

Gatsby



Full-stack frameworks! →

“Opinionated”
React frameworks



React frameworks offer many other features: server-side rendering (SSR), static site generation (SSG), better developer experience (DX), etc.

A React & Next Ultimate Course

(In Hindi)

Section

How React Works In Background

Lecture

Section Summary: Practical Takeaways

PRACTICAL SUMMARY



A **component** is like a blueprint for a piece of UI that will eventually exist on the screen. When we “use” a component, React creates a **component instance**, which is like an actual physical manifestation of a component, containing props, state, and more. A component instance, when rendered, will return a **React element**

```
<Question />
```

```
function Question()
```



“Rendering” only means **calling component functions** and calculating what DOM elements need to be inserted, deleted, or updated. It has nothing to do with writing to the DOM. Therefore, **each time a component instance is rendered and re-rendered, the function is called again**



Only the **initial app render** and **state updates** can cause a render, which happens for the **entire application**, not just one single component



When a component instance gets re-rendered, **all its children will get re-rendered as well**. This doesn't mean that all children will get updated in the DOM, thanks to reconciliation, which checks which elements have actually changed between two renders. But all this re-rendering can still have an impact on performance (more on that later in the course 🙌)

PRACTICAL SUMMARY



Diffing is how React decides which DOM elements need to be added or modified. If, between renders, a certain React element **stays at the same position in the element tree**, the corresponding DOM element and component state will stay the same. If the element **changed to a different position**, or if it's a **different element type**, the DOM element and state will be destroyed



Giving elements a key prop allows React to distinguish between multiple component instances. **When a key stays the same across renders**, the element is kept in the DOM. This is why we need to use keys in lists. **When we change the key between renders**, the DOM element will be destroyed and rebuilt. We use this as a **trick to reset state**



Never declare a new component inside another component! Doing so will re-create the nested component every time the parent component re-renders. React will always see the nested component as **new**, and therefore **reset its state** each time the parent state is updated



The logic that produces JSX output for a component instance (“render logic”) is **not allowed to produce any side effects**: no API calls, no timers, no object or variable mutations, no state updates. **Side effects are allowed in event handlers and useEffect** (next section 🖐)

PRACTICAL SUMMARY



The DOM is updated in the commit phase, **but not by React, but by a “renderer” called ReactDOM**. That’s why we always need to include both libraries in a React web app project. We can use other renderers to use React on different platforms, for example to build mobile or native apps



Multiple state updates inside an event handler function are **batched**, so they happen all at once, **causing only one re-render**. This means we can **not access a state variable immediately after updating it**: state updates are **asynchronous**. Since React 18, batching also happens in timeouts, promises, and native event handlers.



When using events in event handlers, we get access to a **synthetic event object**, not the browser’s native object, so that **events work the same way across all browsers**. The difference is that **most synthetic events bubble**, including focus, blur, and change, which do not bubble as native browser events. Only the scroll event does not bubble



React is a library, not a framework. This means that you can assemble your application using your favorite third-Levely libraries. The downside is that you need to find and learn all these additional libraries. No problem, as you will learn about the most commonly used libraries in this course

Effects And Data Fetching

A React & Next Ultimate Course

(In Hindi)

Section

Effects And Data Fetching

Lecture

The Component Life Cycle

Component (Instance) Life Cycle



**Mount / Initial
Render**



Optional

Re-Render



Unmount



Component instance is rendered for the **first time**



Fresh state and props are **created**

Happens When:



State Change



Props Change



Parent re-render



Context Change



Component instance is **destroyed** and **removed**



State and props are **destroyed**



We can define code to run at these specific points in time

A React & Next Ultimate Course

(In Hindi)

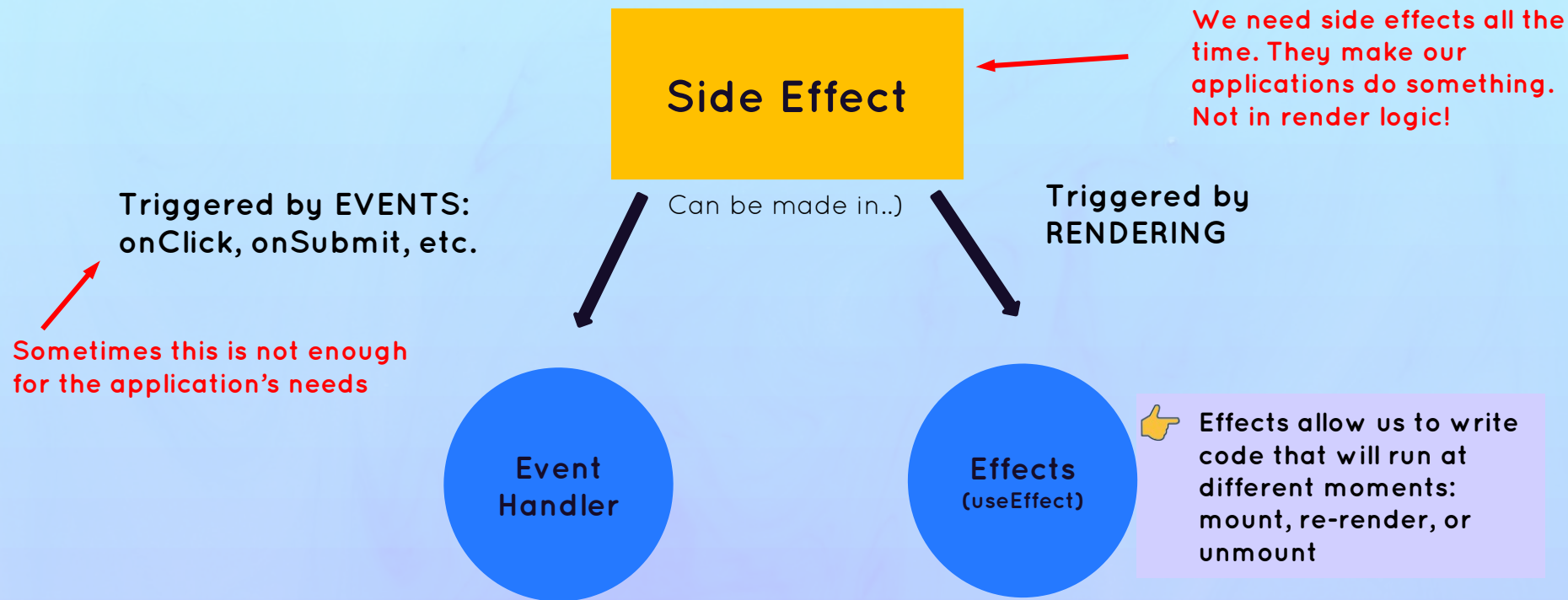
Section

Effects And Data Fetching

Lecture

A First Look At Effects

Where To Create Side Effect



REVIEW: A side effect is basically any “interaction between a React component and the world outside the component”. We can also think of a side as “code that actually does something”. Examples: Data fetching, setting up subscriptions, setting up timers, manually accessing the DOM, etc.

Event Handler VS. Effects

Event Handler

```
function handleSearch() {  
  fetch(`http://www.omdbapi.com/?apikey=${KEY}&s=Parasite`)  
    .then((res) => res.json())  
    .then((data) => setMovies(data.Search));  
}
```

Produce the same
result, but at
different moments

Effects (useEffect)

```
useEffect(function () {  
  fetch(`http://www.omdbapi.com/?apikey=${KEY}&s=Parasite`)  
    .then((res) => res.json())  
    .then((data) => setMovies(data.Search));  
  return () => console.log("CleanUp");  
}, []);
```

Effect

Cleanup
Function

Dependency array



When?

Thinking about
synchronization
, **not** lifecycles

👉 Executed when the **corresponding event happens**

👉 Used to **react** to an event



Executed **after the component mounts** (initial render), and **after subsequent re-renders** (according to dependency array)



Used to keep a component **synchronized with some external system** (in this example, with the API movie data)

👉 **Preferred way of creating side effects!**

A React & Next Ultimate Course

(In Hindi)

Section

Effects And Data Fetching

Lecture

The useEffect Dependency Array

What's The useEffect **Dependency Array**?

The Dependency Array

- 👉 By default, effects run **after every render**. We can prevent that by passing a **dependency array**
- 👉 Without the dependency array, React doesn't know **when** to run the effect
- 👉 Each time one of the dependencies changes, the effect will be executed again
- 👉 Every **state variable** and **prop** used inside the effect **MUST** be included in the dependency array

```
const title = props.movie.Title;
const [userRating, setUserRating] = useState("");

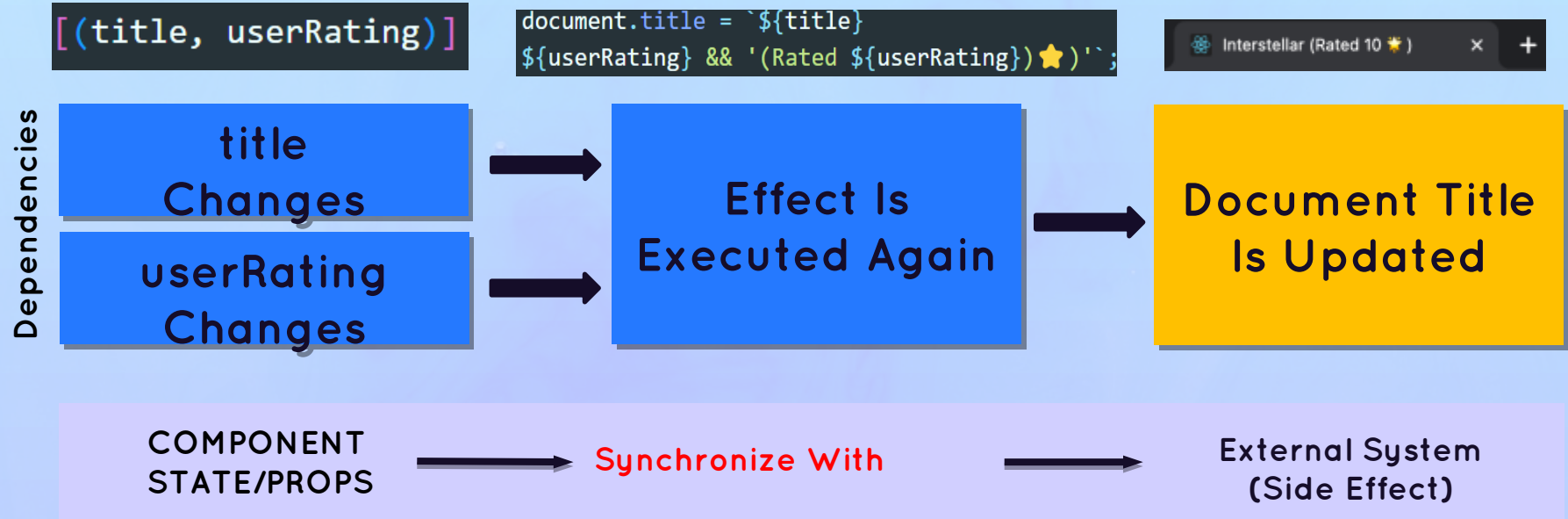
useEffect(
  Complexity is 5 Everything is cool!
  function () {
    if (!title) return;
    document.title = `${title}
    ${userRating} && '(Rated ${userRating} ★)';
    return () => (document.title = "popcornMovies");
  }, [title, userRating]
);
```

Otherwise, we get a **"stale closure"**. We will go more into depth in a future section 👉

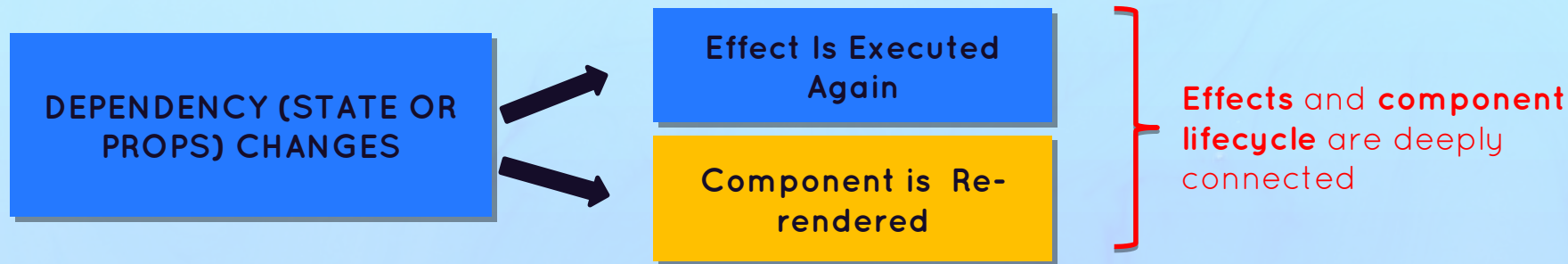
USEEFFECT IS A **SYNCHRONIZATION** MECHANISM

The Mechanics Of Effects

- 👉 `useEffect` is like an **event listener** that is listening for one dependency to change. **Whenever a dependency changes, it will execute the effect again**
- 👉 Effects **react** to updates to state and props used inside the effect (the dependencies). So **effects are “reactive”** (like state updates re-rendering the UI)



SYNCHRONIZATION AND LIFECYCLE



👉 We can use the dependency array to run effects **when the component renders or re-renders**

🔄 SYNCHRONIZATION

```
useEffect(fn, [x, y, z]);
```



Effect synchronizes with **x**, **y**, and **z**

```
useEffect(fn, []);
```



Effect synchronizes with **no state/props**

```
useEffect(fn);
```



Effect synchronizes with **everything**

🐣 Life Cycle

Runs on **mount** and **re-renders** triggered by updating **x**, **y**, or **z**

Runs only on **mount** (initial render)

Runs on **every render** (usually bad 🚫)

When Are Effects Executed?

title = 'The Shawshank Redemption'



MOUNT (INITIAL RENDER)

Commit

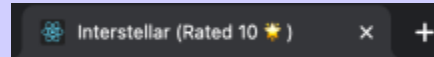
Browser Paint

Effect

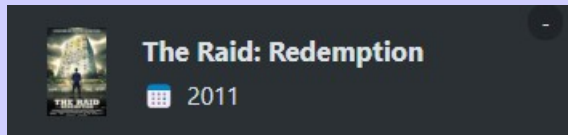
```
<MovieDetails />;
```

If an effect sets state, an **additional render** will be required

```
document.title = `${title}  
${userRating} && '(Rated ${userRating})★)';
```



title = 'The Raid: Redemption'



title Changes

Re-Render

Commit

Layout Effect

Browser Paint

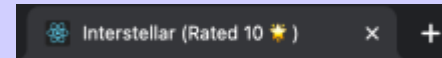


Effect

Another type of effect that is **very rarely** necessary (useLayoutEffect)

```
[(title, userRating)]
```

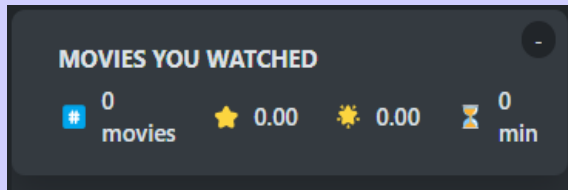
```
document.title = `${title}  
${userRating} && '(Rated ${userRating})★)';
```



Unmount



time



A React & Next Ultimate Course

(In Hindi)

Section

Effects And Data Fetching

Lecture

The useEffect Cleanup Function

When Are Effects Executed?

title = 'The Shawshank Redemption'



MOUNT (INITIAL RENDER)

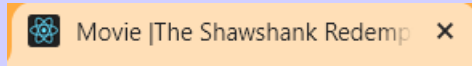
Commit

Browser Paint

Effect

```
<MovieDetails />;
```

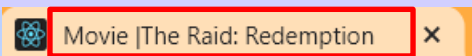
```
document.title = `${title}  
${userRating} && '(Rated ${userRating})★)';
```



```
() => (document.title = "popcornMovies");
```



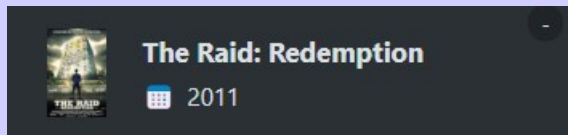
```
document.title = `${title}  
${userRating} && '(Rated ${userRating})★)';
```



```
() => (document.title = "popcornMovies");
```



title = 'The Raid: Redemption'



title Changes

Re-Render

Commit

Layout Effect

Browser Paint

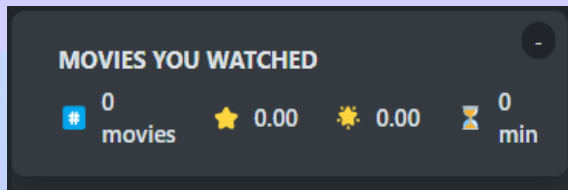
Cleanup

Effect

Unmount

Cleanup

time



The Cleanup Function

useEffect Cleanup Function

- 👉 Function that we can **return from an effect** (optional)
- 👉 Runs on two different occasions:
 - 1 Before the effect is **executed again**
 - 2 After a component has **unmounted**
- 👉 Necessary whenever the side effect **keeps happening after the component has been re-rendered or unmounted**
- 👉 Each effect should do **only one thing!** Use one **useEffect hook for each side effect**. This makes effects easier to clean up

Component
Renders



Execute effect if
dependency array
includes updated
data

Component
Unmounts



Execute
cleanup function

🌲 **Effect**

🧹 **Potential
Cleanup**

- | | | |
|----------------------|---|-----------------------|
| 👉 HTTP request | ➡ | 👉 Cancel request |
| 👉 API subscription | ➡ | 👉 Cancel subscription |
| 👉 Start timer | ➡ | 👉 Stop timer |
| 👉 Add event listener | ➡ | 👉 Remove listener |

Custom Hooks, Refs, And More State

A React & Next Ultimate Course

(In Hindi)

Section

Custom Hooks, Refs and More State

Lecture

React Hooks and Their Rules

What are React Hooks?

React Hooks

- 👉 Special built-in functions that allow us to **“hook” into React internals**:
 - 👉 Creating and accessing **state** from Fiber tree
 - 👉 Registering **side effects** in Fiber tree
 - 👉 Manual **DOM selections**
 - 👉 Many more...
- 👉 Always start with **“use”** (useState, useEffect, etc.)
- 👉 Enable easy **reusing of non-visual logic**: we can compose multiple hooks into our own **custom hooks**
- 👉 Give **function components** the ability to own state and run side effects at different lifecycle points (before v16.8 only available in class **components**)

Overview of All **Built-In** Hooks

Most Used

- ✅ useState
- ✅ useEffect
- 👉 useReducer
- 👉 useContext

👏 As of React v18.x

Less Used

- 👉 useRef
- 👉 useCallback
- 👉 useMemo
- 👉 useTransition
- 👉 useDeferredValue
- ❌ useLayoutEffect
- ❌ useDebugValue
- ❌ useImperativeHandle
- ❌ useID

Only For Library

- ❌ useSyncExternalStore
- ❌ useInsertionEffect

✅ Have **Learned**

👉 **Will** Learn

❌ Will **not** Learn

Rules of Hooks



Rules of Hooks

1

Only call hooks at the top level



Do **NOT** call hooks inside **conditionals**, **loops**, **nested functions**, or after an **early return**



This is necessary to ensure that hooks are always called in the **same order** (hooks rely on this)

2

Only call hooks from React functions

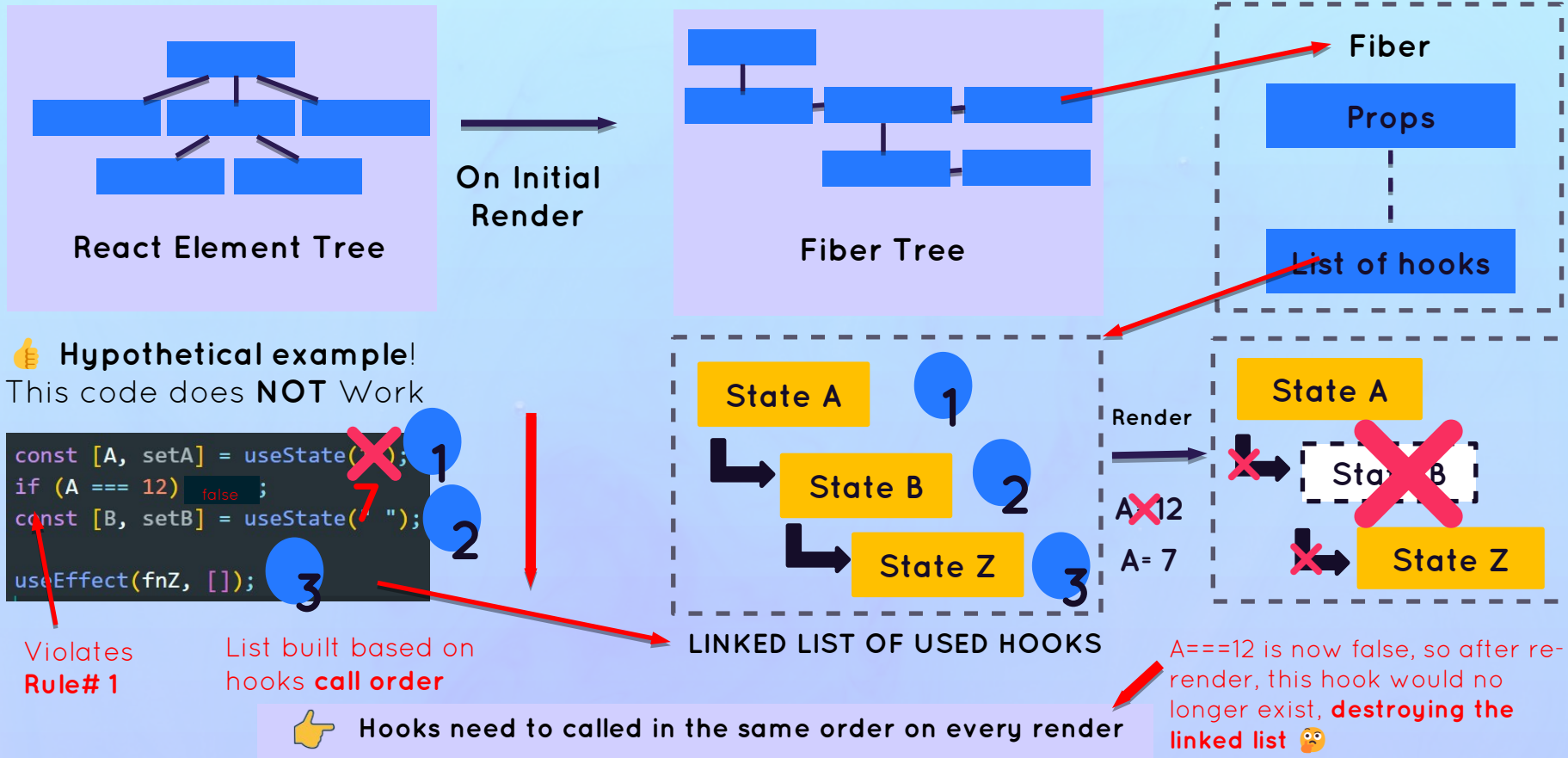


Only call hooks inside a **function component** or a **custom hook**

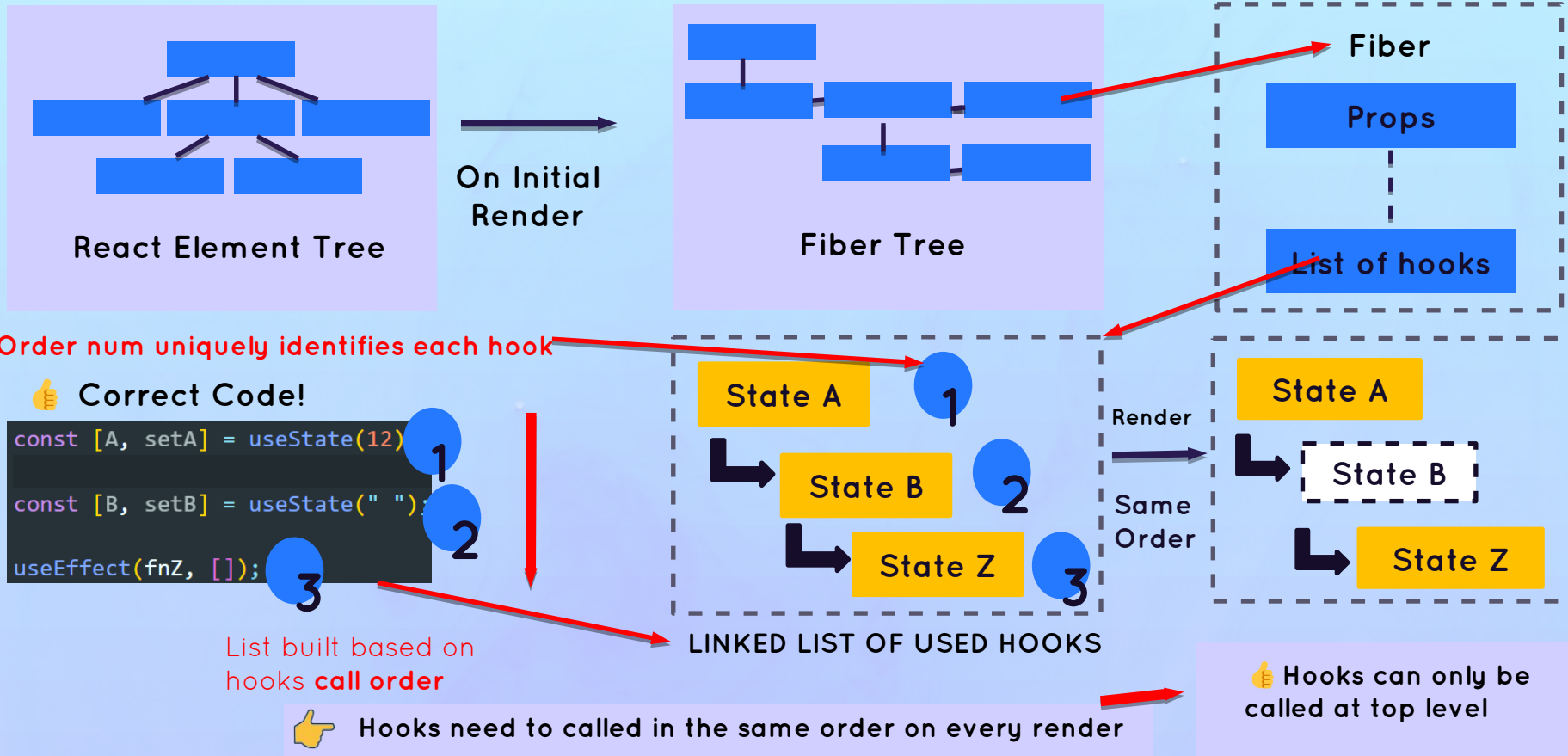


These rules are **automatically enforced** by React's ESLint rules

Hooks Rely On Call Order



Hooks Rely On Call Order



A React & Next Ultimate Course

(In Hindi)

Section

Custom Hooks, Refs and More State

Lecture

useState Summary

Summary of Defining And Updating State

1 Creating State

Simple

```
const [count, setCount] = useState(12);
```

Based on Function
(lazy evaluation)

```
const [count, setCount] = useState(() => localStorage.getItem("count"));
```



Function must be **pure** and accept **no arguments**. Called only **on initial render**

Make sure to **NOT** mutate objects or arrays, but to **replace** them

2 Updating State

Simple

```
setCount(1000);
```

Based on current State

```
setCount((c) => c + 1);
```



Function must be **pure** and return next state

A React & Next Ultimate Course

(In Hindi)

Section

Custom Hooks, Refs and More State

Lecture

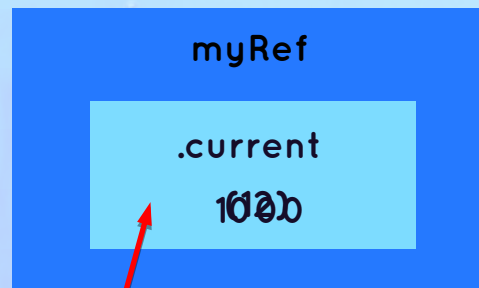
An Introduction: useRef Hook

What is Refs?

Ref with useRef

- 👉 “Box” (object) with a **mutable** `.current` property that is **persisted across renders** (“normal” variables are always reset)
- 👉 Two Big use cases:
 - 1 Creating a variable that stays the same between renders (e.g. `previous state`, `setTimeout id`, etc.)
 - 2 Selecting and storing DOM elements
- 👉 Refs are for **data that is NOT rendered**: usually only appear in event handlers or effects, not in JSX (otherwise use state)
- 👉 Do **NOT** read write or read `.current` in render logic (like state)

```
const myRef = useRef(12);
```

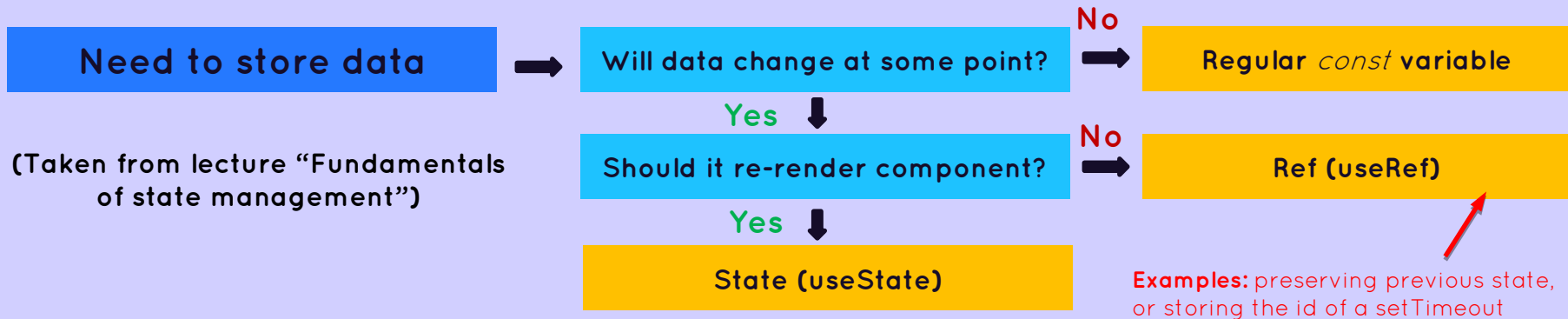


We can write to and read from the ref using `.current`

```
myRef.current = 1000;
```

State VS Refs

	Persists Across Renders	Updating Causes Re-render	Immutable	Asynchronous Update
State	✓	✓	✓	✓
Refs	✓	✗	✗	✗



A React & Next Ultimate Course

(In Hindi)

Section

Custom Hooks, Refs and More State

Lecture

What Are Custom Hooks?
When To Create?

Reusing Logic With Custom Hooks

👉 "I need to reuse:"

UI

Logic

Does logic contain any **hooks**?

No

Yes

Component

Regular
Function

Custom
Hook

👉 Allow us to reuse **non-visual logic** in multiple components

👉 One custom hook should have **one purpose**, to make it **reusable** and **portable** (even across multiple projects)

👉 **Rules of hooks** apply to custom hooks too

```
function usefetch(url) {  
  const [data, setData] = useState([]);  
  const [isLoading, setIsLoading] = useState(false);  
  Complexity is 3 Everything is cool!  
  useEffect(function () {  
    fetch(url)  
      .then((res) => res.json())  
      .then((res) => setData(res));  
  }, []);  
  return [data, isLoading];  
}
```

Function name needs to start with use

Needs to use one or more hooks

Unlike components, can receive and return any relevant data (usually [] or {})

React Before
Hooks: Class-
Based React

A React & Next Ultimate Course

(In Hindi)

Section

REACT BEFORE HOOKS: CLASSBASE

Lecture

Class Components
VS
Functional Components

Function Components **Vs.** Class Components

Exists since beginning,
but without hooks



Functional Components

Class Components

Introduced in

v16.8 (2019, with hooks)

v0.13 (2015)

How to Create

JavaScript function (any type)

ES6 class, extending `React.Component`

Reading Props

Destructuring or `props.X`

`this.props.X`

Local State

`useState` Hook

Hooks are **THE**
big difference

`this.setState()`

Side Effects/lifecycle

`useEffect` Hook

Lifecycle methods

Event handler

Functions

Class Methods

Running JSX

Return JSX from function

Return JSX from render method

Advantages

- 👉 Easier to build (less boilerplate code)
- 👉 Cleaner code: `useEffect` combines all lifecycle-related code in a single place
- 👉 Easier to share stateful logic
- 👉 We don't need **this** keyword anymore

- 👉 Lifecycle might be easier to understand for beginners

Level 3



Advanced React + Redux

Advanced useReducer Hook

A React & Next Ultimate Course

(In Hindi)

Section

The Advanced useReducer Hook

Lecture

Managing State With useReducer

Why useReducer?



STATE MANAGEMENT WITH `useState` IS NOT ENOUGH IN CERTAIN SITUATIONS:

1

When components have **a lot of state variables and state updates**, spread across many event handlers **all over the component**

2

When **multiple state updates** need to happen **at the same time** (as a reaction to the same event, like “starting a game”)

3

When updating one piece of state **depends on one or multiple other pieces of state**



IN ALL THESE SITUATIONS, `useReducer` CAN BE OF GREAT HELP

Managing State With useReducer

State With useReducer

- 👉 An alternative way of setting state, ideal for **complex state** and **related pieces of state**
- 👉 Stores related pieces of state in a **state** object Like `setState` With **superpowers**
- 👉 `useReducer` needs **reducer** function containing **all logic to update state**. Decouples state logic from component
- 👉 **reducer**: pure function (**no side effects!**) that takes current state and action, **and returns the next state**
- 👉 **action**: object that describes **how to update state**
- 👉 **dispatch**: function to trigger state updates, by “**sending**” actions from **event handlers** to the **reducer**

Instead of
`setState()`

```
const [state, dispatch] = useReducer(reducer, initialState);
```

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'inc':  
      return state + 1;  
    case 'dec':  
      return state - 1;  
    case 'setCount':  
      return action.payload;  
    default:  
      throw new Error("Unknown action type");  
  }  
}
```

How Reducer Update State?

```
const [state, dispatch] = useReducer(reducer, initialState);
```

useReducer

👉 Updating state
in a component



dispatch



Current
State

reducer

Returns



Next
State



Re-Render

Just like `array.reduce()`, reducers
accumulate ("reduce") **actions over time**

action

type = 'updateDay'
payload = 12

Object that contains information on how
the reducer should update stat

useState

setState

Update

Update State

Next(Updat
e state)



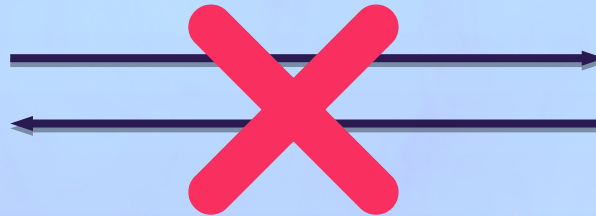
Re-Render

A Mental Model For Reducers

👉 **REAL-WORLD TASK:** WITHDRAWING ₹50,000 FROM YOUR BANK ACCOUNT



**YOU DO NOT GO TO YOUR BANK AND
TAKE MONEY STRAIGHT FROM THE BANK'S
VAULT 😊**



A Mental Model For Reducers

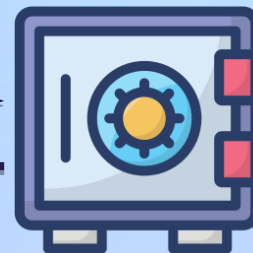
👉 **REAL-WORLD TASK:** WITHDRAWING ₹50,000 FROM YOUR BANK ACCOUNT

```
type: "withdraw", payload: { amount: 5000, account: 3577 }
```



(How to make the update)

Action



Dispatcher

(Who requests the update)

Reducer

(Who makes the update)

State

(What needs to be update)

A React & Next Ultimate Course

(In Hindi)

Section

The Advanced useReducer Hook

Lecture

useState VS useReducer

useState VS useReducer

useState

- 👉 Ideal for **single, independent pieces of state** (numbers, strings, single arrays, etc.)
- 👉 Logic to update state is placed directly in event handlers or effects, **spread all over one or multiple components**
- 👉 State is updated by **calling setState** (setter returned from useState)

- 👉 **Imperative** state updates

```
setScore(0);  
setPlaying(true);  
setTimerSec(0);
```

- 👉 **Easy** to understand and to use

useReducer

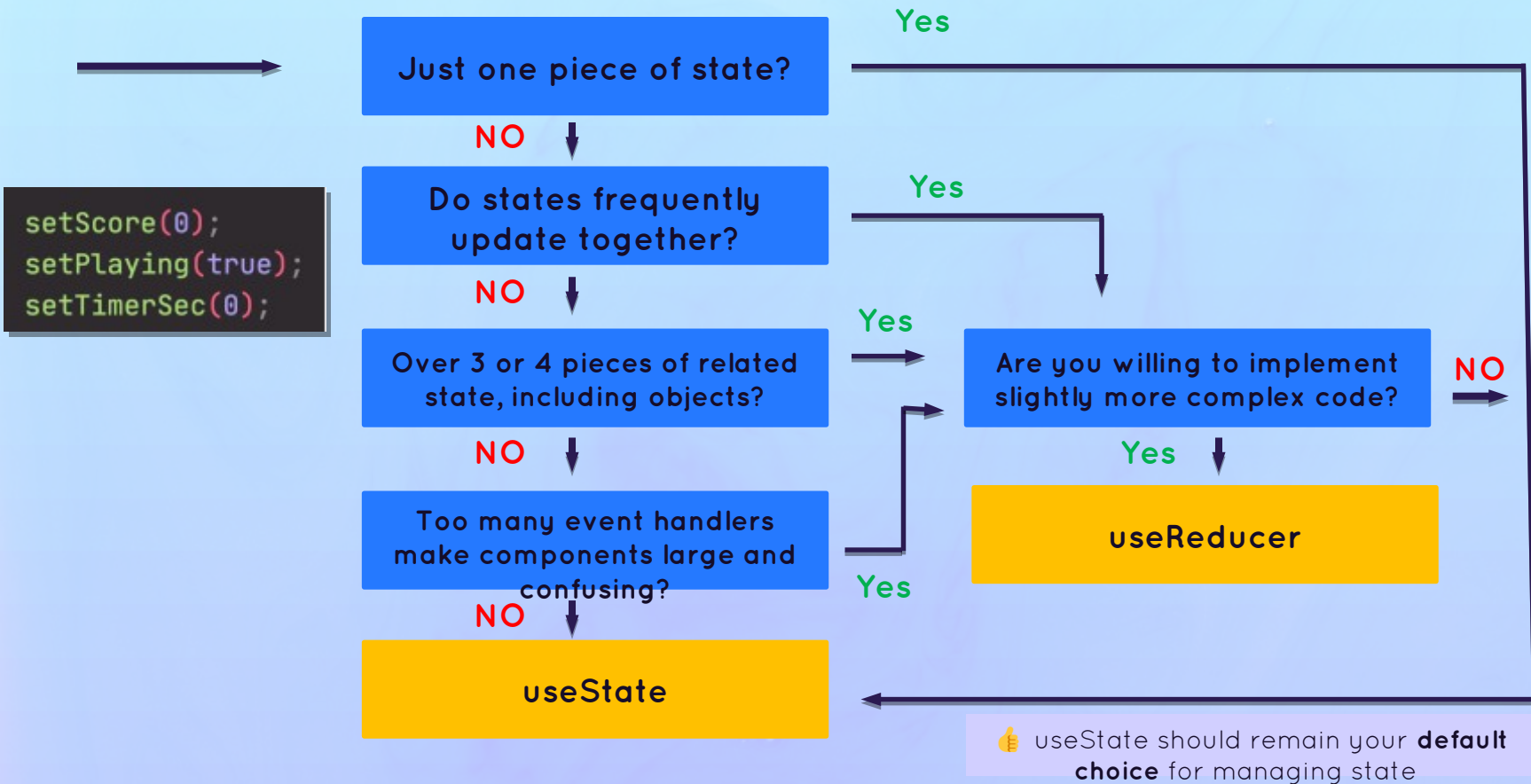
- 👉 Ideal for multiple **related pieces of state** and **complex state** (e.g. object with many values and nested objects or arrays)
- 👉 Logic to update state lives in **one central place, decoupled from components**: the reducer
- 👉 State is updated by **dispatching an action** to a reducer

- 👉 **Declarative** state updates: complex state transitions are **mapped** to actions

```
dispatch({ type: "startGame" });
```

- 👉 More **difficult** to understand and implement

When To Use useReducer?



React Router: Building Single Page Applications (SPA)

A React & Next Ultimate Course

(In Hindi)

Section

React Router : Building Single Page Application
(SPA)

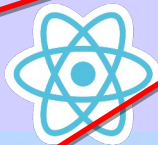
Lecture

Routing & Single-Page Application
(SPAs)

What is Routing?

Routing

- 👉 With routing, we match **different URLs** to **different UI views** (React components): **routes**
- 👉 This enables users to **navigate between different applications screens**, using the browser URL
- 👉 Keeps the UI **in sync** with the current browser URL
- 👉 Allows us to build **Single-Page Applications**

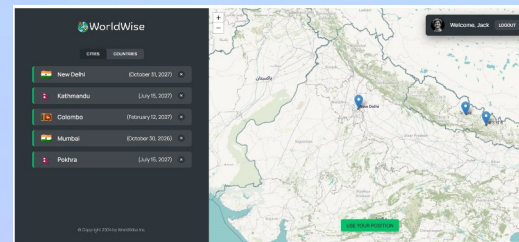
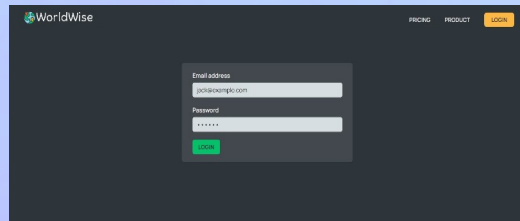
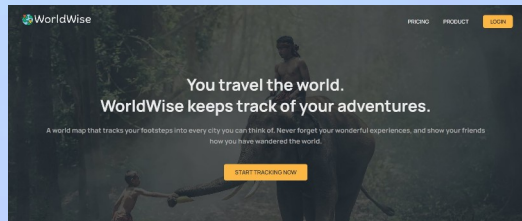


React
Router

www.example.com/

www.example.com/login

www.example.com/app

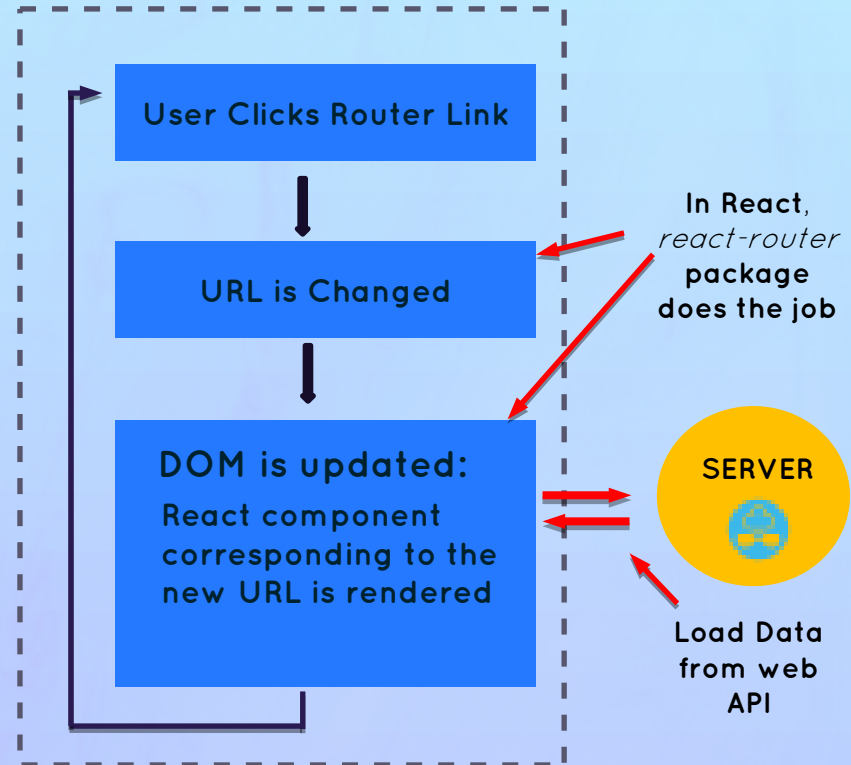


Single-Page Application(SPA)

Single-Page Application

- 👉 Application that is **executed entirely on the client** (browsers)
- 👉 **Routes**: different URLs correspond to different views (components)
- 👉 **JavaScript** (React) is used to update the page (DOM)
- 👉 **The page is never reloaded**
- 👉 Feels like a **native app**
- 👉 Additional data **might be loaded** from a web API

SPA RUNNING ON CLIENT



A React & Next Ultimate Course

(In Hindi)

Section

React Router : Building Single Page Application
(SPA)

Lecture

Styling Options For React Applications

Styling Options in React

React doesn't care about styling

Styling Options

Where

How?

Scope

Based On

Inline CSS



JSX element

Style prop

JSX element

CSS

Local

CSS or Sass File



External File

className prop

Entire App

CSS

Global cause problem

CSS Module



One external file
Per component

className prop

Component

CSS

CSS-in-JS



External file or
Component file

Create new
component

Component

JavaScript

Utility-first CSS



tailwindcss

JSX elements

className prop

JSX element

CSS



Alternative to styling with CSS : UI libraries like MUI Chakra UI, Mantine etc



Mantine

A React & Next Ultimate Course

(In Hindi)

Section

React Router : Building Single Page Application
(SPA)

Lecture

Storing State In URL

The URL For State Management

👉 The URL is an excellent place to store UI state and an alternative to useState in some situations!
Examples: open/closed panels, currently selected list item, list sorting order, applied list filters

- 1 Easy way to store state in a **global place**, accessible to **all components** in the app
- 2 Good way to “**pass**” **data** from one page into the next page
- 3 Makes it possible to **bookmark and share** the page with the exact UI state it had at the time

www.example.com/app/cities/new-delhi?lat=28.61&lng=77.23



React
Router

tools

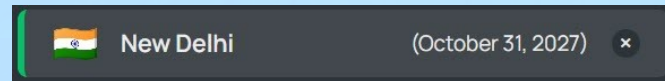
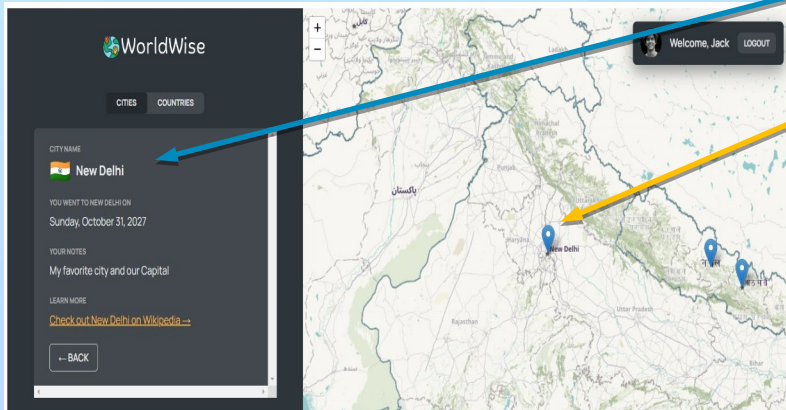
↑
Path

↑
Params

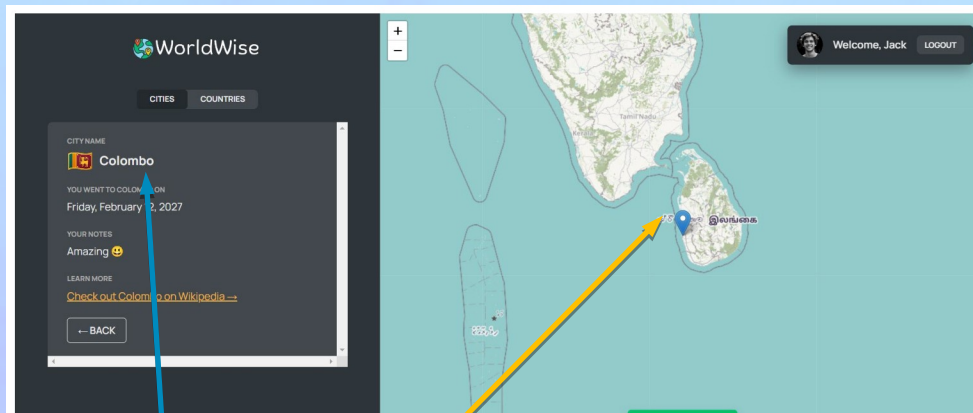
↑
Query string

Params & Query String

www.example.com/app/cities/new-delhi?lat=28.61&lng=77.23



City name and **GPS location** were retrieved from the **URL** instead of application state!



www.example.com/app/cities/colombo?lat=6.92&lng=79.36

Advanced State Management: The Context API

A React & Next Ultimate Course

(In Hindi)

Section

ADVANCED STATE MANAGEMENT: THE
CONTEXT API

Lecture

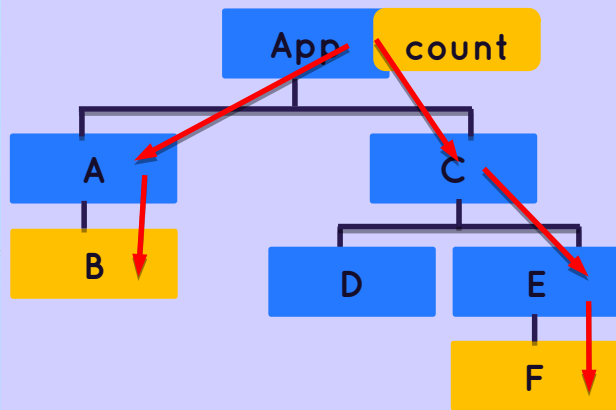
What is the Context API?

A Solution To Prop Drilling

👉 TASK: Passing state into multiple deeply nested child components

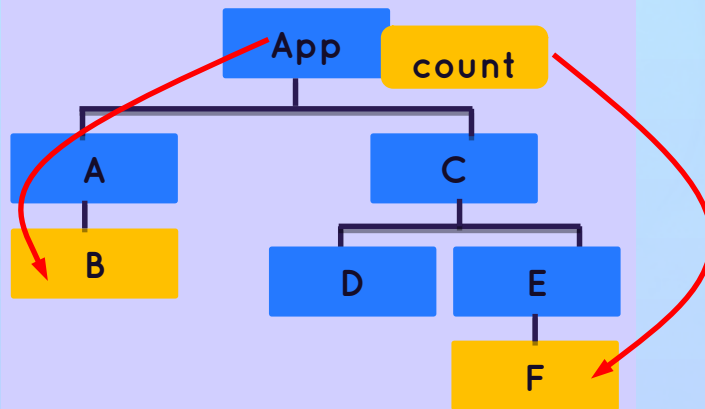
💡 Solution1: Passing Props

Components
that need
count state



❌ PROBLEM: "PROP DRILLING"

✅ Solution2: Context API



👉 READ STATE FROM EVERYWHERE

👉 Remember that a good solution to "prop drilling" is better component composition (see "Thinking in React" section)

A Solution To Prop Drilling

Context API



System to pass data throughout the app **without manually passing props** down the tree



Allows us to “**broadcast**” **global state** to the entire app

1

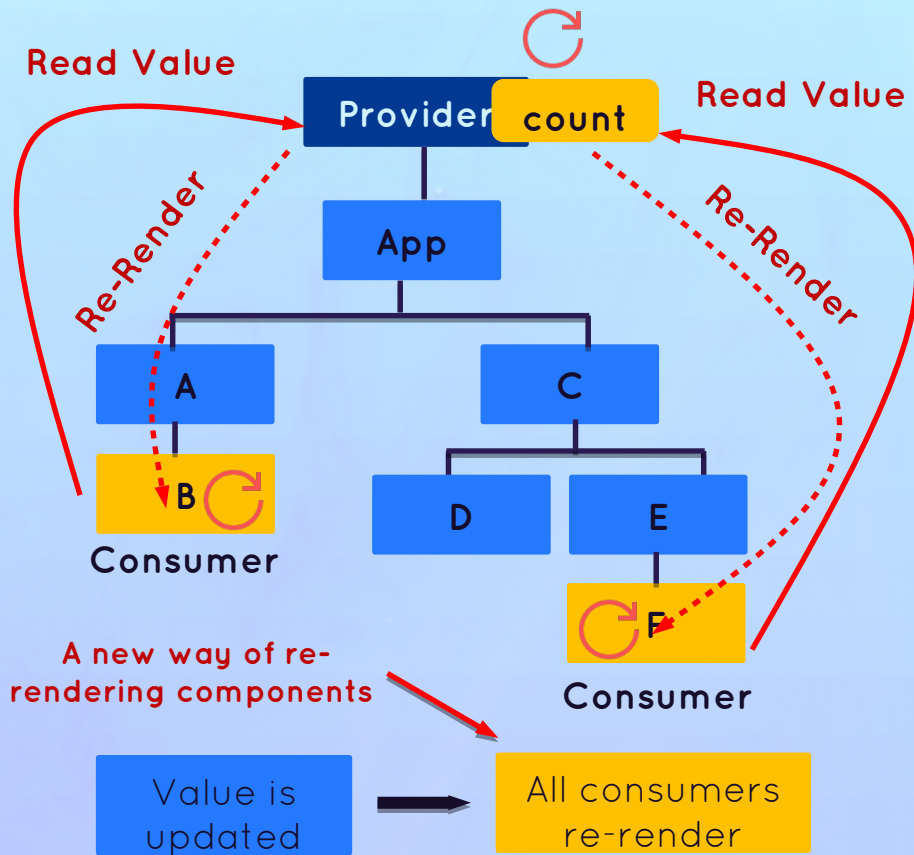
Provider: gives all child components access to value

2

Value: data that we want to make available (usually state and functions)

3

Consumers: all components that read the provided context value



A React & Next Ultimate Course

(In Hindi)

Section

ADVANCED STATE MANAGEMENT: THE
CONTEXT API

Lecture

Thinking in React : Advance State
Management

Review: What is State Management

 **State management:** Giving each piece of state the right **home**

✓ **When** to use state

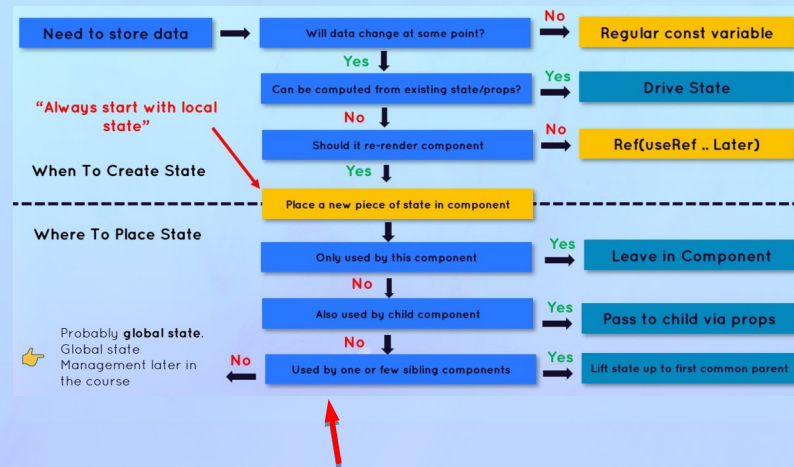
✓ **Types** of state (**accessibility**): local vs. global

👉 **Types** of state (**domain**): UI vs. remote

👉 **Where** to place each piece of state

👉 **Tools** to manage all types of state

This Lecture!



From Lecture "Fundamentals of State Management". You can keep using this 🙌

Types Of State

1

State Accessibility

“If this component was rendered twice, should a state update in one of them reflect in the other one?”

No



Local State

VS.



Global State



Needed only by **one or few component**



Only accessible in **component and child components**



Might be needed by **many components**



Accessible to **every component** in the application



2

State Domain



Remote State

VS.



UI State



All application data **loaded from a remote server** (API)



Usually **asynchronous**



Needs re-fetching + updating



Everything else 😊



Theme, list filters, form data, etc.



Usually **synchronous** and stored in the application



State Placement Options

 Where to place State	Tools 	When To Use? 
 Local component	useState, useReducer, or useRef	Local State
 Parent Component	useState, useReducer, or useRef	Lifting Up State
 Context	Context API + useState or useReducer	Global state (preferably UI state)
 3 rd Levely Library	Redux, React Query, SWR, Zustand, etc.	Global state (remote or UI)
 URL	React Router	Global state, passing between pages
 Browser	Local Storage, session Storage, etc.	Storing data in user's browser

State Management **Tools** Options

🤔 How to manage different types of state in practice?

1 State Accessibility

🏠 Local State

🌐 Global State

2 State Domain

✍️ UI State

🌐 Remote State

Mostly in small applications

👉 useState
👉 useReducer
👉 useRef

👉 fetch + useEffect +
useState / useReducer

👉 Context API + useState /
useReducer
👉 Redux, Zustand, Recoil, etc.
👉 React Router

👉 Context API + useState/useReducer
👉 Redux, Zustand, Recoil, etc.

👉 React Query
👉 SWR
👉 RTK Query

Tools highly
specialized in
handling
remote state

Performance Optimization And Advanced useEffect

A React & Next Ultimate Course

(In Hindi)

Section

Performance Optimization & Advance useEffect

Lecture

Performance Optimization
&
Wasted Renders

Performance Optimization Tools

1 Prevent Wasted Renders

- 👉 memo
- 👉 useMemo
- 👉 useCallback
- 👉 Passing element as *children* or regular prop

2 Improve App Speed Responsiveness

- 👉 useMemo
- 👉 useCallback
- 👉 useTransition

3 Reduce Bundle Size

- 👉 Using fewer Third-Level packages
- 👉 Code splitting and lazy loading

😬 This list of tools and techniques is, by no means, exhaustive. You're already doing many optimizations by following the best practices I have been showing you 🙌

When Does A Components Instance **Re-render**?

- ❓ A component instance only gets re-rendered in 3 different situations:



**State
Changes**



**Context
Changes**



**Parent
Re-render**

Creates the false impression that **changing props** re-renders a component. This is **NOT** true.



Remember: a render does not mean that the DOM actually gets updated, it just means the component function gets called. But this can be an expensive operation.



Usually no problem, as React is very fast!



Wasted render: a render that didn't produce any change in the DOM



Only a problem when they happen **too frequently** or when the **component is very slow**

A React & Next Ultimate Course

(In Hindi)

Section

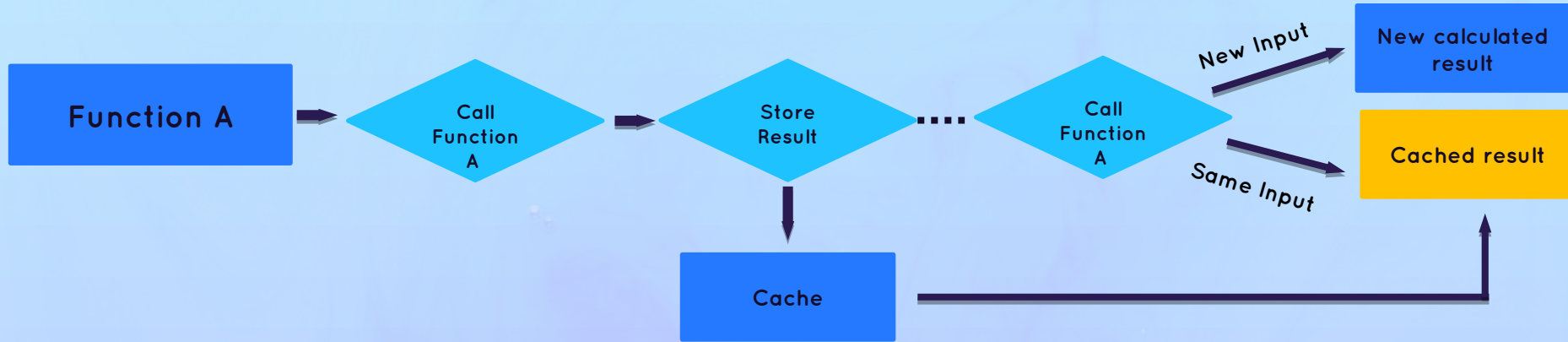
Performance Optimization & Advance useEffect

Lecture

Understanding Memo

What is Memoization?

- ❖ **Memorization:** Optimization technique that executes a pure function once and saves the result in memory. If we try to execute the function again with the **same arguments as before**, the previously saved result will be returned, **without executing the function again**.



- 👉 Memorize **components** with memo
- 👉 Memorize **objects** with useMemo
- 👉 Memorize **functions** with useCallback

1

Prevent wasted renders

2

Improve app speed/responsiveness

The Memo Function

memo

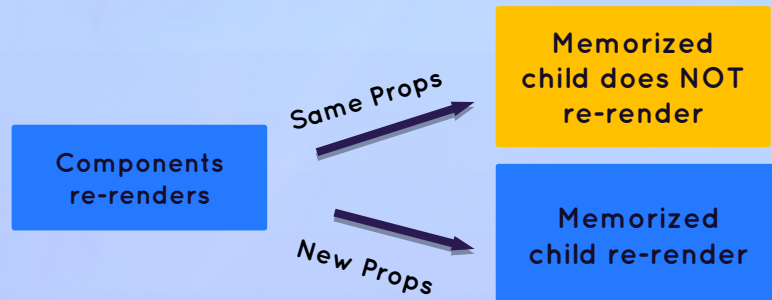
Memorized
component

- 👉 Used to create a component that will **not re-render** when its **parent re-renders**, as long as the **props stay the same between renders**
- 👉 **Only affects props!** A memorized component will still re-render when its **own state changes** or when a **context that it's subscribed to changes**
- 👉 Only makes sense when the component is **heavy** (slow rendering), **re-renders often**, and does so **with the same props**

❌ REGULAR BEHAVIOR (NO MEMO)



✅ MEMOIZED CHILD WITH MEMO



A React & Next Ultimate Course

(In Hindi)

Section

Performance Optimization & Advance useEffect

Lecture

Understanding useMemo & useCallback

An **Issue** With Memo

In React, everything is **re-created on every render** (including objects and functions)



In JavaScript, two objects or functions that look the same, **are actually different** (`{}` $\neq$ `{}`)

Therefore



If objects or functions are passed as props, the child component will always see them as **new props on each re-render**



If props are different between re-renders, memo **will not work**

Solution



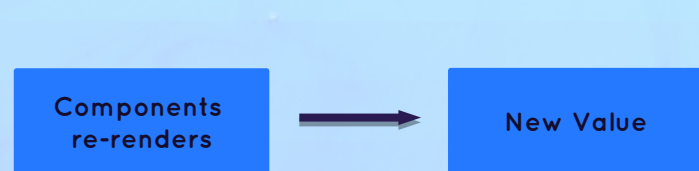
We need to memoize objects and functions, to make them stable (preserve) between re-renders (memoized `{}` $=$ memoized `{}`)

Two New Hooks: `useMemo` & `useCallback`

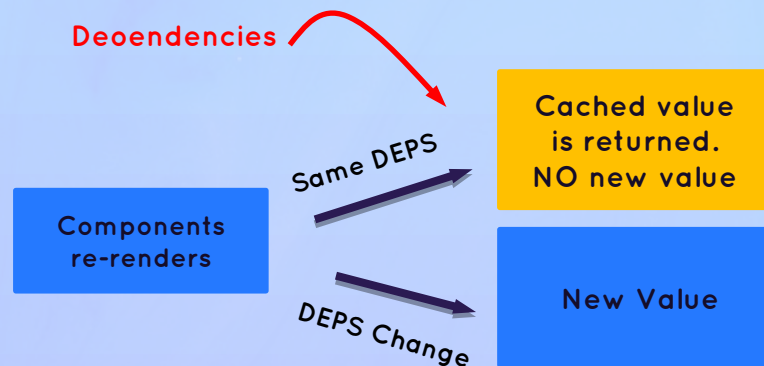
UseMemo & useCallback

- 👉 Used to memoize values (`useMemo`) and functions (`useCallback`) **between renders**
- 👉 Values passed into `useMemo` and `useCallback` will be stored in memory ("cached") **and returned in subsequent re-renders, as long as dependencies ("inputs") stay the same**
- 👉 `useMemo` and `useCallback` have a **dependency array** (like `useEffect`): whenever one **dependency changes**, the value will be **re-created**

❌ REGULAR BEHAVIOR (NO USEMEMO)



✅ MEMOIZING A VALUE WITH USEMEMO



Two New Hooks: `useMemo` & `useCallback`

UseMemo & usecallback

- 👉 Used to memoize values (`useMemo`) and functions (`useCallback`) **between renders**
- 👉 Values passed into `useMemo` and `useCallback` will be stored in memory ("cached") **and returned in subsequent re-renders, as long as dependencies ("inputs") stay the same**
- 👉 `useMemo` and `useCallback` have a **dependency array** (like `useEffect`): whenever one **dependency changes**, the value will be **re-created**
- 👉 Only use them for one of the three **use cases!**

Three Big Uses Cases:

1

Memoizing props to prevent wasted renders (together with `memo`)

2

Memoizing values to avoid expensive re-calculations on every render

3

Memoizing values that are used in dependency array of another hook

For example to avoid infinite `useEffect` loops

A React & Next Ultimate Course

(In Hindi)

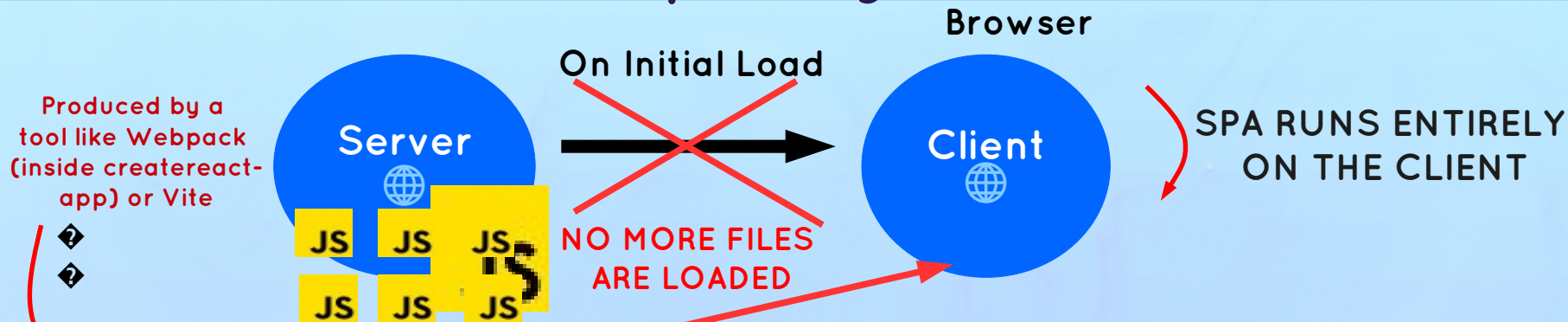
Section

Performance Optimization & Advance useEffect

Lecture

Optimizing Bundle Size With Code Splitting

The Bundle & Code Splitting



👉 **Bundle:** JavaScript file containing the **entire application code**. Downloading the bundle will load **the entire app at once**, turning it into a SPA

👉 **Bundle size:** Amount of JavaScript users have to download to start using the app. One of the most important things to be optimized, so that the bundle takes **less time to download**

👉 **Code splitting:** Splitting bundle into multiple Levels that can be **downloaded over time** ("lazy loading")

A React & Next Ultimate Course

(In Hindi)

Section

Performance Optimization & Advance useEffect

Lecture

Don't Optimize Prematurely!

Don't Optimize Prematurely!

Do

- ✓ Find performance bottlenecks using the Profiler and Visual inspection (laggy UI)
- ✓ Fix those real performance issues
- ✓ Memoize expensive re-renders
- ✓ Memoize expensive calculations
- ✓ Optimize context if it has many consumers and changes often
- ✓ Memoize context value + child components
- ✓ Implement code splitting + lazy loading for SPA routes

Don't

- ❓ Don't optimize prematurely!
- ❓ Don't optimize anything if there is nothing to optimize...
- ❓ Don't wrap all components in memo()
- ❓ Don't wrap all values in useMemo()
- ❓ Don't wrap all functions in useCallback()
- ❓ Don't optimize context if it's not slow and doesn't have many consumers

A React & Next Ultimate Course

(In Hindi)

Section

Performance Optimization & Advance useEffect

Lecture

UseEffect Rules and Best Practice

useEffect Dependency Array Rules

Dependency Array Rules

- 👉 Every **state variable**, **prop**, and context value used inside the effect **MUST** be included in the dependency array
- 👉 All “**reactive values**” must be included! That means any function or variable that reference any other reactive value
- 👉 Dependencies choose themselves: **NEVER** ignore the exhaustive-deps ESLint rule!
- 👉 Do **NOT** use **objects** or **arrays** as dependencies (objects are recreated on each render, and React sees new objects as **different**, `{ } !== { }`)

Reactive value: state, prop, or context value, or any other value that references a reactive value

```
const [nombre, setNombre] = useState(5);
const [duration, setDuration] = useState(0);
const mins = Math.floor(duration);
const secs = (duration - mins) * 60;

const formatDur = function () {
  return `${mins}:${secs < 10 ? "0" : ""}${secs}`;
};

useEffect(
  function () {
    document.title = `${nombre} -exercise workout(${formatDur()})`;
  },
  [nombre, formatDur]
);
```

All reactive values used in effect



The **same rules** apply to the dependency arrays of other hooks: `useMemo` and `useCallback`

Removing **Unnecessary** Dependencies

REMOVING FUNCTION DEPENDENCIES

- ❖ Move function **into the effect**
- ❖ If you need the function in multiple places, **memoize it** (useCallback)
- ❖ If the function doesn't reference any reactive values, move it **out of the component**
- ❖

REMOVING OBJECT DEPENDENCE

- 👉 Instead of including the entire object, include **only the properties you need** (primitive values)
- 👉 If that doesn't work, use the same strategies as for functions (**moving** or **memoizing** object)

OTHER STRATEGIES

- 👉 If you have **multiple related reactive values** as dependencies, try using a **reducer** (useReducer)
- 👉 You don't need to include setState (from useState) and dispatch (from useReducer) in the dependencies, as **React guarantees them to be stable** across renders

When **NOT** to Use An Effect



Effects should be used as a last resort, when no other solution makes sense. React calls them an “escape hatch” to step outside of React

THREE CASES WHERE EFFECTS ARE OVERUSED:

Avoid these as a beginner

1

Responding to a user event. An event handler function should be used instead

2

Fetching data on component mount. This is fine in small apps, but in real-world app, a library like React Query should be used

3

Synchronizing state changes with one another (setting state based on another state variable). Try to use derived state and event handlers

We actually do this in the current project, but for a good reason

Redux & Modern Redux Toolkit (With Thunks)

A React & Next Ultimate Course

(In Hindi)

Section

Redux and Modern Redux Toolkit
(With Thunks)

Lecture

Introduction To Redux



What is Redux?

Redux

- ❓ 3rd-Level library to manage **global state**
- ❓ **Standalone** library, but easy to integrate with React apps using react-redux library
- ❓ All global state is stored in one **globally** accessible store, which is easy to update using “**actions**” (like useReducer)
- ❓ It's conceptually similar to using the Context API + useReducer
- ❓ Two “versions”: **(1)** Classic Redux, **(2)** Modern Redux Toolkit

We will learn both



Global store is
update



All consuming
components re-
render



You need to have a really good understanding of the useReducer hook in order to understand Redux!

Do You Need To **Learn** Redux?



Historically, Redux was used in most React apps for all global state. Today, that has changed, because there are many alternatives. **Many apps don't need Redux anymore**, unless they need a lot of global UI state.



You might not need to learn Redux...



WHY LEARN REDUX IN THIS COURSE?

1

Redux can be hard to learn, and this course teaches it well 🤔

2

You will encounter Redux code in your job, so you should understand it

3

Some apps do require Redux (or a similar library)

REDUX USE CASES

Historically, Redux was used in most React apps for all global state. Today, that has changed, because there are many alternatives. **Many apps don't need Redux anymore**, unless they need **a lot of global UI state**



Local State



UI State



useState



useReducer



useRef



fetch + useEffect +
useState / useReducer



Remote State



Global State



Context API + useState /
useReducer



Redux, Zustand, Recoil, etc.



React Router



Context API + useState/useReducer



Redux, Zustand, Recoil, etc.



React Query



SWR



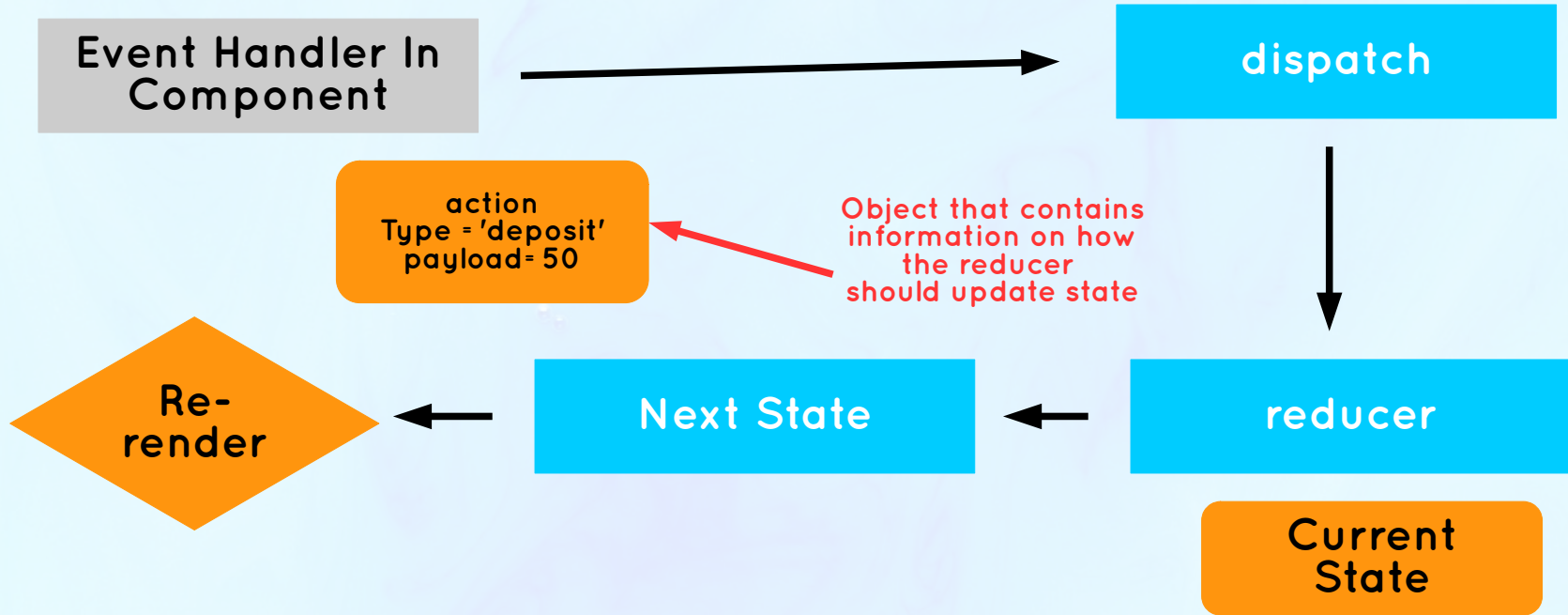
RTK Query

Ideal use case for
Redux, when there is
lots of state that
updates frequently

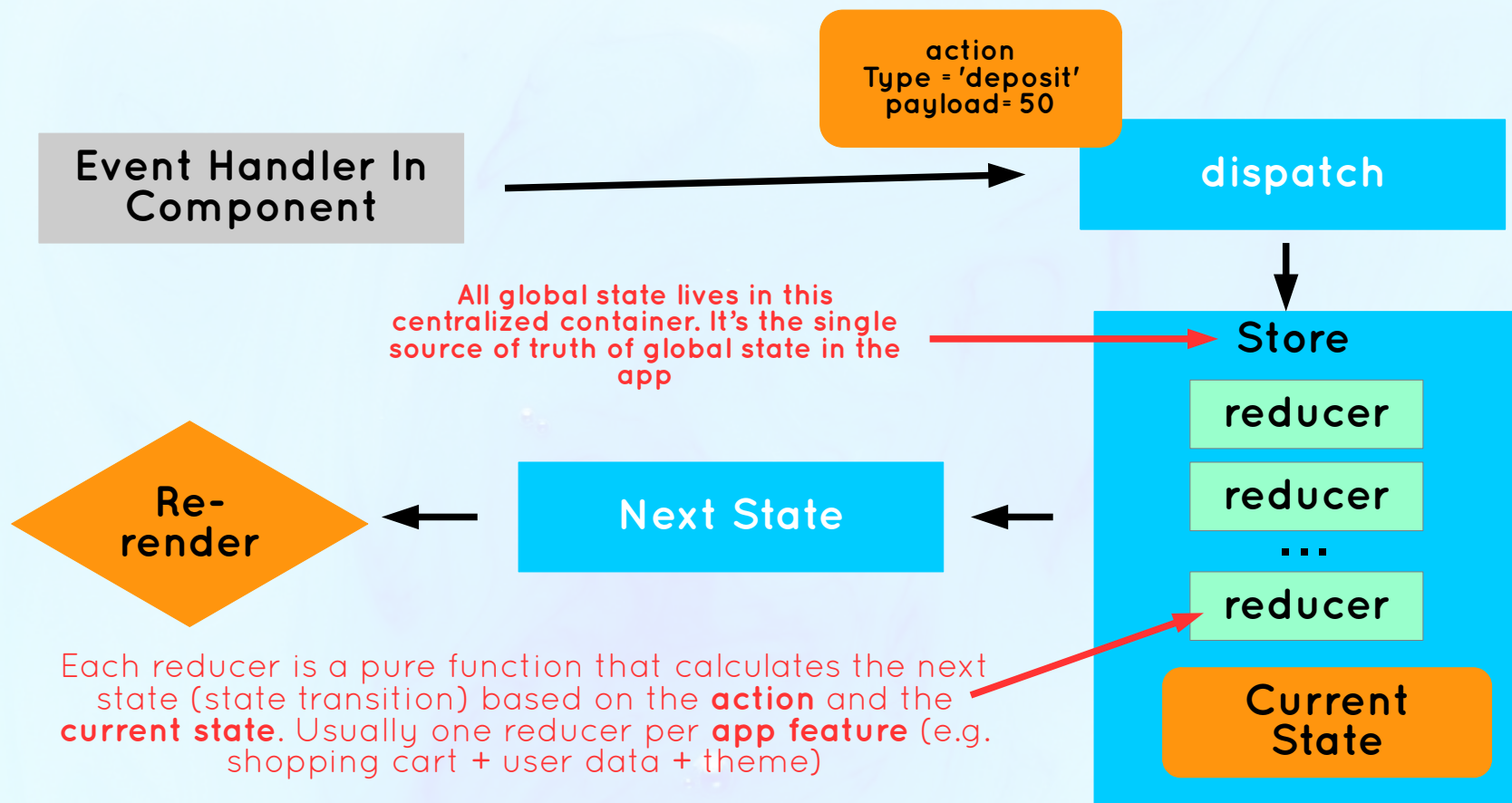
For remote
global
state, we have
better,
more specialized
tools

**THE REASON
WHY MANY
APPS DON'T
USE REDUX**

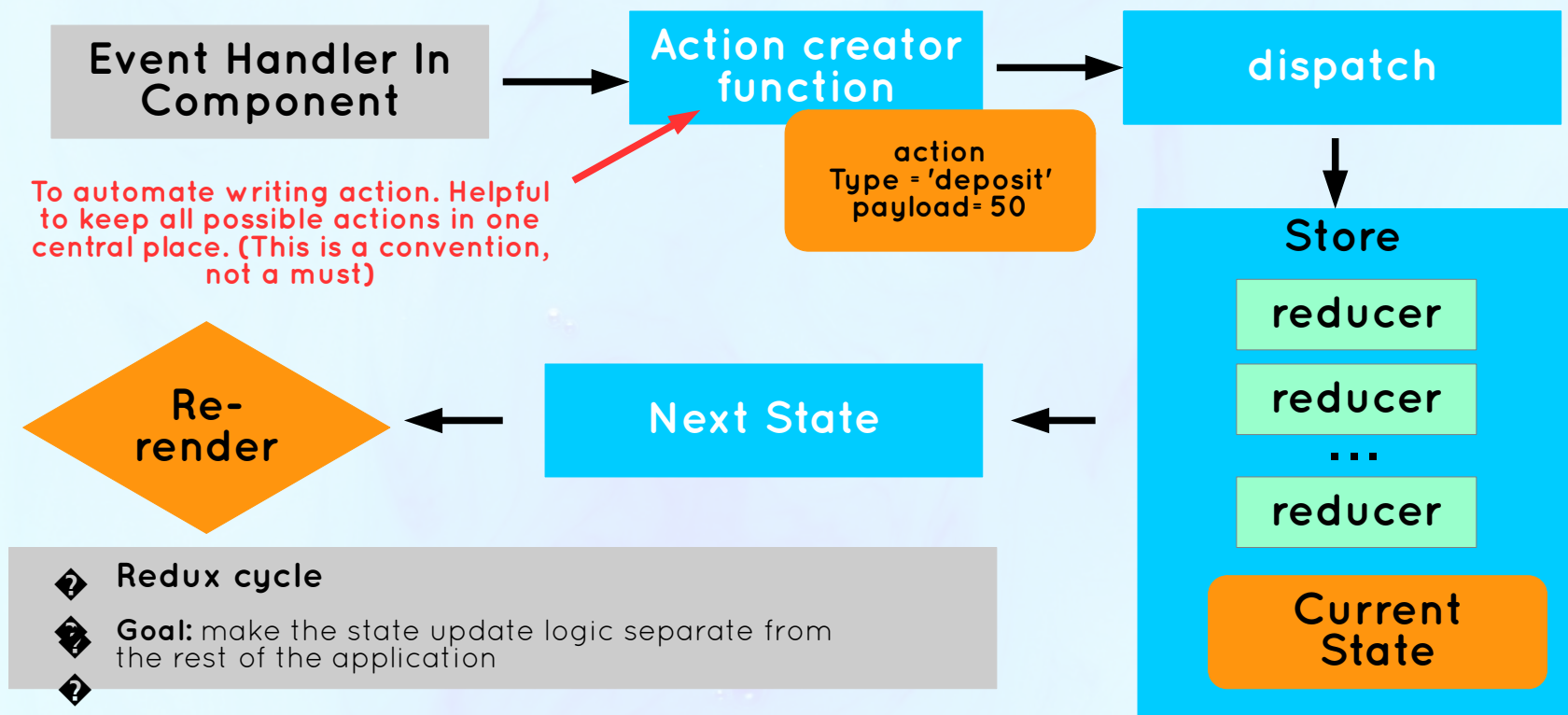
THE MECHANISM OF THE USEREDUCER HOOK



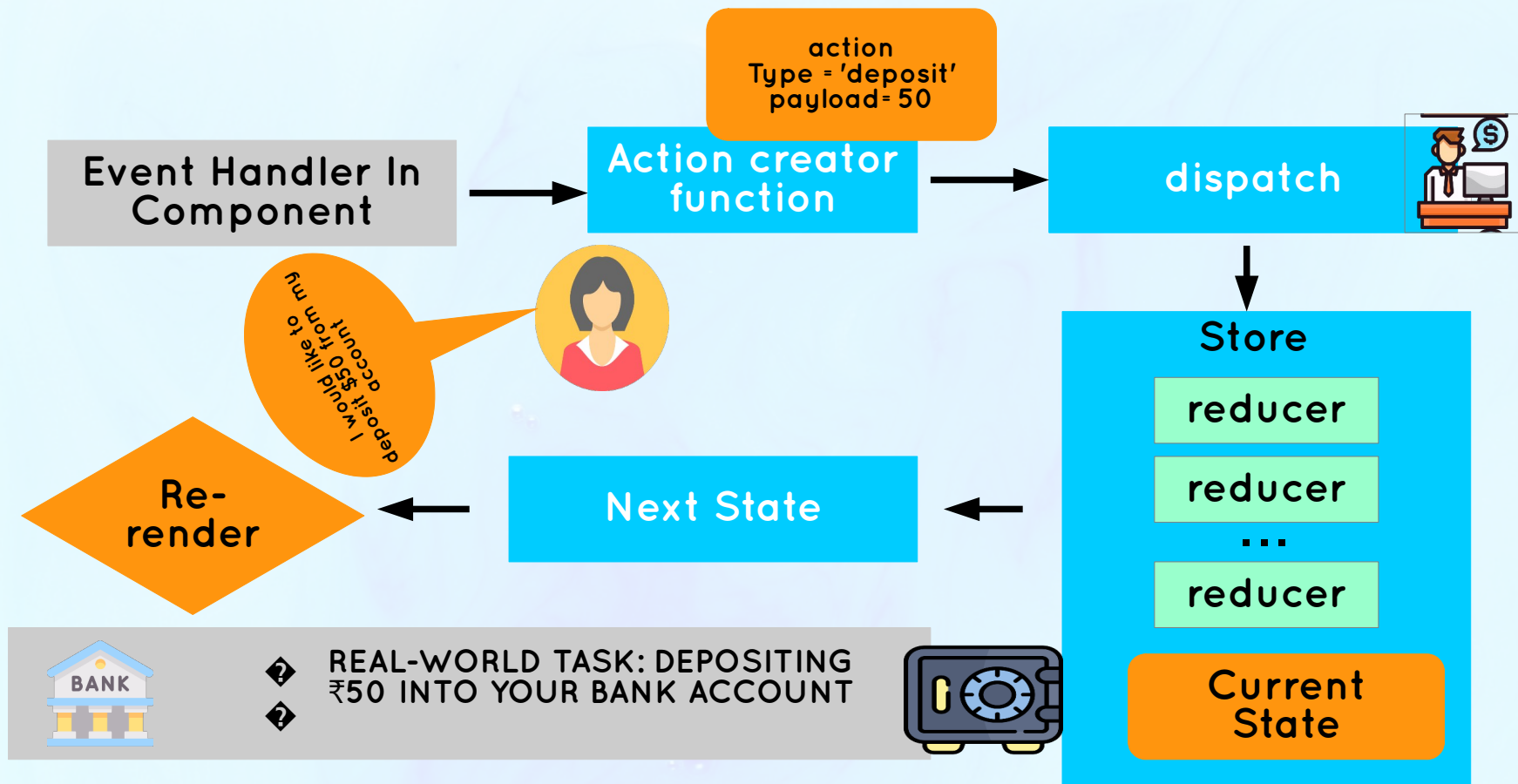
THE MECHANISM OF REDUX



THE MECHANISM OF REDUX



THE MECHANISM OF REDUX



A React & Next Ultimate Course

(In Hindi)

Section

Redux and Modern Redux Toolkit
(With Thunks)

Lecture

Redux Middle ware and Thunks



What is Redux Middleware?

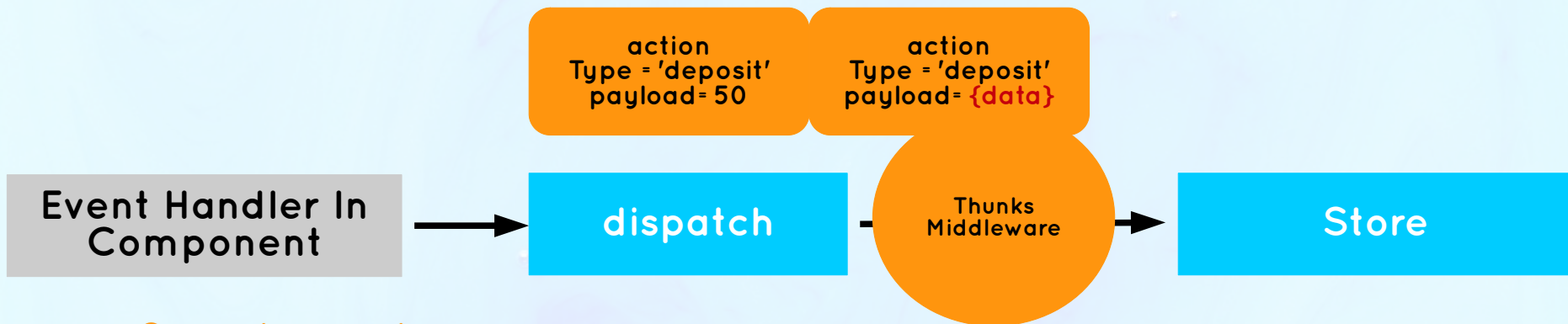
Where to make an **asynchronous API call** (or any other async operation) in Redux?

A function that sits between dispatching the action and the store. Allows us to run code after dispatching, but before reaching the reducer in the store.



Redux Thunks?

Where to make an asynchronous API call (or any other async operation) in Redux?



❖ Can make asynchronous operations and then dispatch



❖ Fetching data in components is not ideal



❖ Perfect for asynchronous code



❖ API calls, timers, logging, etc.



❖ The place for side effects



❖ No asynchronous operations



❖ Reducers need to be pure functions



A React & Next Ultimate Course

(In Hindi)

Section

Redux and Modern Redux Toolkit
(With Thunks)

Lecture

What is Redux Toolkit (RTK)

What is Redux Toolkit?

Redux Toolkit

- 👉 The **modern and preferred** way of writing Redux code
- 👉 An **opinionated** approach, forcing us to use Redux best practices
- 👉 100% compatible with “classic” Redux, allowing us to **use them together**
- 👉 Allows us to write **a lot less code** to achieve the same result (less “boilerplate”)
- 👉 Gives us 3 big things (but there are many more...):
 - 1 We can write code that “**mutates**” state inside reducers (will be converted to **immutable** logic behind the scenes by “Immer” library)
 - 2 Action creators are **automatically** created
 - 3 **Automatic** setup of thunk middleware and DevTools

A React & Next Ultimate Course

(In Hindi)

Section

Redux and Modern Redux Toolkit
(With Thunks)

Lecture

Redux VS Context API



Context API VS Redux

Context API + useReducer

- 👍 Built into React
- 👍 Easy to set up a **single context**
- 👎 Additional state “slide” requires new context **set up from scratch** (“provider hell” in App.js)
- 👎 **No** mechanism for async operations
- 👎 Performance optimization is a **pain**
- 👎 Only React DevTools

Redux

- 👎 Requires additional package (larger bundle size)
- 👎 More work to set up **initially**
- 👍 Once set up, it’s easy to create **additional state “slices”**
- 👍 Supports **middleware** for async operations
- 👍 Performance is optimized **out of the box**
- 👍 Excellent DevTools

Keep in mind that we should not use these solutions for remote state

When to Use Context API or Redux?

Context API + useReducer

“Use the Context API for global state management in **small** apps”

👉 When you just need to share a value that **doesn't change often** [Color theme, referred language, authenticated user, ...]

👉 When you need to solve a simple **prop drilling** problem

👉 When you need to manage state in a **local sub-tree** of the app

These are not
super common
in UI state

Redux

“Use Redux for global state management in **large** apps”

👉 When you have lots of global UI state that needs to be **updated frequently** (because Redux is optimized for this) [Shopping cart, current tabs, complex filters or search, ...]

👉 When you have **complex state** with nested objects and arrays (because you can mutate state with Redux Toolkit)

For example in the compound component pattern

There is no right answer that fits every project. It all depends on the project needs!

Level 4

Professional
React
Development

React Router With Data Loading (V 6.4+)

A React & Next Ultimate Course

(In Hindi)

Section

React Router With Data Loading (V6+)

Lecture

Application Planning



The Project: Fast React Pizza Co.

REMEMBER OUR VERY FIRST PROJECT?

— INDIAN REACT PIZZA CO. —

OUR MENU



Paneer Tikka

Tomato sauce, paneer, tikka masala sauce, bell peppers, onions, and mozzarella cheese.

99



Tandoori Chicken Pizza

Tomato sauce, tandoori chicken, red onions, bell peppers, and mozzarella cheese.

149



Masala Pizza

Tomato sauce, Indian spices, garam masala, cumin, and coriander

79



Uttapam Pizza

Dosa batter, traditional uttapam toppings and cheese

129



Keema Pizza

Tomato sauce, spiced minced meat, onions, and mozzarella cheese.

159



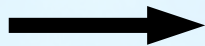
Onion Pizza

Tomato sauce, Thinly sliced & caramelized onions, Mozzarella

89

We are open until 22:00. Come visit us or order online.

Order



Fast React Pizza Co.



Now the same restaurant (business) needs a simple way of allowing customers to **order pizzas and get them delivered** to their home



We were hired to build the application front-end



How To Plan & Build A React App

FROM THE EARLIER “THINKING IN REACT” LECTURE:

- 1 Break the desired UI into **components**
- 2 Build a **static** version (no state yet)
- 3 Think about **state management + data flow**



This works well for small apps with **one page and a few features**



In **real-world apps**, we need to adapt this process



How To Plan & Build A React App

1

Gather application requirements and features

This is just a rough overview. In the real-world, things are never this linear

2

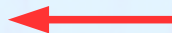
Divide the application into **pages**



Think about the **overall** and **page-level** UI



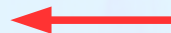
Break the desired UI into **components**



From Earlier



Design and build a **static** version (no state yet)



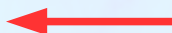
From Earlier

3

Divide the application into feature categories



Think about **state management + data flow**



From Earlier

4

Decide on what libraries to use (technology decisions)



Project Requirements From The Business

Step 1

- 👉 Very simple application, where users can order **one or more pizzas from a menu**
- 👉 Requires **no user accounts** and no login: users just input their names before using the app
- 👉 The pizza menu can change, so it should be loaded from an API
- 👉 Users can add multiple pizzas to a **cart** before ordering
- 👉 Ordering requires just the **user's name, phone number, and address**
- 👉 If possible, **GPS location** should also be provided, to make delivery easier
- 👉 User's can **mark their order as "priority"** for an additional 20% of the cart price
- 👉 Orders are made by sending a POST request with the order data (user data + selected pizzas) to the API
- 👉 Payments are made on delivery, so **no payment processing** is necessary in the app
- 👉 Each order will get a **unique ID** that should be displayed, so the **user can later look up their order** based on the ID
- 👉 Users should be able to mark their order as "priority" order **even after it has been placed**



Done

From these requirements, we can understand the features we need to implement



Project **Requirements** From The Business

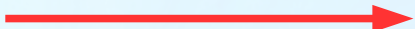
Step 2 + 3

Feature Categories

Necessary Pages

1

User



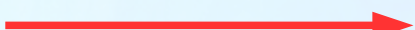
1

Homepage

/

2

Menu



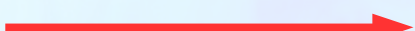
2

Pizza Menu

/menu

3

Cart



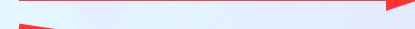
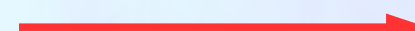
3

Cart

/cart

4

Order



4

Placing a new order

/order/new

5

Looking up an order

/order/:orderId



All features can be placed into one of these. So this is what the app will essentially be about

State Management + Technology Decision

Step 3 + 4

STATE
"DOMAINS" /
"SLICES"

1

User → Global UI state (no accounts, so stays in app)

2

Menu → Global remote state (menu is fetched from API)

3

Cart → Global UI state (no need for API, just stored in app)

4

Order → Global remote state (fetched and submitted to API)

Types of
State

These usually
map quite nice to
the app feature

This is just one of
many tech stacks we
could have chosen



Routing



Styling



Remote State
Management



UI State
Management



The standard for React SPAs



Trendy way of styling applications that we want to learn



New way of fetching data right inside React Router (v6.4+) that is worth exploring ("**render-as-you-fetch**" instead of "**fetch-on-render**"). Not really state management, as it doesn't persist state.



State is fairly complex. Redux has many advantages for UI state. Also, we want to practice Redux a bit more

Tailwind CSS Crash Course: Styling App

A React & Next Ultimate Course

(In Hindi)

Section

Tailwind CSS Crash Course: Styling The App

Lecture

WHAT IS TAILWIND CSS?

What Is **Tailwind** CSS?

Tailwind CSS



“A utility-first CSS framework packed with utility classes like flex, text-center and rotate-90 that can be composed to build any design, directly in your markup (HTML or JSX)”



Utility-first CSS approach: writing tiny classes with one single purpose, and then combining them to build entire layouts



In tailwind, **these classes are already written for us**. So we're not gonna write any new CSS, but instead use some of tailwind's hundreds of classes



Good & Bad About Tailwind

Good



You don't need to think about class names



No jumping between files to write markup and styles



Immediately understand styling in any project that uses tailwind



Options have been taken for you, which makes UIs look better and more consistent



Saves a lot of time, e.g. on responsive design



Docs and VS Code integration are great

Bad



Markup (HTML or JSX) looks very unreadable with lots of class names (you get used to it)



You have to learn a lot of class names (but after a day of usage you know fundamentals)



You need to install and set up tailwind on each new project



You're giving up on "vanilla CSS" 😂



Many people love to hate on tailwind for no reason. Please don't be that person! Try it before judging 🙌

A React & Next Ultimate Course

(In Hindi)

This Section

Adding Redux &
Advance React
Router



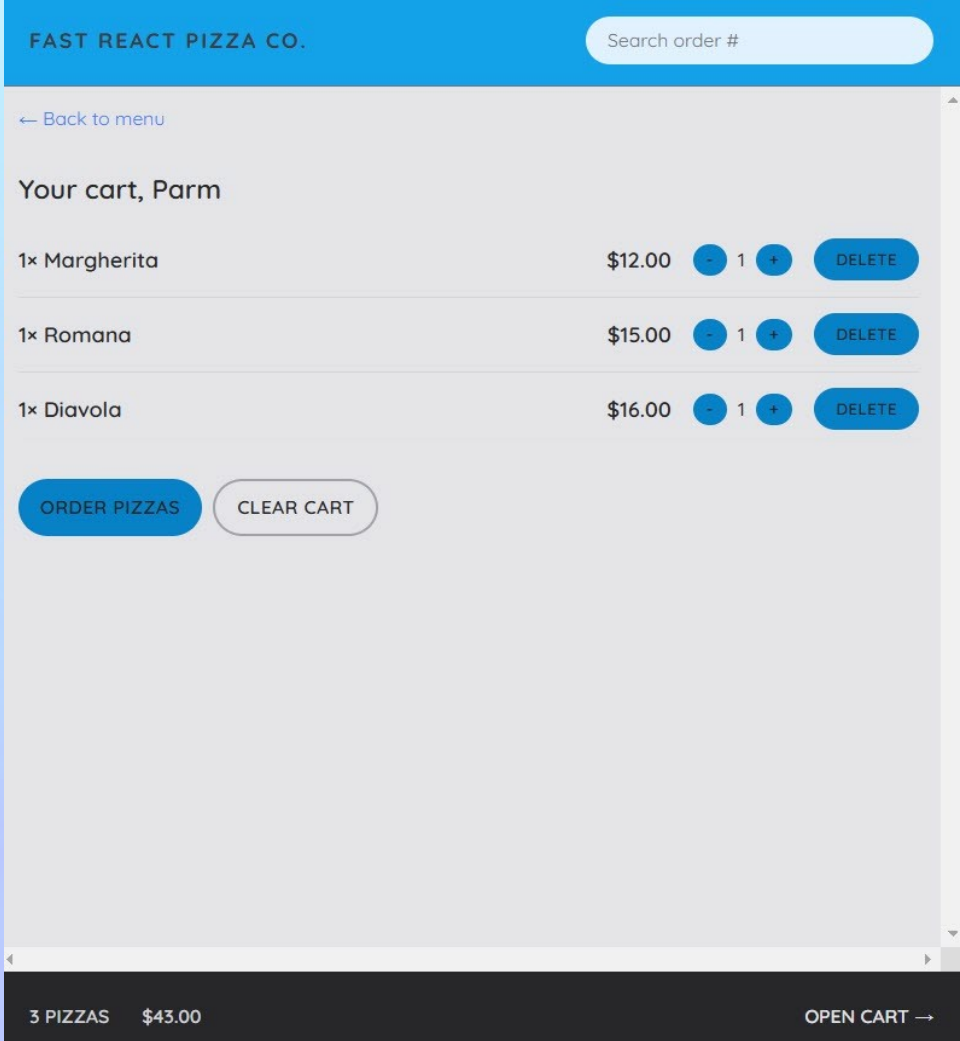
Implementing a **shopping cart**



Advance Levels of React Router



Real-World use cases of Redux



A React & Next Ultimate Course

(In Hindi)

Section

Adding Redux & Advance React Router

Lecture

Modeling The “User” State With Redux
Toolkit





Project Requirements From The Business



Very simple application, where users can order **one or more pizzas from a menu**



Requires **no user accounts** and no login: users just input their names before using the app



The pizza menu can change, so it should be loaded from an API



Users can add multiple pizzas to a **cart** before ordering



Ordering requires just the **user's name, phone number, and address**



If possible, **GPS location** should also be provided, to make delivery easier



User's can **mark their order as "priority"** for an additional 20% of the cart price



Orders are made by sending a POST request with the order data (user data + selected pizzas) to the API



Payments are made on delivery, so **no payment processing** is necessary in the app



Each order will get a **unique ID** that should be displayed, so the **user can later look up their order** based on the ID



Users should be able to mark their order as "priority" order **even after it has been placed**

Setting UP Big Project + Styled Components

A React & Next Ultimate Course

(In Hindi)

Section

Setting Up Our Biggest Project + Styled
Components

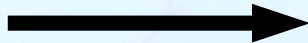
Lecture

Application Planning

The Project: The Maclo Cottage



The Maclo Cottage



“The Maclo Cottages” is a small boutique **hotel** with 7 luxurious wooden cottages



They need a custom-built application to manage everything about the hotel: **bookings, cottage and guests**



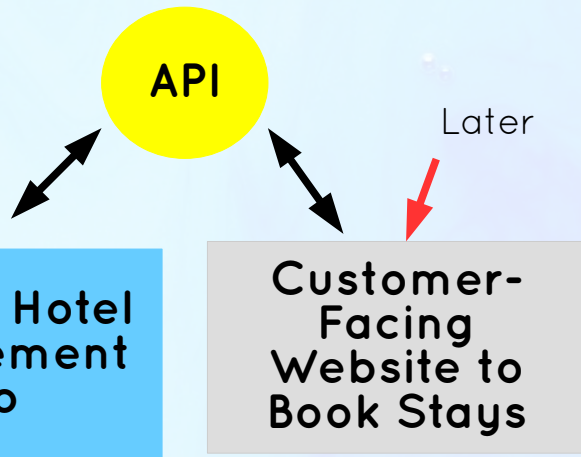
This is the **internal application** used inside the hotel to **check in guests as they arrive**



They have nothing right now, so they **also need the API**



Later they will probably want a **customer-facing website** as well, where customers will be able to **book stays**, using the same API



Review: How To Plan A React Application

1

Gather application requirements and features

2

Divide the application into **pages**

3

Divide the application into **feature categories**

4

Decide on what **libraries** to use (technology decisions)



Project Requirements From The Business

Step 1

- Users of the app are hotel employees. They need to be logged into the application to perform tasks
- New users can only be signed up inside the applications (to guarantee that only actual hotel employees can get accounts)
- Users should be able to upload an avatar, and change their name and password
- App needs a table view with all cottages, showing the cottages photo, name, capacity, price, and current discount
- Users should be able to update or delete a cottage, and to create new cottage (including uploading a photo)
- App needs a table view with all bookings, showing arrival and departure dates, status, and paid amount, as well as cabin and guest data
- The booking status can be “unconfirmed” (booked but not yet checked in), “checked in”, or “checked out”. The table should be filterable by this important status
- Other booking data includes: number of guests, number of nights, guest observations, whether they booked breakfast, breakfast price
- Users should be able to delete, check in, or check out a booking as the guest arrives (no editing necessary for now)
- Bookings may not have been paid yet on guest arrival. Therefore, on check in, users need to accept payment (outside the app), and then confirm that payment has been received (inside the app)
- On check in, the guest should have the ability to add breakfast for the entire stay, if they hadn't already
- Guest data should contain: full name, email, national ID, nationality, and a country flag for easy identification
- The initial app screen should be a dashboard, to display important information for the last 7, 30, or 90 days:
 - A list of guests checking in and out on the current day. Users should be able to perform these tasks from here
 - Statistics on recent bookings, sales, check ins, and occupancy rate
 - A chart showing all daily hotel sales, showing both “total” sales and “extras” sales (only breakfast at the moment)
 - A chart showing statistics on stay duration's, as this is an important metric for the hotel
- Users should be able to define a few application-wide settings: breakfast price, min and max nights/booking, max guests/booking
- App needs a dark mode



Project Requirements From The Business

Step 2



Users of the app are hotel employees. They need to be logged into the application to perform tasks



New users can only be signed up inside the applications (to guarantee that only actual hotel employees can use the app)



Users should be able to upload an avatar, and change their name and password

Authentication



App needs a table view with all cottages, showing the cottages photo, name, capacity, price, and current status



Users should be able to update or delete a cottage, and to create new cottage (including uploading a photo)

Cottages



App needs a table view with all bookings, showing arrival and departure dates, status, and paid amount, as well as cabin and guest data



The booking status can be “unconfirmed” (booked but not yet checked in), “checked in”, or “checked out”. The status is filterable by this important status



Other booking data includes: number of guests, number of nights, guest observations, whether they booked breakfast, breakfast price

Bookings



Users should be able to delete, check in, or check out a booking as the guest arrives (no editing necessary after check in)



Bookings may not have been paid yet on guest arrival. Therefore, on check in, users need to accept payment and then confirm that payment has been received (inside the app)



On check in, the guest should have the ability to add breakfast for the entire stay, if they hadn't already

Check in/out



Guest data should contain: full name, email, national ID, nationality, and a country flag for easy identification



The initial app screen should be a dashboard, to display important information for the last 7, 30, or 90 days:

Guests



A list of guests checking in and out on the current day. Users should be able to perform these tasks



Statistics on recent bookings, sales, check ins, and occupancy rate



A chart showing all daily hotel sales, showing both “total” sales and “extras” sales (only breakfast at the moment)



A chart showing statistics on stay duration's, as this is an important metric for the hotel

Dashboard



Users should be able to define a few application-wide settings: breakfast price, min and max nights/bookings



App needs a dark mode

Settings

Features And Pages

Step 2+3

Feature Categories

1	Bookings	1
2	Cottages	2
3	Guests	3
4	Dashboard	4
5	Check in and out	5
6	App Setting	6
7	Authentication	7
		8

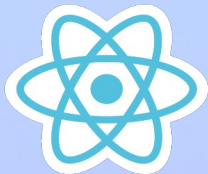
Necessary Pages

Dashboard	/dashboard
Bookings	/booking
Cottages	/cottages
Booking Check In	/checkin:/bookingId
App Setting	/settings
User Sign Up	/users
Login	/login
Account Setting	/account

Client Side Rendering(CSR) **VS** Server Side Rendering(SSR)

CSR With Plain React

- 👉 Used to build **Single**-Page Applications (SPAs)
- 👉 All HTML is rendered on the **client**
- 👉 All JavaScript needs to be downloaded before apps start running: **bad for performance**
- 👉 **One perfect use case:** apps that are used “internally” as tools inside companies, that are entirely hidden behind a login



↗ This is exactly what we want to build in this project

SSR With Framework

- 👉 Used to build **Multi**-Page Applications (MPAs)
- 👉 Some HTML is rendered in the **server**
- 👉 **More performative**, as less JavaScript needs to be downloaded
- 👉 The **React team** is moving more and more in this direction

NEXT.JS

Technology Decision

Step 4

👉 Routing



The standard for React SPAs

👉 Styling



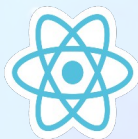
Very popular way of writing component-scoped CSS, right inside JavaScript. A technology worth learning

👉 Remote State Management



The best way of managing remote state, with features like caching, automatic re-fetching, pre-fetching, offline support, etc. Alternatives are SWR and RTK Query, but this is the most popular

👉 UI State Management



Context API

There is almost no UI state needed in this app, so one simple context with useState will be enough. No need for Redux

👉 Form Management



Handling bigger forms can be a lot of work, such as manual state creation and error handling. A library can simplify all this

👉 Other Tools

React icons / React hot toast / Recharts / date-fns / Supabase

Supabase Crash Course: Building Back-end

A React & Next Ultimate Course

(In Hindi)

Section

Supabase Crash Course: Building A Back-End!

Lecture

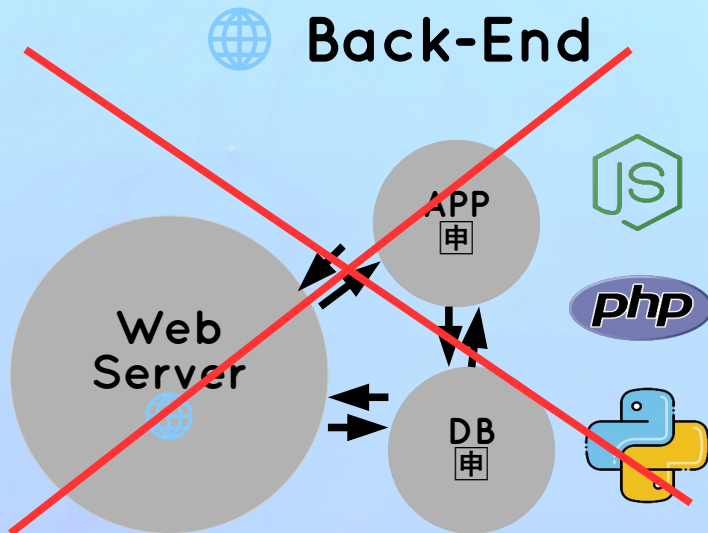
What is Supabase?



What Is Supabase?

Supabase

- 👉 Service that allows developers to easily **create a back-end with a PostgreSQL database**
- 👉 Automatically creates a **database** and **API** so we can easily request and receive data from the server
- 👉 **No** back-end development needed
- 👉 Perfect to get up and running **quickly!**
- 👉 Not just an API: Supabase also comes with easy-to-use **user authentication and file storage**



WITH SUPABASE, WE DON'T NEED
TO DO ANY OF THIS MANUALLY!
IT'S ALL INCLUDED



supabase

A React & Next Ultimate Course

(In Hindi)

Section

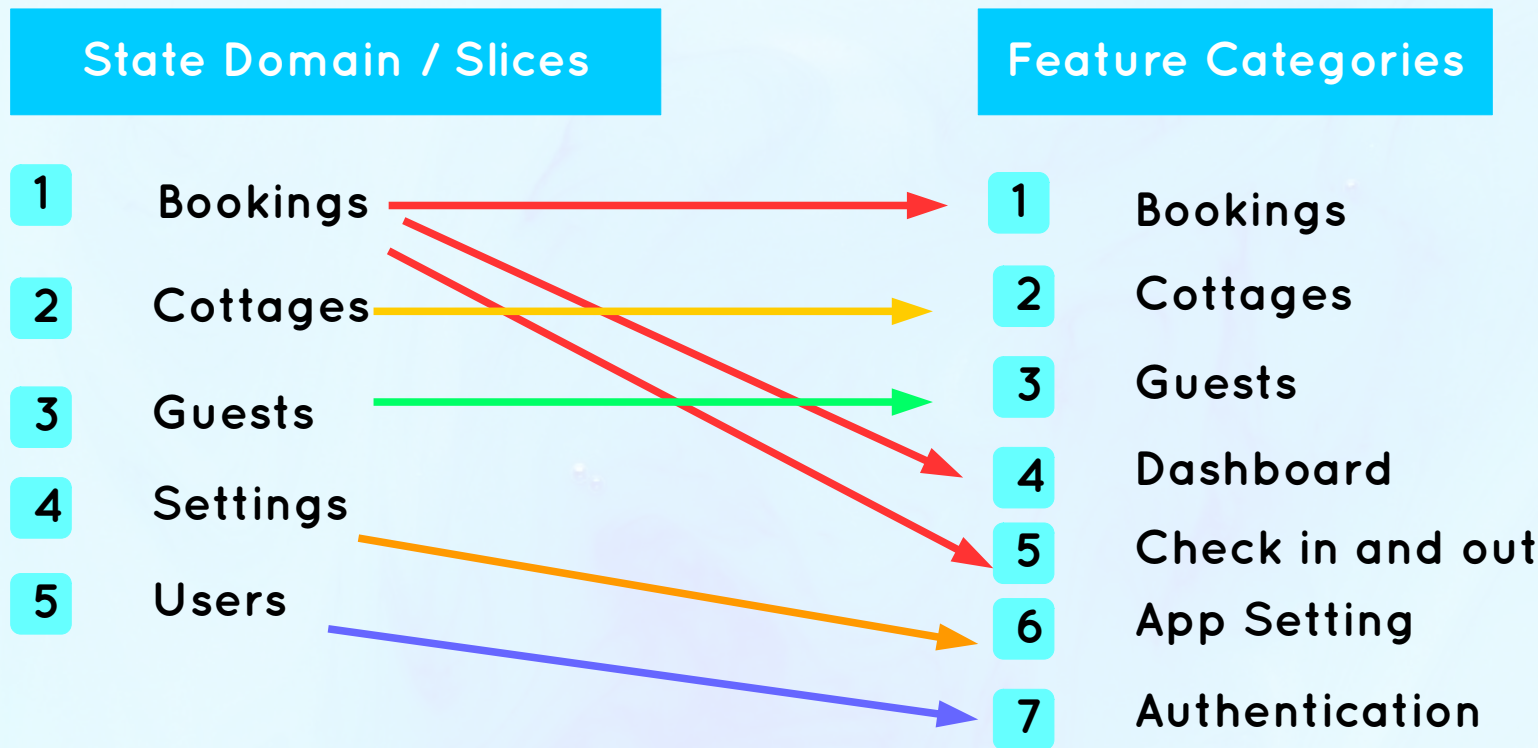
Supabase Crash Course: Building A Back-End!

Lecture

Modeling Application State



Modeling State

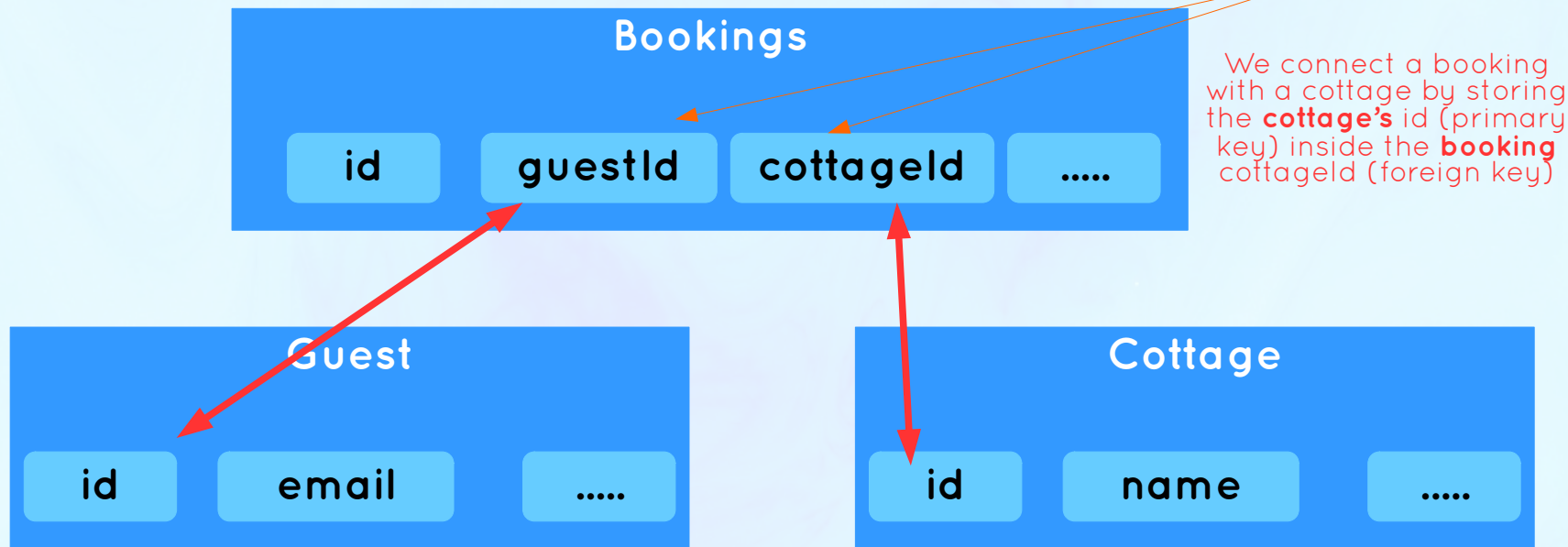


👉 All this state will be **global remote state**, stored within Supabase

👉 There will be one **table** for each state “slice” in the **database**

The Bookings Table

- 👉 **Bookings** are about a **guest** renting a **cottage**
- 👉 So a booking needs information about what **guest** is booking which **cottage**: we need to connect them
- 👉 Supabase uses a Postgres DB, which is SQL (relational DB). So we **join** tables using **foreign keys**



React Query: Managing Remote State

A React & Next Ultimate Course

(In Hindi)

Section

React Query: Managing Remote State

Lecture

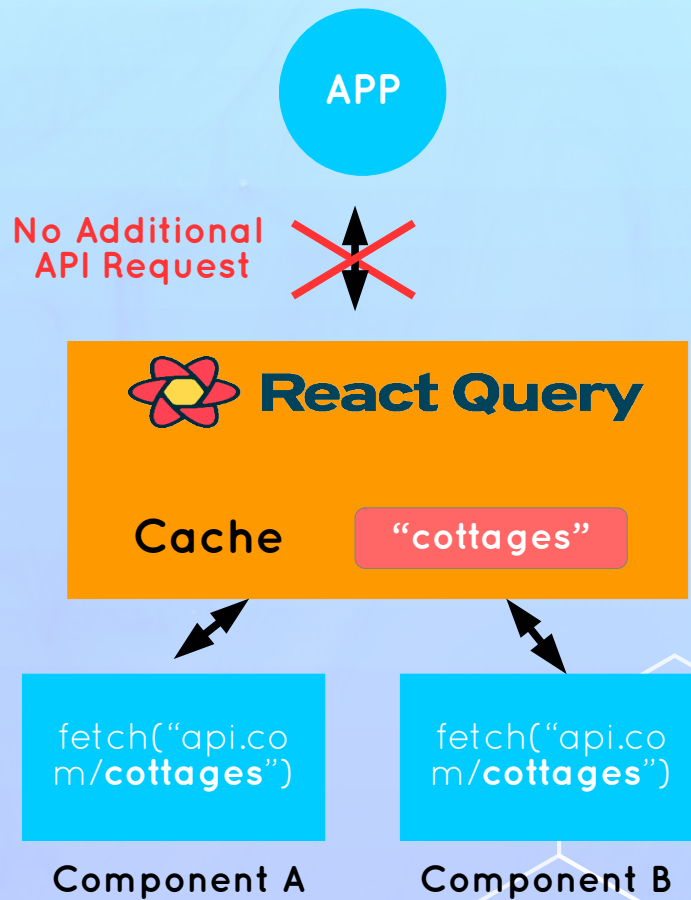
What is React Query?



What Is React Query?

React Query

- 👉 Powerful library for managing **remote(Server) state**
- 👉 Many features that allow us to write a lot less code, while also **making the UX a lot better**:
 - 👉 Data is stored in cache
 - 👉 Automatic loading and error state
 - 👉 Automatic re-fetching to keep state synced
 - 👉 Pr-fetching
 - 👉 Easy remote state mutation (updating)
 - 👉 Offline support
- 👉 Needed because remote state is **fundamentally** different from regular (UI) state



Advance React Patterns

A React & Next Ultimate Course

(In Hindi)

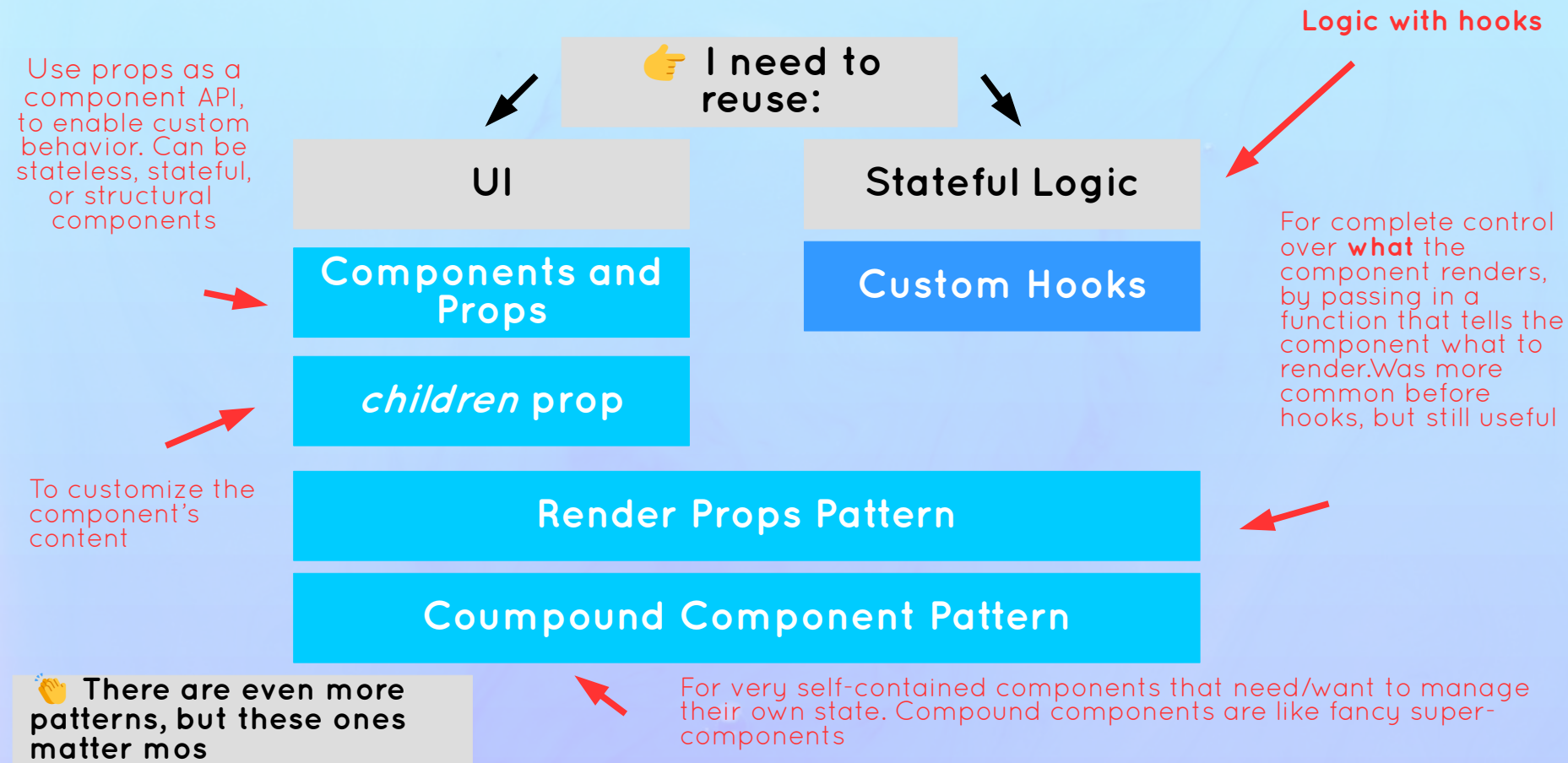
Section

Advance React Patterns

Lecture

An Overview Of Re usability In React

How to Reuse Code In React?



Level 5



Full Stack React Development

Overview of Next.JS With The ‘App’ Router

A React & Next Ultimate Course

(In Hindi)

Section

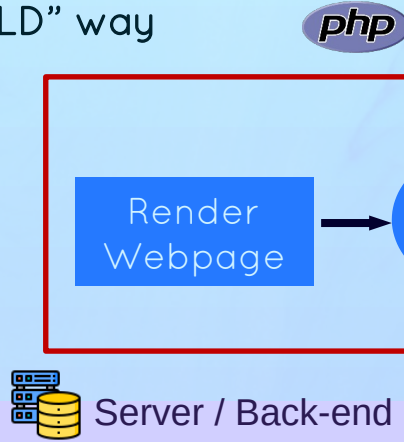
Overview of Next.js With The “App” Router

Lecture

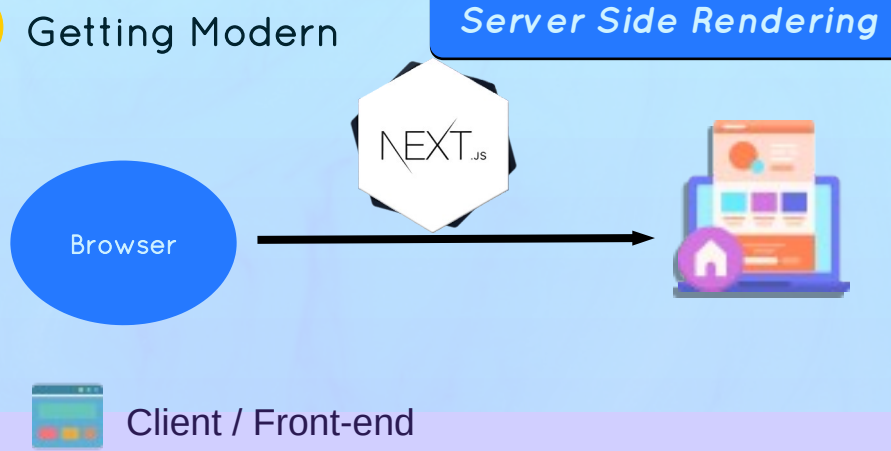
**An Overview Of Server-Side Rendering
(SSR)**

Review: Rise Of Single-Page Application

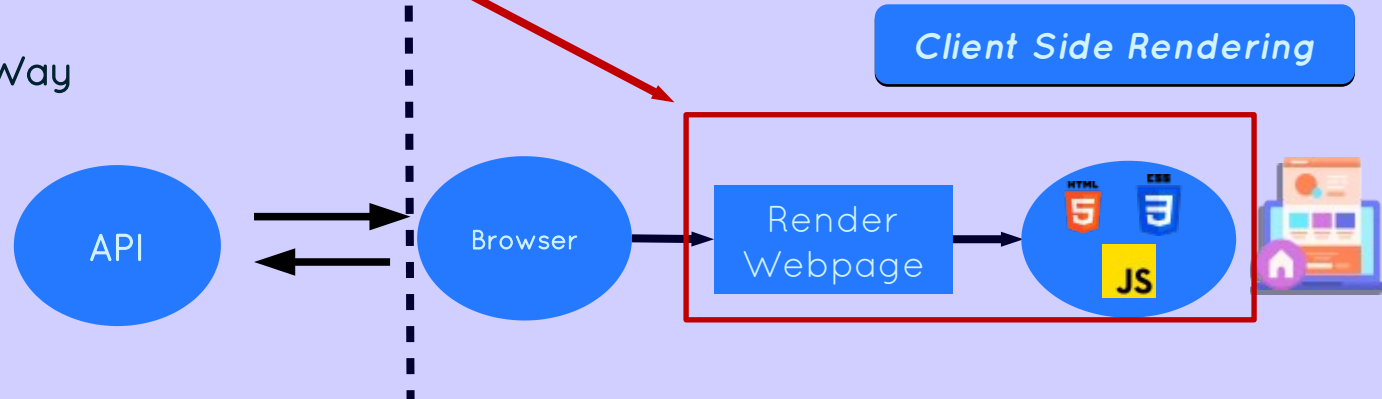
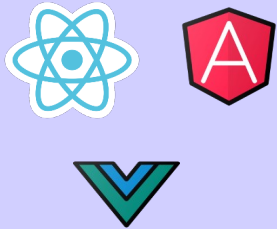
1 The "OLD" way



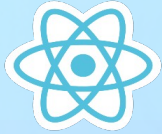
3 Getting Modern



2 The "Modern" Way



Client-Side Rendering(CSR) VS Server-Side Rendering(SSR)



Client-Side Rendering

❖ HTML is rendered on the client (the user's computer) using JavaScript

❖ **Slower initial page loads:**

❖ Bigger JavaScript bundle needs to be downloaded before app starts running

❖ Data is fetched after components mount

❖ **Highly interactive:** All the code and content has already been loaded (except data)

❖ **SEO can be problematic**



Server-Side Rendering

❖ HTML is rendered on the server (the developer's computer)

❖ **Faster initial page loads:**

❖ Less JavaScript needs to be downloaded and executed

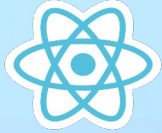
❖ Data is fetched before HTML is rendered

❖ **Less interactive:** Pages might be downloaded on demand and require full page reloads

❖ **SEO-friendly:** Content is easier for search engines to index



Client-Side Rendering(CSR) VS Server-Side Rendering(SSR)



Client-Side Rendering

- ◆ **SPAs:** Perfect for building highly interactive web apps
- ◆
- ◆ **Apps that don't need SEO:**
- ◆
- ◆ Apps that are used "internally" as tools inside companies
- ◆
- ◆ Apps that are entirely hidden behind a login
- ◆

This is what we've been doing so far



Server-Side Rendering

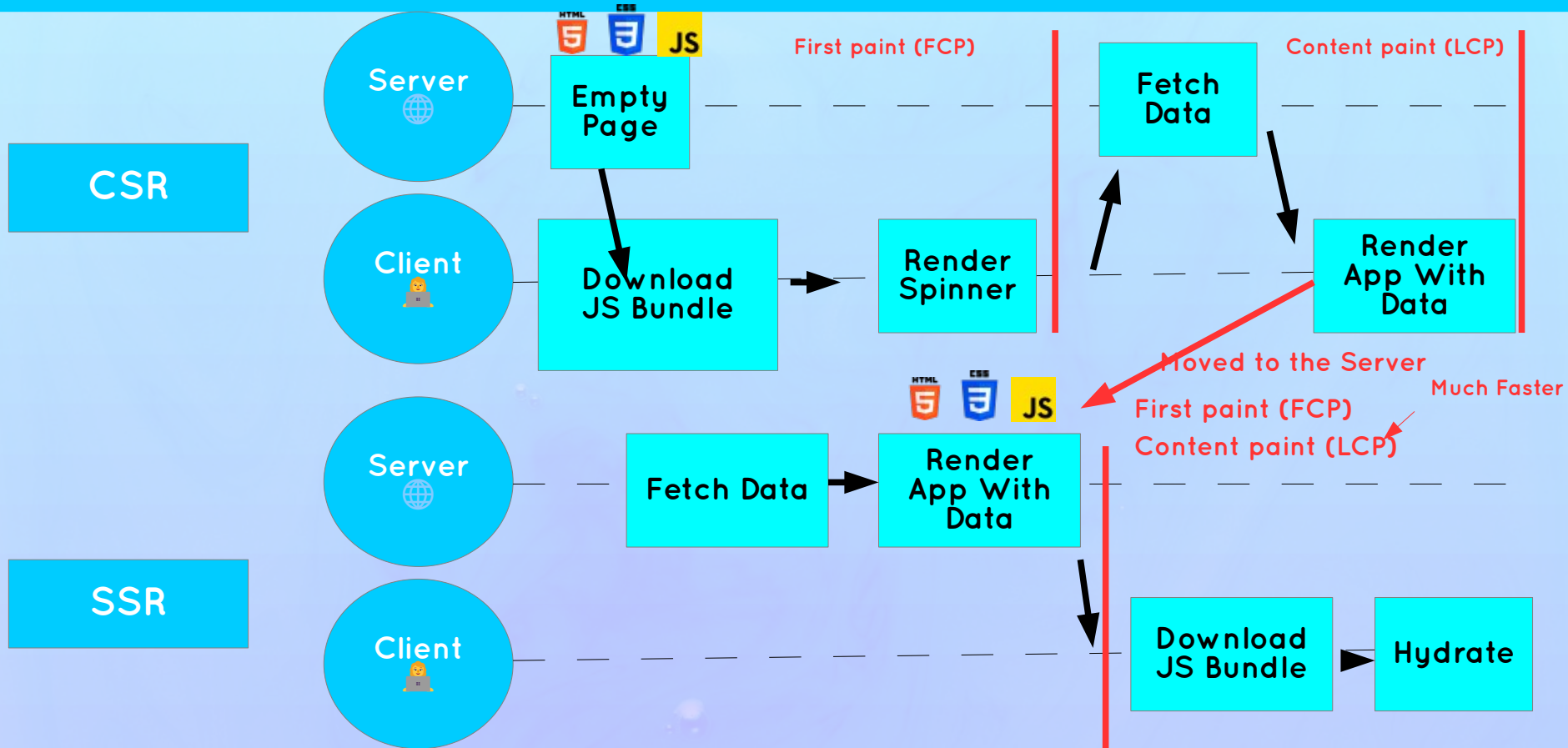
- ◆ **Content-driven websites or apps where SEO is essential:** E-commerce, blogs, news, marketing websites, etc
- ◆

More on this later

Two Type Of SSR:

- 1 Static:** HTML generated at build time (often called Static Site Generation, or SSG)
- 2 Dynamic:** HTML generated each time server receives new request (some call only this SSR)

Typical Timeline Of CSR VS. SSR With Data Fetching



A React & Next Ultimate Course

(In Hindi)

Section

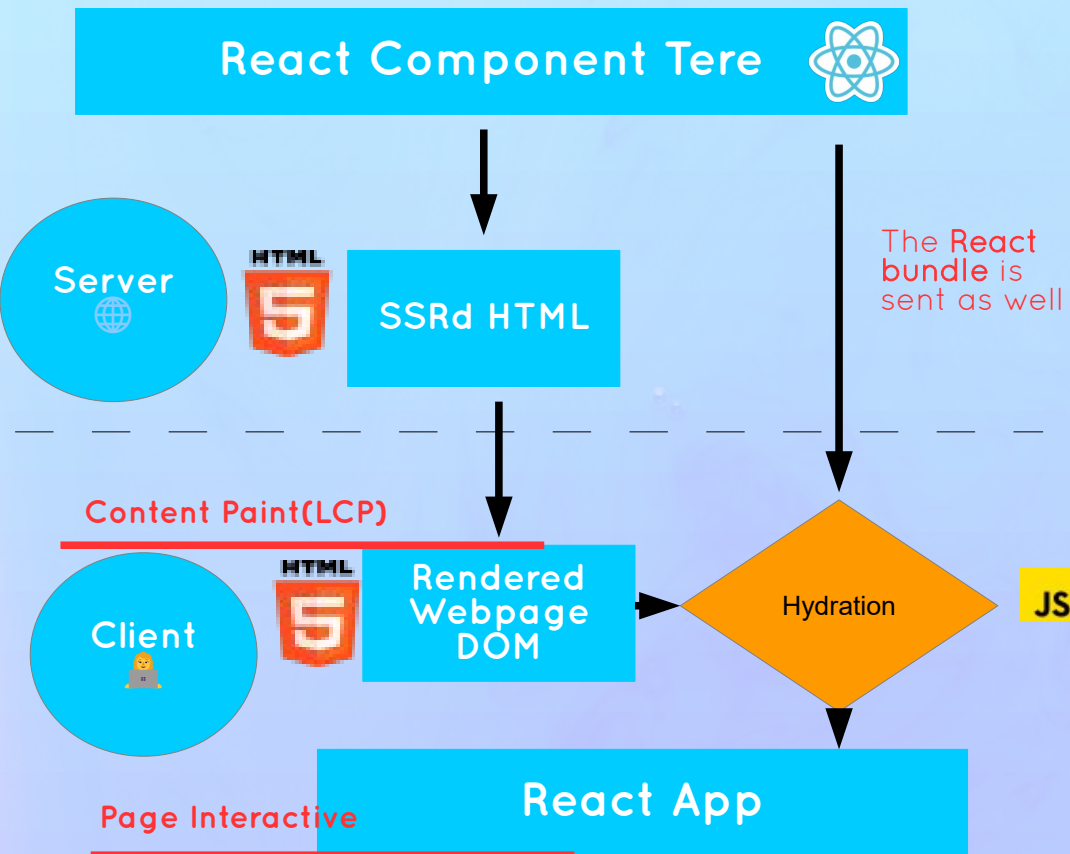
Overview of Next.Js With The “App” Router

Lecture

Hydration: The Missing Level



What Is Hydration?



Hydration

- ❖ Adds back the **interactivity and eventhandlers** that were lost when HTML was server-side rendered
- ❖ Watering the “dry” HTML with the “water” of interactivity and event handlers
- ❖ React builds the component tree on the client and compares it with the actual SSRd DOM: **They must be the same** so React can adopt it!
- ❖ **Common hydration error causes:** incorrect HTML element nesting, different data used for rendering, using browser-only APIs, side effects, etc.

A React & Next Ultimate Course

(In Hindi)

Section

Overview of Next.Js With The “App” Router

Lecture

What is Next.JS?



What Is Next.JS?

Next.JS

“THE REACT FRAMEWORK FOR THE WEB”



👉 **Meta-framework built on top of React:** we still use components, props, react hooks, etc.

❓ **Opinionated way of building React apps:** Set of conventions and best practices regarding routing, data fetching, etc.

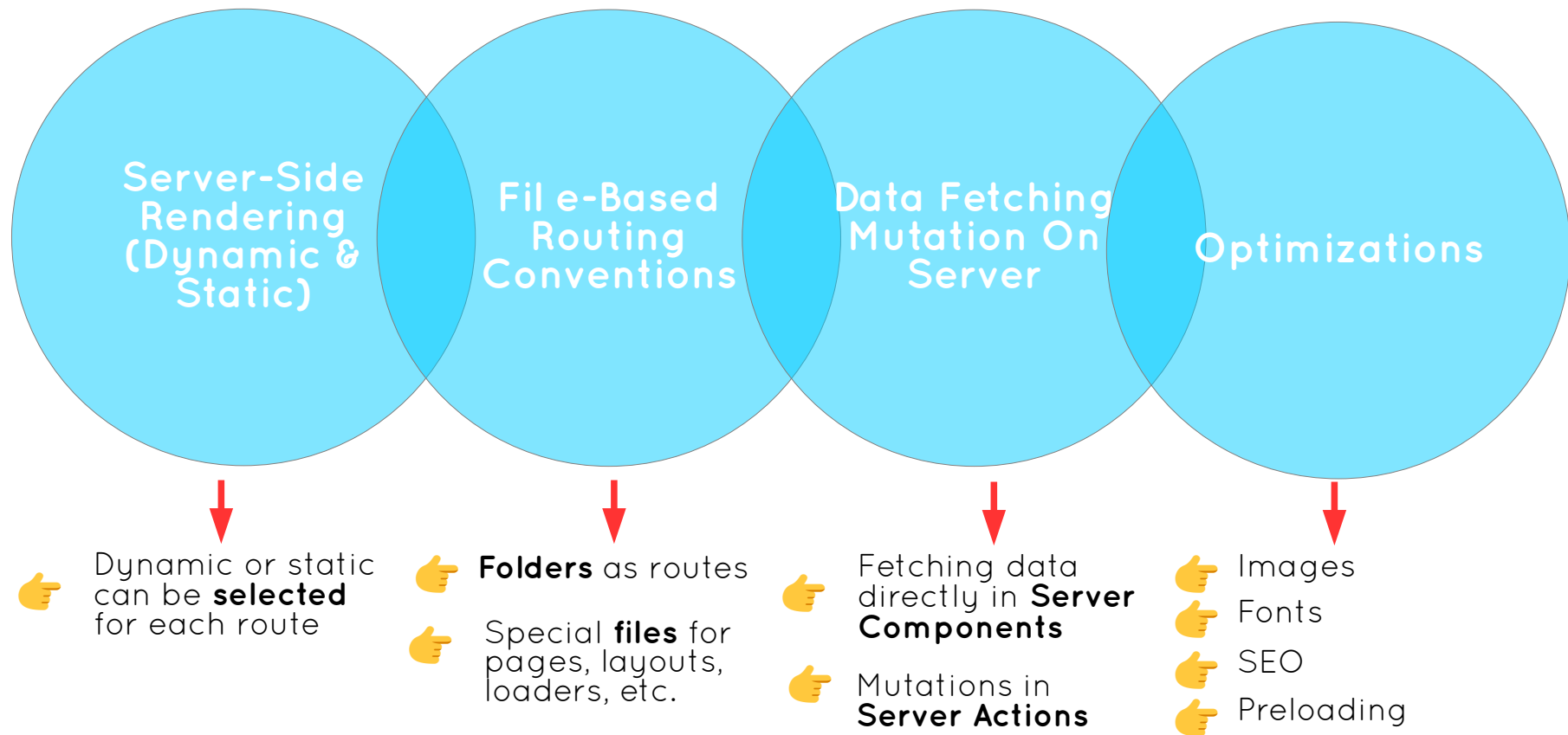
❓ Allows us to build **complex full-stack** web apps and sites

❓ Allows us to use **cutting-edge React** features that need to be integrated into a framework: Suspense, Server Components, Server Actions, streaming, etc.



React's full-stack
architecture vision

The **Next.JS** Key Ingredients



Two Flavors Of Next.JS : “App” & “Pages” Router

MODERN NEXT.JS: “APP” ROUTER

- ❓ Introduced in Next.js 13.4 (2023)
- ❓ Recommend for **new projects**
- ❓ Implements **React’s full-stack architecture**: Server Components, Server Actions, Streaming, etc.
- ❓ Easy fetching with **fetch()** right in components
- ❓ Extremely easy to create layouts, loaders, etc.
- ❓ More advanced routing (parallel routing, etc.)
- ❓ Better **DX** (Developer Experience) and **UX**
- ❓ Caching is very aggressive and confusing
- ❓ Steep learning curve (but it’s React)
- ❓

LEGACY NEXT.JS: “PAGES” ROUTER

- ❓ The first Next.js router since v1 (2016)
- ❓ **Still supported** and updated in the future
- ❓ Overall more simple and easy to learn
- ❓ Simple things like layouts are confusing to implement
- ❓ Data fetching using Next.js-specific APIs such as `getStaticProps` and `getServerSideProps`
- ❓

We’re gonna learn the “app” router from the start. At the end, there is a section on the fundamentals of the “pages” router

A React & Next Ultimate Course

(In Hindi)

Section

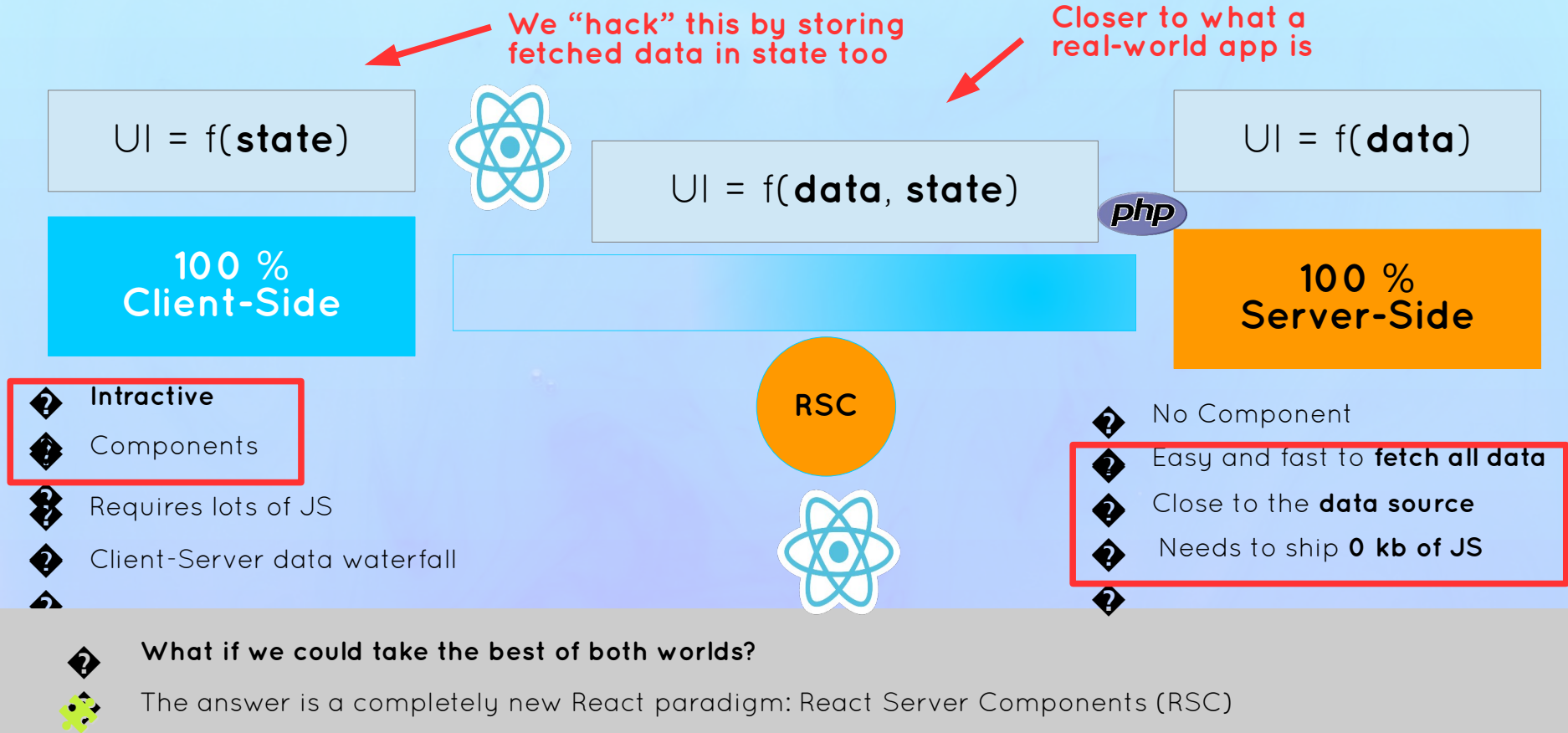
Overview of Next.js With The “App” Router

Lecture

What Are React Server Components?



Why React Server Components?



What Are React Server Components?

React Server Components (RSC)

- ❓ A new **full-stack architecture** for React apps
- ❓ Introduces the server as an integral Level of React component trees: **server components**
- ❓ We write **frontend code** next to **backend code** in a natural way that “feels” like regular React
- ❓ RSC is NOT active by default in new React apps (e.g. Vite apps): it needs to be implemented by a framework like Next.js (“app router”)

Name of the
new PARADIGM

1

Client Component

UI = f(**state**)

- ❓ “Regular” components
- ❓ Created with “**use client**” directive at the top of the module

2

Server Component

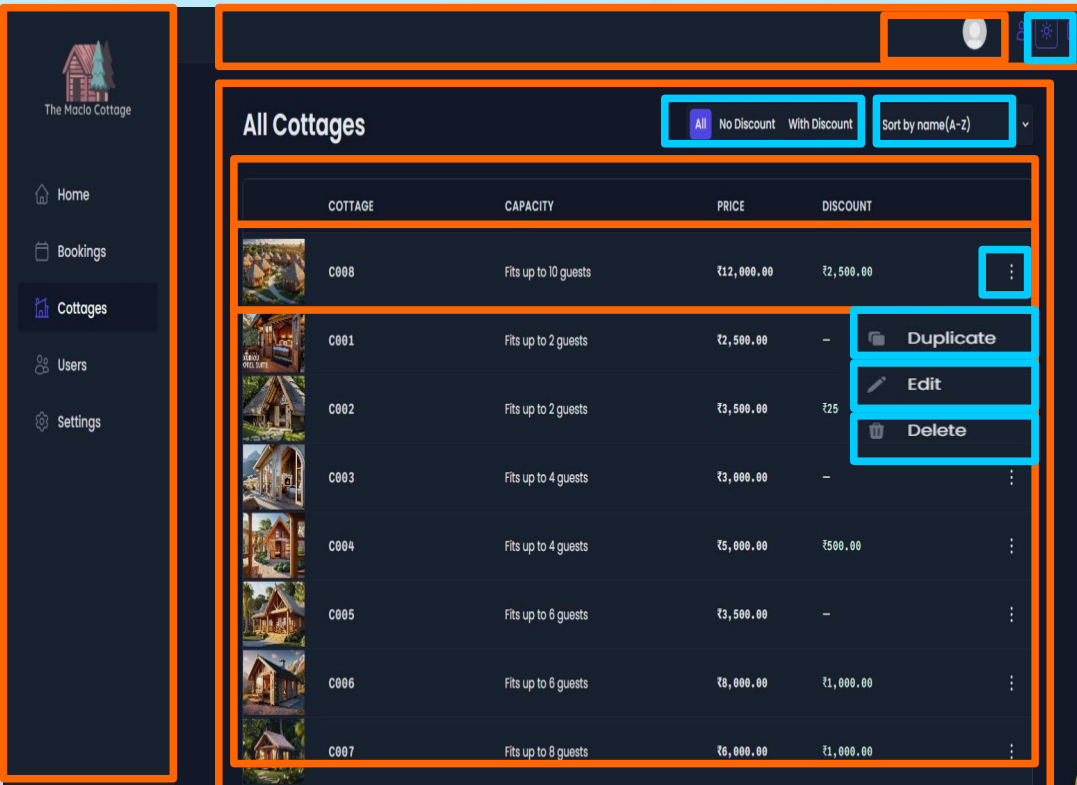
UI = f(**data**)

- ❓ Components that are only rendered on the server
- ❓ Don't make it into the bundle (0 kb)
- ❓ We can build the back-end with React!
- ❓ Default in apps that use the RSC architecture (like Next.js)

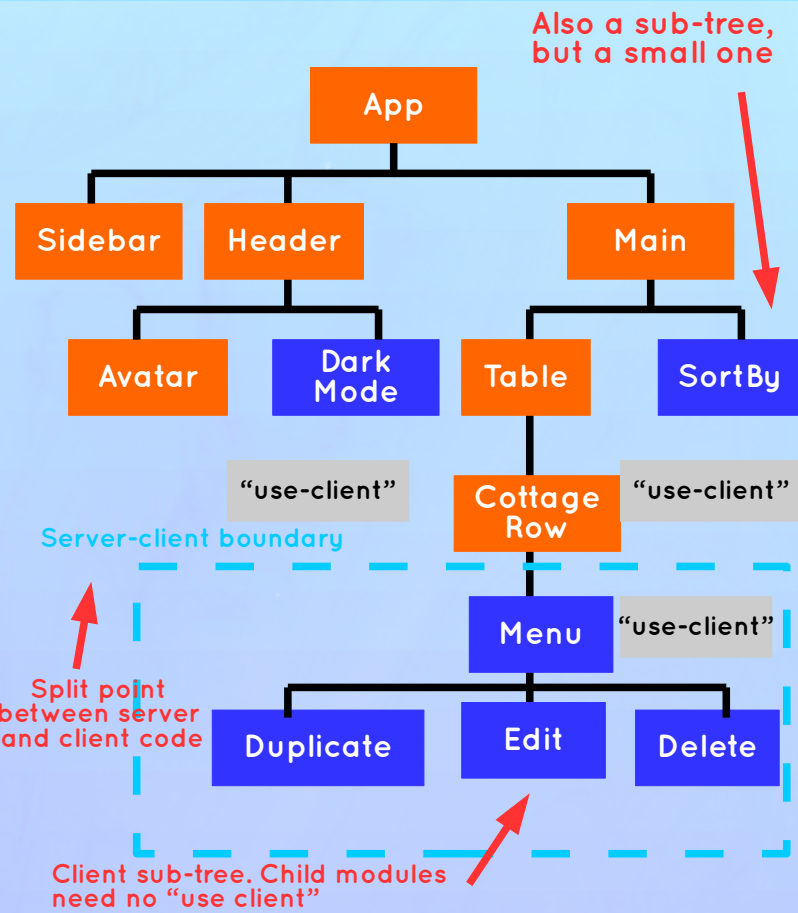
Name of the new
COMPONENT TYPE

An Example + The Server-Client Boundary

Client Components



Server Components

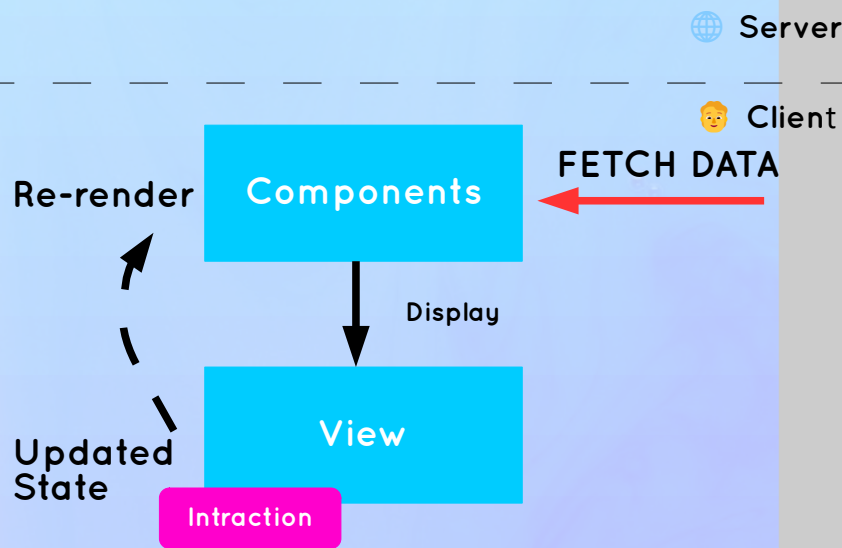


Server Components **VS.** Client Components

	Client Component	“use client”	Server Component	Default
 State/hooks	 Yes		 No	
 Lifting state up	 Yes		 N.A.	
 Props	 Yes		 YES (Must be serializable when passed to client components. No functions or classes)	
 Data fetching	 Also possible, with library		 Preferred. Use <code>async/await</code> in component	
 Can import	Only client components (can't go back in the client-server boundary)		Client and server components	
 Can render	Client components and server components passed as props		Client and server components	
 When re-render?	On state change		On URL change (navigation)	

A Simple New Mental Model

“Traditional” React



React With RSC

FETCH DATA

Server Components

Re-render

PROPS

Re-render

Client
Components

Display

Updated
State

View

Interaction

Navigation

Updated
URL

The **Good & Bad** of RSC Architecture

Good

- ◆ We can compose entire full-stack apps **with React components alone** (+ server actions ◆)
- ◆
- ◆ **One single codebase** for front and back-end
- ◆
- ◆ Server components have **more direct and secure access** to the data source (no API, no exposing API keys, etc.)
- ◆
- ◆ **Eliminate client-server waterfalls** by fetching all the data needed for a page at once before sending it to the client (not each component)
- ◆
- ◆ **“Disappearing code”**: server components ship no JS, so they can import huge libraries “for free”
- ◆

Bad

- ◆ Makes React **more complex**
- ◆
- ◆ More things to **learn and understand**
- ◆
- ◆ Things like **Context API** don't work in server components
- ◆
- ◆ **More decisions** to make: “Should this be a client or a server component?”, “Should I fetch this data on the server or the client?”, etc.
- ◆
- ◆ Sometimes you still need to **build an API** (for example if you also have a mobile app)
- ◆
- ◆ Can only be used within a **framework**
- ◆

Starting To Build “The Maclo Hotel” Website

A React & Next Ultimate Course

(In Hindi)

Section

Starting To Build The “The Maclo Cottages”
Website

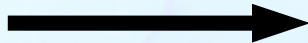
Lecture

Project Planning: “The Maclo Cottages
Website” Customer Website

The Project: The Maclo Cottage



The Maclo Cottage



Remember: “The Maclo Cottages” is a small boutique **hotel** with 8 luxurious wooden cottages



We built their application to manage everything about the hotel: **bookings, cottages and guests**



We also build the **API** using Supabase



Now they need a **customer-facing website** where guests can learn about the hotel, browse all cottages, reserve a cottage, and create and update their profile



Updating data in the internal app should update the website, so we use the same DB and API



We are hired again 😁



API



Internal Hotel
Management
App

Customer-
Facing
Website to
Book Stays



Project Requirements From The Business

👉 Users of the app are potential guests and actual guests

👉 Guests should be able to learn all about The Maclo Cottages Hotel

About

👉 Guests should be able to get information about each cottage and see booked dates

👉 Guests should be able to filter cottages by their maximum guest capacity

Cottages

👉 Guests should be able to reserve a cottage for a certain date range

👉 Reservations are not paid online. Payments will be made at the property upon arrival.
Therefore, new reservations should be set to “unconfirmed” (booked but not yet checked in)

👉 Guests should be able to view all their past and future reservations

👉 Guests should be able to update or delete a reservation

Reservations

👉 Guests need to sign up and log in before they can reserve a cottage and perform any operation

👉 On sign up, each guest should get a profile in the DB

Authentication

👉 Guests should be able to set and update basic data about their profile to make check-in hotel faster

Profile

Features + Pages

Step 2+3

Feature Categories

- 1 About
- 2 Cottages
- 3 Reservations
- 4 Authentication
- 5 Profile

Necessary Pages

- 1 Homepage /
- 2 About Page /about
- 3 Cottages Overview /cottages/
- 4 Cottage Detail /cottage:/cottageId
- 5 Login /login
- 6 Reservation List /account/reservations
- 7 Edit Reservation /account/reservations/edit
- 8 Update Profile /account/profile

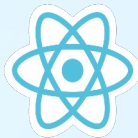
Technology Decision

👉 Framework



The most popular React meta-framework. Handles routing, SSR, data fetching and even remote state management (in a way...), therefore replacing many tools we had to include before

👉 UI State Management



Context API

We might still need global UI state in a Next.js app. For that, we can use the Context API, Redux, or any of the other solutions. In this case the Context API will be enough.

👉 DB/ API



We'll use the data and API we already built in the first "Maclo Cottages" project. If you skipped that project, please go back to the "Supabase" section to set everything up

👉 Styling



Modern way of writing CSS. Extremely easy to integrate into Next.js. Most styles and markup will be pre-written anyway in this project.

Data Fetching Caching And Rendering

A React & Next Ultimate Course

(In Hindi)

Section

Data Fetching, Caching, And Rendering

Lecture

What Is React Suspense?



What is React **Suspense**?

Suspense



Built-in React component that we can use to catch/isolate components (or entire subtrees) that are not ready to be rendered (“suspending”)



What causes a component to be suspending?

1

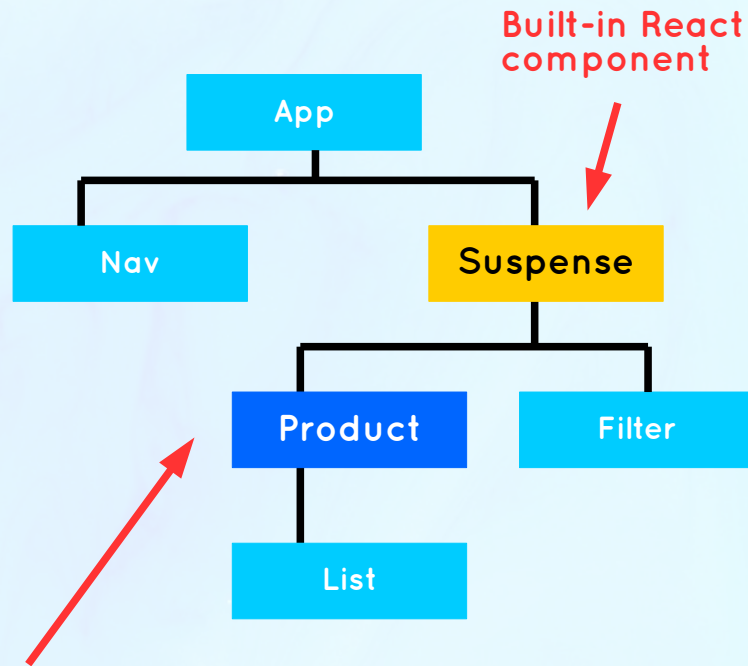
Fetching data (with a supported library)

2

Loading code (with React’s lazy loading)



Native way to support asynchronous operations in a declarative way (no more isLoading states and render logic 🤖)



Component in Next.js
that will fetch data
(will be suspending)

How Does Suspense Works?

While rendering, suspending component is found



Go to closest Suspense parent (“boundary”) and discard already rendered children

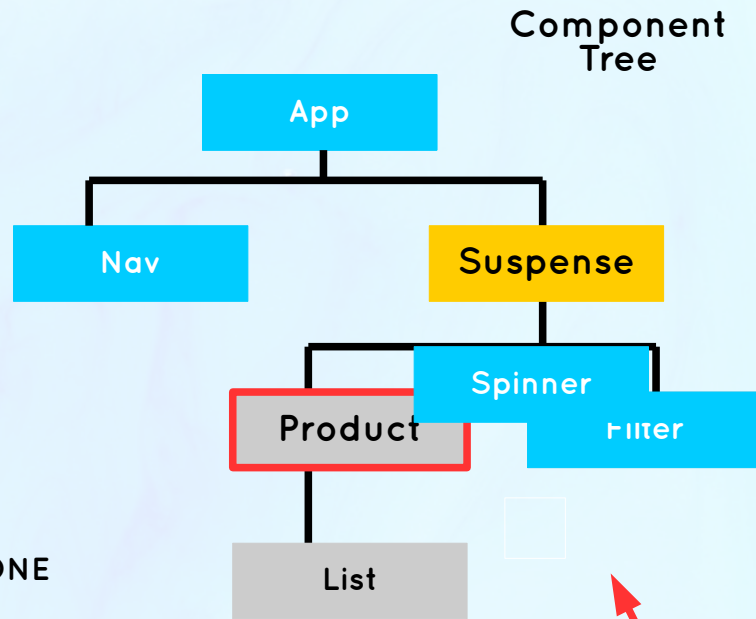


Display fallback component/JSX



Render subtree under Suspense boundary

AFTER ASYNC WORK IS DONE



These haven't been rendered yet



Important: Components do **NOT** automatically suspend just because an async operation is happening inside them. Integrating async operations with Suspense is hard, so we use libraries (React Query, Next.js, etc)

The diagram illustrates the Fiber Tree Representation of a React component tree during a Suspense boundary. The tree structure is as follows:

- App** (Root) is the parent of **Nav** and **Suspense**.
 - Nav** is a child of **App**.
 - Suspense** is a sibling of **Nav**.
- Suspense** is the parent of **Activity**, **Product**, and **List**.
 - Activity** is a child of **Suspense**.
 - Product** is a child of **Suspense**.
 - List** is a child of **Suspense**.
- Activity** is the parent of **Spinner** and **Filter**.
 - Spinner** is a child of **Activity**.
 - Filter** is a child of **Activity**.

Annotations and Flow:

- A yellow diamond labeled **Reconciliation + Diffing** points to the **Suspense** node.
- A red arrow points from the text **Triggers Suspense by throwing promise throw new Promise(...)** to the **Suspense** node.
- A red arrow points from the text **Another Built-in component** to the **Spinner** node.
- A red arrow points from the text **Fallback** to the **Filter** node.
- A red arrow points from the text **State is preserved in subsequent suspending** to the **Product** node.
- A red arrow points from the text **Activity= "hidden"** to the **Spinner** node.
- A red arrow points from the text **How does Suspense actually know that a component is suspending?** to the **Suspense** node.



How does Suspense actually know that a component is suspending?



Fallback will **NOT** be shown again if the Suspense trigger is wrapped in a **transition** (startTransition). In Next.js, that's the case with **page navigation s** We can **reset** the Suspense boundary with a unique **key** prop

A React & Next Ultimate Course

(In Hindi)

Section

Data Fetching, Caching, And Rendering

Lecture

**Different Types Of SSR: Static VS.
Dynamic Rendering**

Server-Side Rendering In Next.js

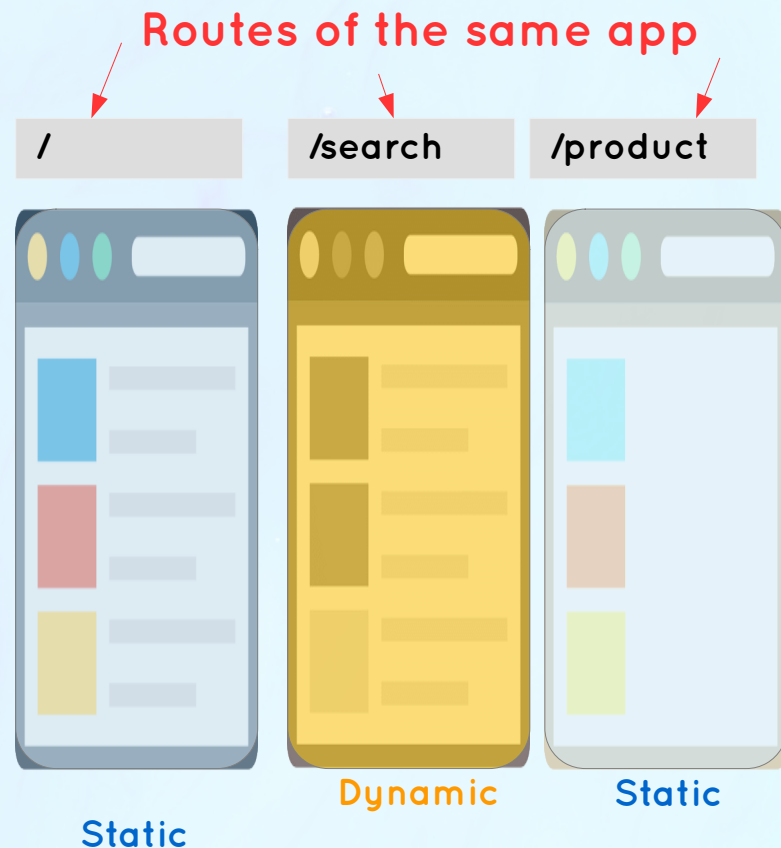
👉 Next.js is a **React** framework, so rendering is done by React, following the rules we learned earlier

👉 **Remember:** Both **Server** and **Client** components are rendered on the server on the initial render

👉 In Next.js, the server-side rendering work is **split by routes**

👉 Each route can be either **static** (also called pre-rendered) or **dynamic**

👉 There is also **Levelial Pre-Rendering** (PPR) which mixes dynamic and static rendering in the same route (more later 🤔🤔)



Static VS. Dynamic Rendering

Static Rendering

👉 HTML is generated at **built time**, or **periodically in the background** by re-fetching data (ISR, more on this later 👉)

👉 Useful when data **doesn't change often** and is **not personalized** to user (e.g. product page)

👉 **Default rendering strategy** in Next.js (even when a page or component fetches data)

👉 When deployed to **Vercel**, each static route is automatically hosted on a CDN

👉 If all routes of an app are static, the entire app can be **exported as a static site** (SSG)

Content Delivery Network

Dynamic Rendering

👉 HTML is generated at **request time** (for each new request reaches the server)

👉 Makes sense if:

- 1 The **data changes frequently** and is **personalized** to the user (e.g. cart)
- 2 Rendering a route requires information that **depends on request** (e.g. search params)

👉 A route **automatically switches to dynamic** rendering in certain conditions (next slide 👉)

👉 When deployed to **Vercel**, each dynamic route becomes a server less function

When Next.js Switches To **Dynamic** Rendering

Usually, developers **don't directly choose** whether a route should be static or dynamic. Next.js will **automatically switch to dynamic** rendering in the following scenarios:

- 1 The route has a **dynamic segment** (page uses params)
- 2 **searchParams** are used in the page component
`/product?quantity=12`
- 3 **headers()** or **cookies()** are used in any of the route's server components
- 4 An **uncached data request** is made in any of the route's server components

This is necessary because any of these values **can not be known** by Next.js at built time

We can also **force** Next.js to render a route dynamically:

- `export const dynamic = 'force-dynamic';` from page.js
- `export const revalidate = 0;` from page.js
- `{ cache: 'no-store' }` added to a **fetch** request in any of the route's server components
- `noStore()` in any of the route's server components

These influence
caching.
More on this later

4

Some Terminolgy You Might Need

- 👉 **Content Delivery Network (CDN):** A network of servers located around the globe that cache and deliver a website's **static content** (HTML, CSS, JS, images) from as close as possible to each user.
- 👉 **Serverless computing:** With the serverless computing model, we can run application code, usually back end code, without managing the server ourselves. Instead, we can just run single functions on a cloud provider: **serverless functions**. The server is initialized and active only for the duration the serverless function is running, unlike a traditional Node.js app where the server is constantly running. Remember: each dynamic route becomes a serverless function.
- 👉 **The “edge”:** “As close as possible to the user”. A CDN is Level of an “edge” network, but there is also **serverless “edge” computing**. This is serverless computing that does not happen on a central server, but on a network that's distributed around the globe, as close as possible to the user (like a CDN but for running code). Important: we can select certain routes to run on the edge when deployed to Vercel.
- 👉 **Incremental Static Regeneration (ISR):** A Next.js feature that allows developers to update the content of a **static page**, in the background, even after the website has already been built and deployed. This happens by **re-fetching the data** of a component or entire route after a certain interval. More on this later 💡

A React & Next Ultimate Course

(In Hindi)

Section

Data Fetching, Caching, And Rendering

Lecture

Partial Pre - Rendering



Levelial Pre-Rendering(PPR) A Whole New Way Of Rendering

👉 **Idea/problem:** Most pages don't need to be 100% static or 100% dynamic

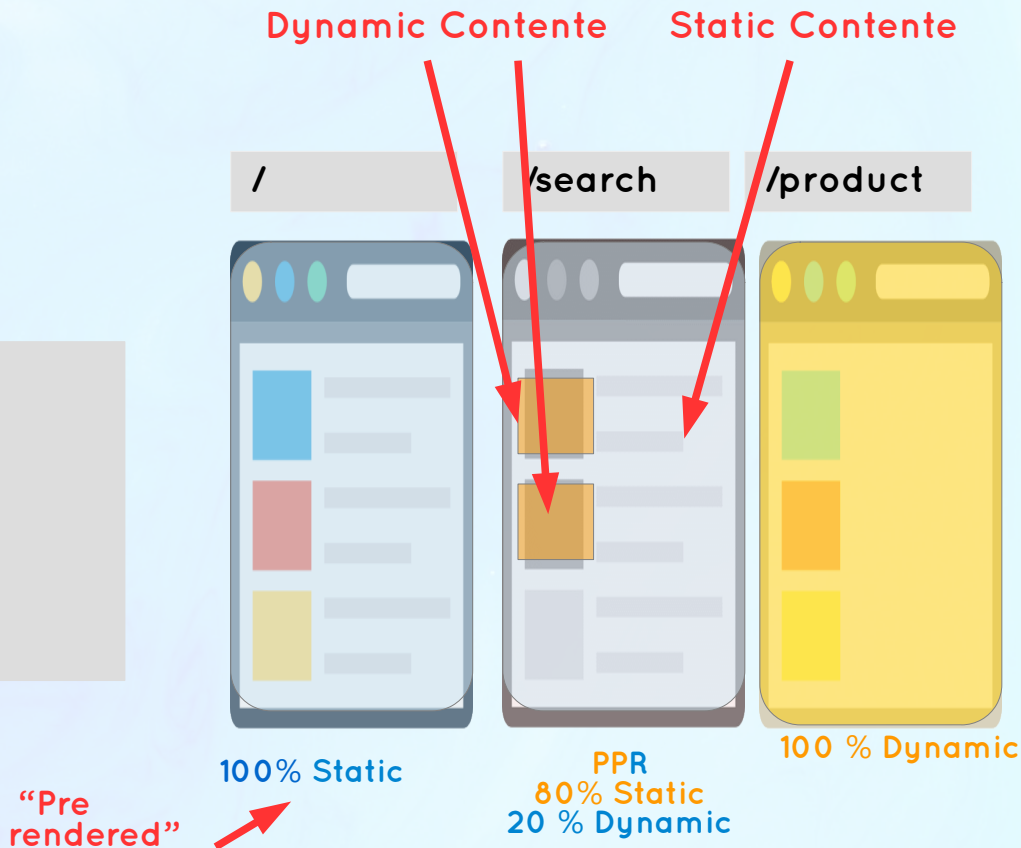
👉 **Solution:** Levelial Pre-Rendering

👉 New rendering strategy that combines **static** and **dynamic** rendering in the same route

1 A **static** (pre-rendered) shell is served immediately from a CDN, leaving holes for dynamic content

2 The slower **dynamic** content is streamed in as it's rendered on the server

👉 **Result:** Even faster pages that can mostly be delivered from the edge (CDN) even when there are small dynamic Levels.



How To **Use** Levelial Pre-Rendering



As of Next.js 15, PPR is **highly experimental** and should not be used in production

- 👉 PPR needs to be **turned on** in config file
- 👉 By default, as much as possible of any route will be **statically rendered**, creating a **static shell**
- 👉 Dynamic Levels (components) should be placed inside **Suspense boundaries**
- 👉 There are no new APIs to learn 💡
- 👉 These boundaries tell Next.js that **anything within the boundary is dynamic**
- 👉 The boundary prevents the dynamic Level (e.g. reading a header or making a non-cached fetch request) from spreading onto the entire route.
- 👉 We provide a **static fallback** to be shown while the dynamic Level is rendering
- 👉 Dynamic components or sub-trees are **inserted** into the static shell as they become available

A React & Next Ultimate Course

(In Hindi)

Section

Data Fetching, Caching, And Rendering

Lecture

How Next.JS Caches Data



The Caching In Next.JS

Next.JS Caching

- 👉 **Caching:** Storing fetched or computed data in a temporary location for future access, instead of having to re-fetch or re-compute the data every time it's needed
- 👉 Next.js caches **very aggressively:** everything that is possible to cache, is cached
- 👉 Next.js provides APIs for **cache revalidation:** removing data from the cache and updating it with **fresh data** (re-fetched or re-computed)
- 👉 Makes Next.js apps **more performant** and **saves costs** (computing and data access)
- 👉 **Caching always ON by default:** strange and unexpected behavior in some situations. Some caches can't be turned off ❓
- 👉 **Very confusing:** Many different Next.js APIs affect and control caching

The Caching Mechanisms

	 REQUEST MEMOIZATION	 Data Cache	 Full Route Cache	 Router Cache
Where	Server	Server	Server	Client
What Data?	Data fetched with similar GET requests (same url and options in fetch function)	Data fetched in a route or a single fetch request	Entire static pages (HTML and RSC payload)	Pre-fetched and visited pages: static and dynamic
How long?	One page request (one render, one user)	Indefinitely, even across de-deploys (can revalidate or opt out)	Until the “Data cache” is invalidated (or app is re-deployed)	30 sec dynamic / 5 min Static (throughout one user session)
Enables	No need to fetch at the top of tree: the same fetch in multiple components only makes one request	Data for static pages + ISR when revalidated	Static pages	SPA-like navigation (instant navigation and no full reloads)
Only in components (not route handlers or server actions) 		This is the behavior in production mode. Caching doesn't work in development		

The Caching Mechanisms



REQUEST
MEMOIZATION



Data Cache



Full Route
Cache



Router
Cache

How to revalidate? **NA**

Revalidating **Data Cache** also
revalidates **Full Route Cache** →



Time-based (automatic) for all data on page: `export const revalidate = <time>;`
(page.js)



Time-based (automatic) for one data request: `fetch('...', { next: { revalidate: <time> } })`



On-demand (manual):
`revalidatePath` or `revalidateTag`



`revalidatePath` or
`revalidateTag` in **SA**



`router.refresh`



`cookies.set` or
`cookies.delete` in **SA**

How to opt out? `AbortController`

These forces page to become
dynamic, which also opts out
of the **Full Route Cache** →



Entire page:
`export const revalidate = 0;` (page.js)



Entire page:
`export const dynamic = 'force-dynamic';` (page.js)



Individual request:
`fetch('...', { cache: 'no-store' })`



Individual server component: `noStore()`

Not possible

Can be very
problematic

Client & Server Interactions

A React & Next Ultimate Course

(In Hindi)

Section

Client And Server Interactions

Lecture

**Blurring The Boundary Between Server
And Client**

The Server-Client **Boundary**: Front-End VS. Back-End



Back-End [API]



MUTATIONS [POST, PUT ...]

JSON DATA [GET]

Front-End



- 👉 **Very clear** server-client boundary
- 👉 Communication happens via an **API**
- 👉 Once JSON arrives from the back-end, the front-end takes over

Next.JS With RSC + SA



Client component

DATA: RENDER DIRECTLY

Back-End

Front-End

Back-End

Server component

Back-End

Back-End

Front-End

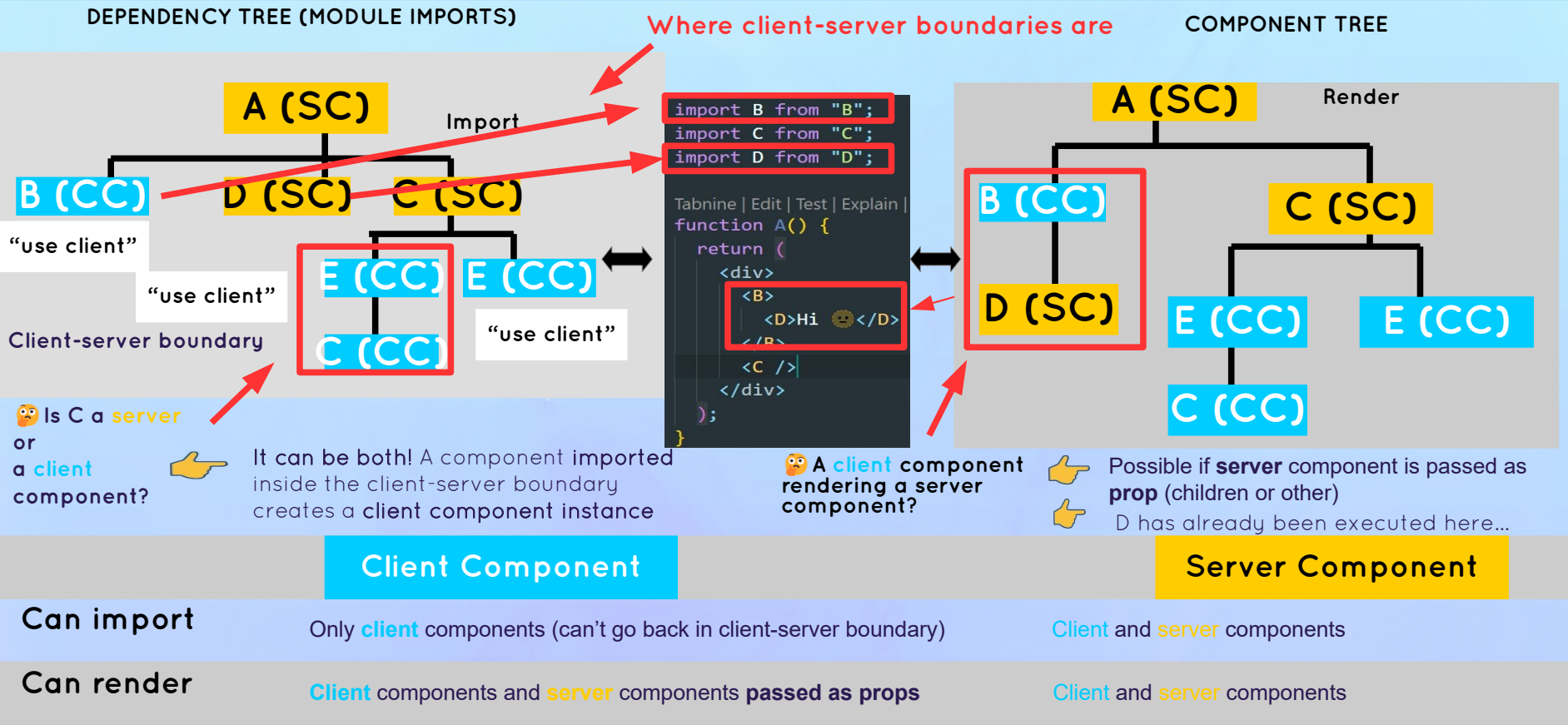
MUTATIONS: SERVER ACTIONS (SA)

DATA: SEND
AS PROPS

Front-End

- 👉 **No clear separation** between front-end and back-end anymore
- 👉 **“Knitting”**: pieces of server and client code interweave (composability)
- 👉 Allow us to build true **full-stack applications** in just one codebase
- 👉 **No need** for an intermediary API in many times

Importing VS. Rendering



Authentication With NextAuth (Auth.JS)

A React & Next Ultimate Course

(In Hindi)

Section

Authentication With NextAuth (Auth.JS)

Lecture

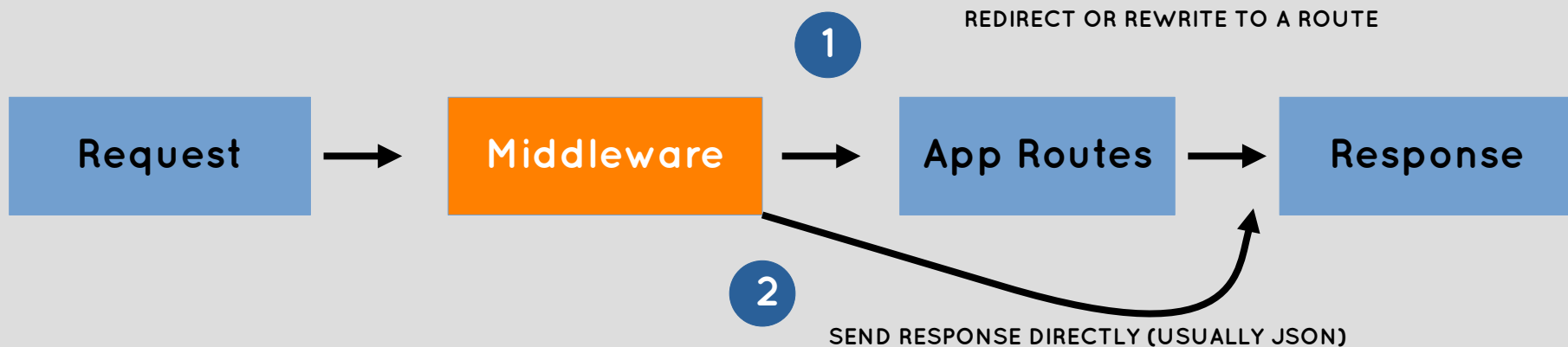
What Is Middle ware In Next.JS



How **Middleware** Works In Next.JS



How **Middleware** Works In Next.JS



MIDDLEWARE DETAILS:



By default, middleware runs **before every route** in a project, but we can specify which paths using a **matcher**



Analogy: chunk of code that's in every page.js component



Only **one** middleware function: needs to be exported from **middleware.js** (or .ts) in the project **root folder**



Middleware needs to **produce a response:**

1

Or

2

USE CASES:



Read and set cookies and headers



Authentication and authorization



Server-side analytics



Redirect based on geolocation



A/B testing

Mutation With Server Actions + Modern React Hooks

A React & Next Ultimate Course

(In Hindi)

Section

Mutation With Server Actions + Modern React
Hooks

Lecture

What Are The Server Actions?

What are Server Actions?

Server Actions



The missing piece in the RSC architecture that enables **interactive full-stack applications**



Async functions that run **exclusively on the server**, allowing us to perform **data mutations**

 **INTERACTIVE FULL-STACK APPLICATIONS**

That is able to handle user input



RSC ARCHITECTURE

1

Data Fetching

2

Mutation

Server
Components



Server
Actions



How To Create Server Actions?

Server Actions



The missing piece in the RSC architecture that enables **interactive full-stack applications**



Async functions that run exclusively on the server, allowing us to perform **data mutations**



Created with the **“use server”** directive at the top of the function or an entire module

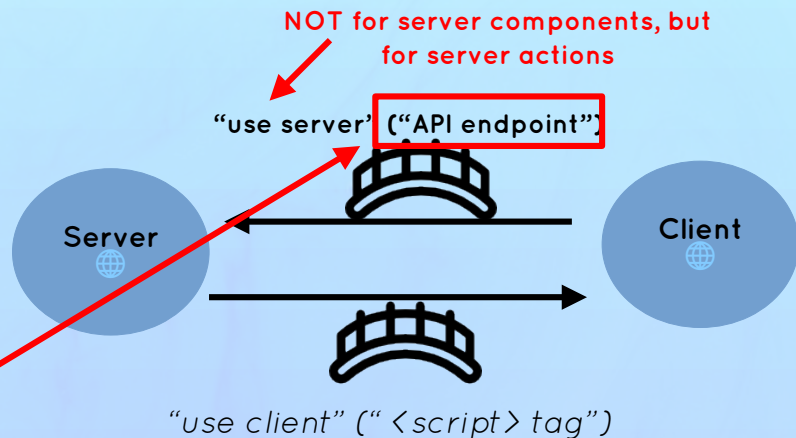


Behind the scenes: Next.js creates **API endpoint** (with URL) for each server action. Whenever a server action is called, a **POST request** is made to its URL (the function itself **never** reaches client)



Unlike server components, server actions actually require a **running web server**

**We don't have to manually create APIs
Or react handlers to mutate data**



SERVER ACTIONS CAN BE DEFINED AT THE TOP OF:

1

An async function in a **server** component. Can be used in component or passed to a **client** component (unlike functions)

2

Stand-alone file: exported functions become server actions that can be imported into **any** component **[recommended]**

How To **Use** Server Actions?

Server Actions



The missing piece in the RSC architecture that enables **interactive full-stack applications**



Async functions that run exclusively on the server, allowing us to perform **data mutations**



Created with the **“use server”** directive at the top of the function or an entire module



Behind the scenes: Next.js creates **API endpoint** (with **URL**) for each server action. Whenever a server action is called, a **POST request** is made to its URL (the function itself **never** reaches client)



Unlike server components, server actions actually require a **running web server**

SERVER ACTIONS CAN BE CALLED FROM:

1

action attribute in a `<form>` element (in **server** and **client** components)

2

Event handlers (only client components)vent handlers (only **client** components)

3

useEffect (only **client** components)



**Code is running on the back-end,
so we need to assume inputs are unsafe**

IN SERVER ACTIONS, WE CAN:



Perform **data mutations** (create, update, delete)



Update the UI with new data: Revalidate cache with `revalidatePath` and `revalidateTag`



Work with cookies



Many more ...